

嵌入式系统

第四讲 ARM硬件结构（三）

教师：郭玉臣

Mail:yuchenguo@tongji.edu.cn



LPC2000系列ARM硬件结构

- 1.LPC2000系列简介
- 2.引脚描述
- 3.存储器寻址
- 4.系统控制模块
- 5.存储器加速模块(MAM)
- 6.外部存储器控制器(EMC)
- 7.引脚连接模块
- 8. GPIO
- 9. 向量中断控制器
- 10.外部中断输入
- 11.定时器0和定时器1
- 12. SPI接口
- 13. I²C接口
- 14. UART(0、1)
- 15. A/D转换器
- 16. 看门狗
- 17. 脉宽调制器(PWM)
- 18. 实时时钟



4.10 外部中断输入

概述

LPC2000系列ARM具有4路外部中断，可以设置为2种类型：

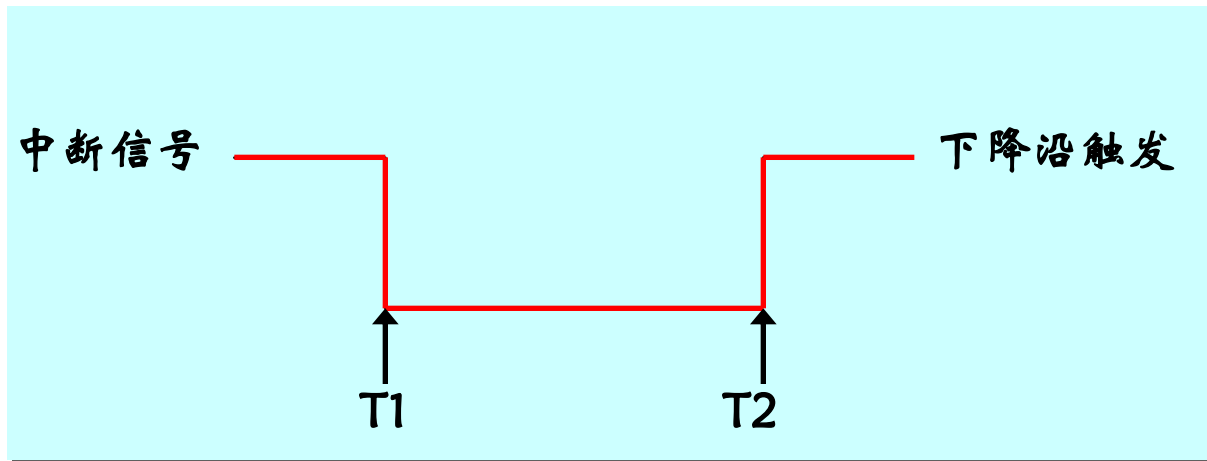
- 边沿触发：
 - 上升沿触发
 - 下降沿触发
- 电平触发：
 - 高电平触发
 - 低电平触发

4.10 外部中断输入

• 边沿触发中断

下降沿触发类型中断的请求和清除时序。

② T2时刻，CPU执行完下降沿控制器的中断服务程序后，中断请求信号回复到高电平。

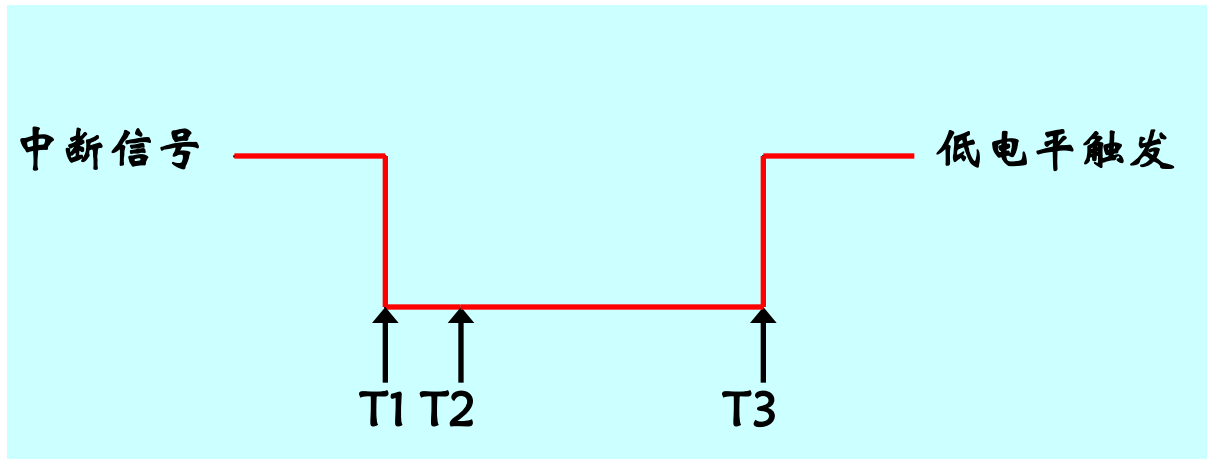


4.10 外部中断输入

• 电平触发中断

低电平触发类型中断的请求和清除时序。

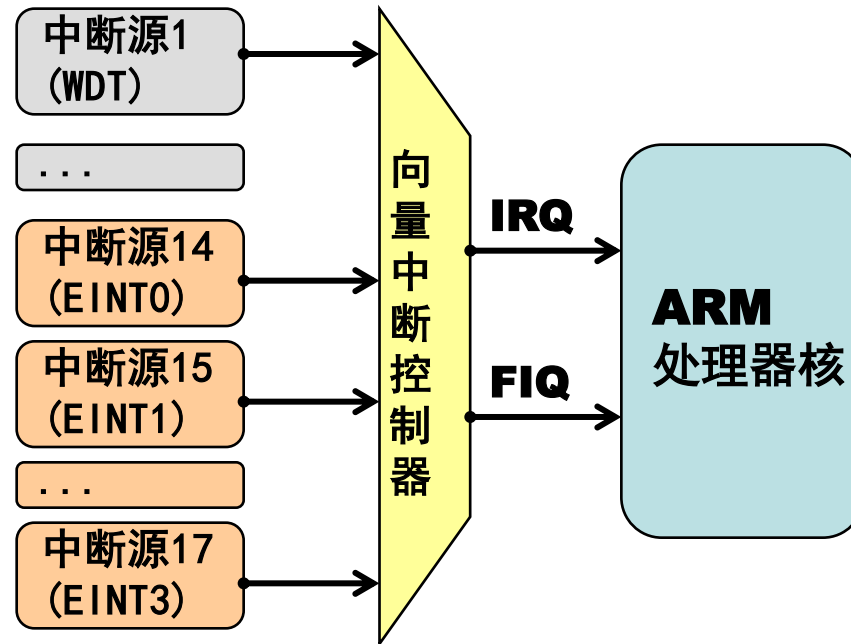
在T1时刻，CPU检测到中断信号由高电平变为低电平。在T2时刻，CPU响应中断，将程序转向中断服务程序。在T3时刻，CPU清除中断，将中断信号由高电平恢复为低电平。



4.10 外部中断输入

- 外部中断源

LPC2000系列微控制器几乎所有的外设部件都可以产生中断。其中外部中断含有4个独立的中断输入。



系统控制模块功能汇总

• 寄存器汇总

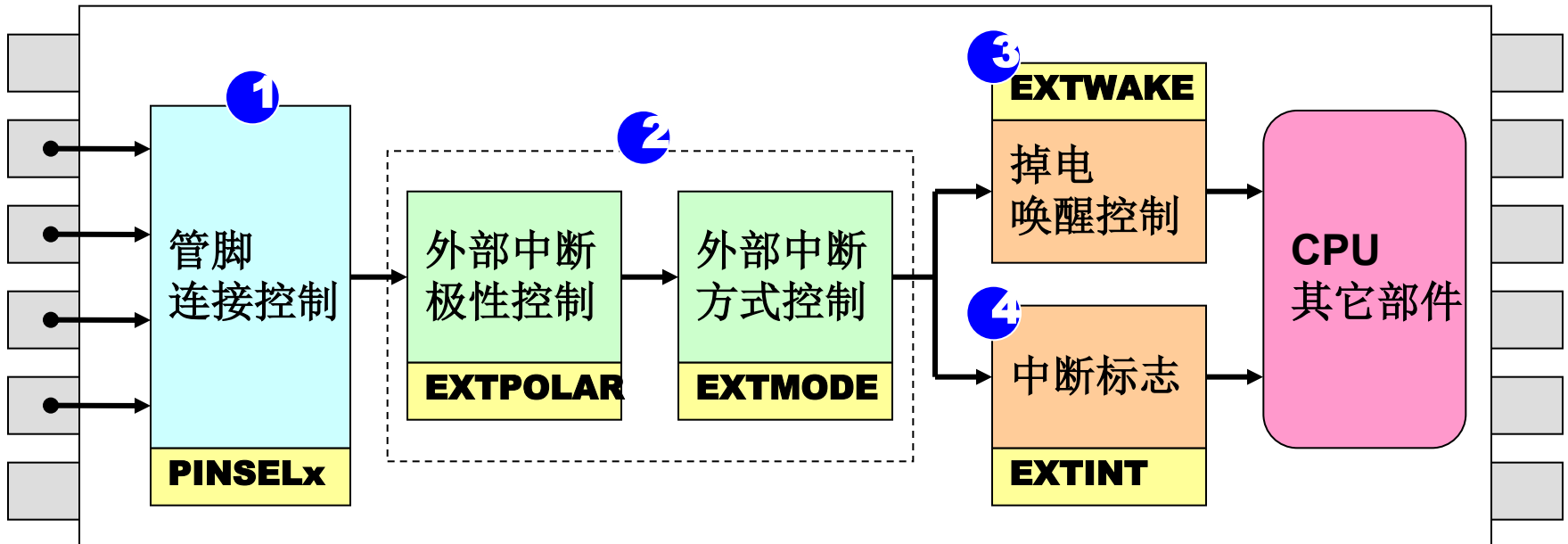
名称	描述	访问	复位值*	地址
EXTINT	外部中断标志寄存器	R/W	0	0xE01FC140
EXTWAKE	外部中断唤醒寄存器	R/W	0	0xE01FC144
EXTMODE	外部中断方式寄存器	R/W	0	0xE01FC148
EXTPOLAR	外部中断极性寄存器	R/W	0	0xE01FC14C

*: 复位值仅指已使用位中保存的数据，不包括保留位的内容。

4.10 外部中断输入

寄存器汇总

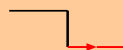
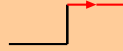
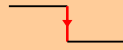
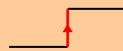
可以通过设置EXTINT寄存器来选择④在睡眠外部中断可以触发。②可以设置EXTMODE寄存器选择③在睡眠外部中断可以触发。



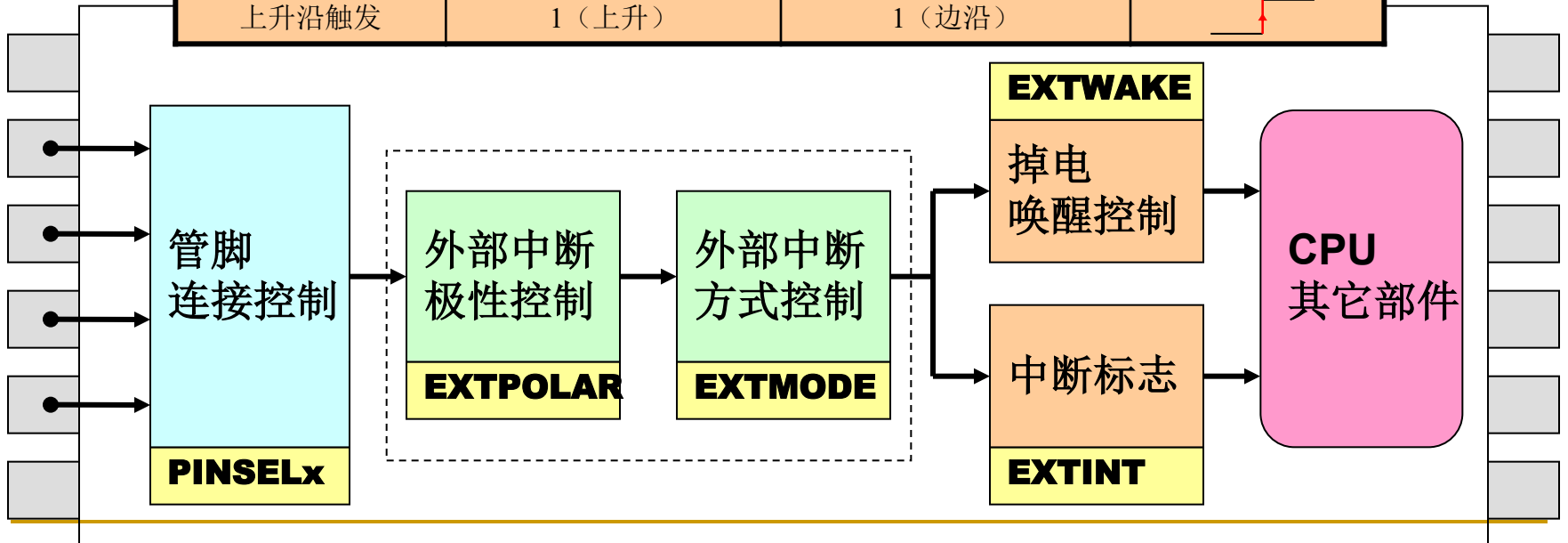
寄存器汇总

外部中断极性和方式寄存器 (EXTINTCR) :

注意：
寄存器
EXTINTCR

设置说明	相应位设置值		信号波形
	极性控制寄存器 (EXTPOLAR)	方式控制寄存器 (EXTMODE)	
低电平触发	0 (低)	0 (电平)	
高电平触发	1 (高)	0 (电平)	
下降沿触发	0 (下降)	1 (边沿)	
上升沿触发	1 (上升)	1 (边沿)	

波形，
EXTINTCR



4.10 外部中断输入

外部中断引脚设置

LPC2000系列芯片允许许多外部引脚连接到系统的外围设备。在数引脚引脚中，**注意**点：某些引脚的速率处理器。外部中断源，除了引脚连接到系统，还配置VLC模式，否则外部中断不能反映在EXTINT寄存器。

外部中断名称

外部中断0 (EXT0) 通过外部中断功能，再进入掉电模式。

边沿触发方式：只使用GPIO端口号最低的那个引脚，并且与极性设置无关。	P0.15	RI
-------------------------------------	-------	----

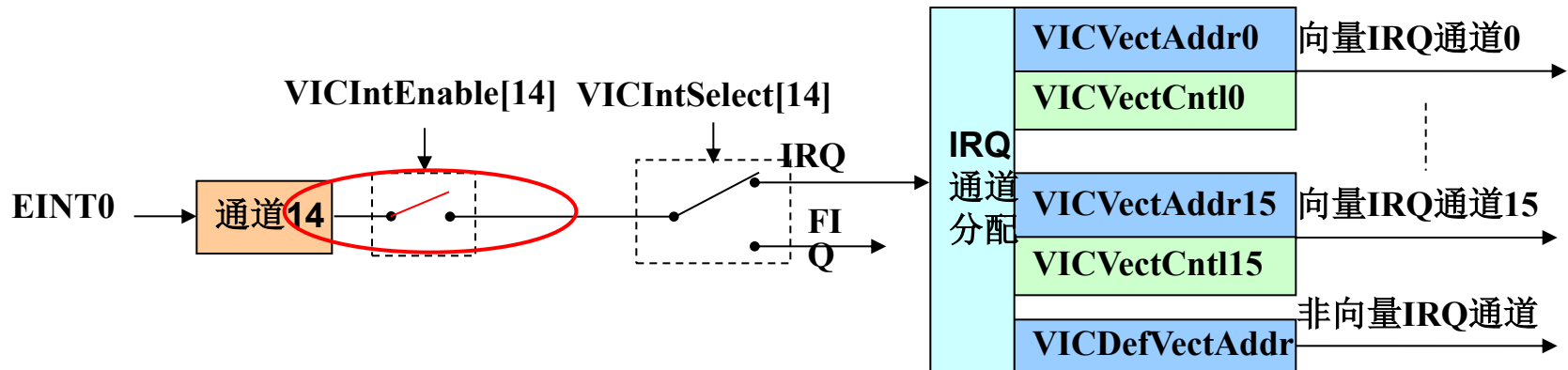
外部中断名称	引脚名	该引脚其它功能
外部中断0 (EINT0)	P0.1	RXD0
外部中断1 (EINT1)	P0.14	DCD
外部中断2 (EINT2)	P0.15	RI
外部中断3 (EINT3)	P0.9	RXD1
	P0.20	SSEL1
	P0.30	

4.10 外部中断输入

外部中断与VIC的关系

外部中断0位于VIC通道14，中断使能寄存器VICIntEnable[14]用来控制通道14的使能：

当VICIntEnable[14] = 0时，通道14中断禁止



注意：这里仅以EINT0为例来进行讲解，EINT1~EINT3与之类似，此处不再重复。

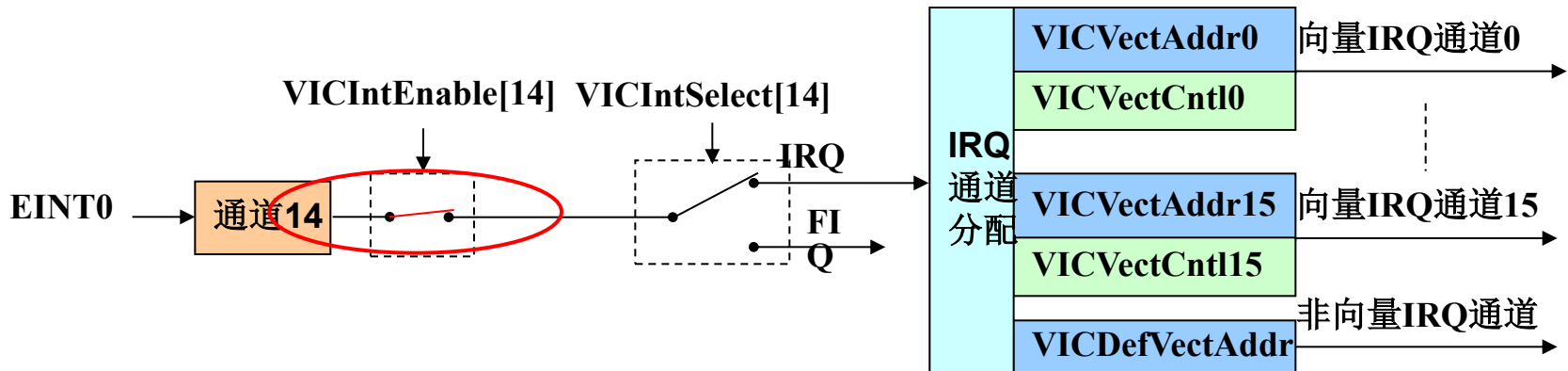
4.10 外部中断输入

• 外部中断与VIC的关系

外部中断0位于VIC通道14，中断使能寄存器VICIntEnable[14]用来控制通道14的使能：

当VICIntEnable[14] = 0时，通道14中断禁止

当VICIntEnable[14] = 1时，通道14中断使能



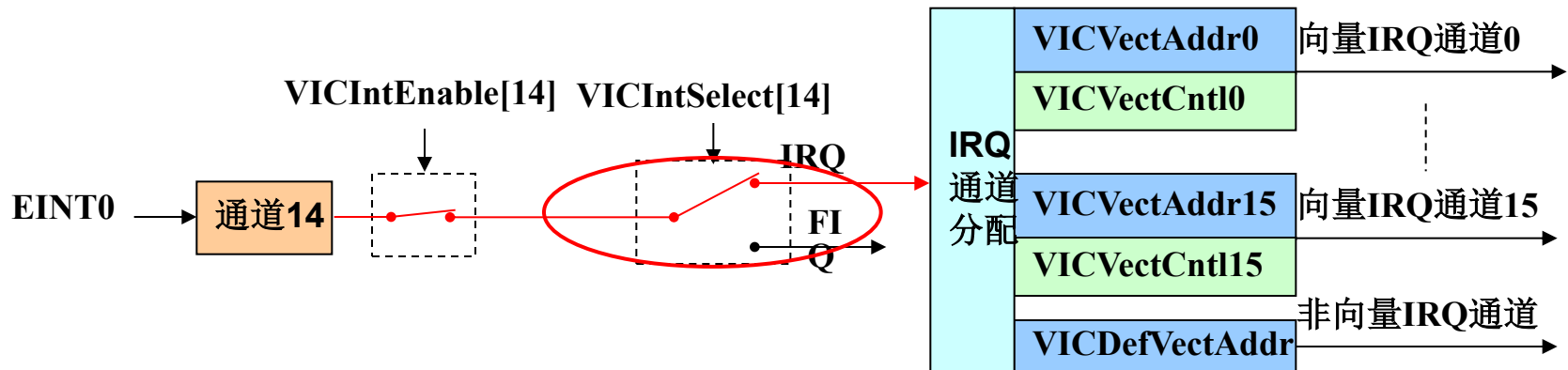
注意：这里仅以EINT0为例来进行讲解，EINT1~EINT3与之类似，此处不再重复。

4.10 外部中断输入

外部中断与VIC的关系

外部中断0位于VIC通道14，中断选择寄存器VICIntSelect[14]用来选择通道14的中断类型：

当VICIntSelect[14] = 0时，通道14分配为IRQ中断



注意：这里仅以EINT0为例来进行讲解，EINT1~EINT3与之类似，此处不再重复。

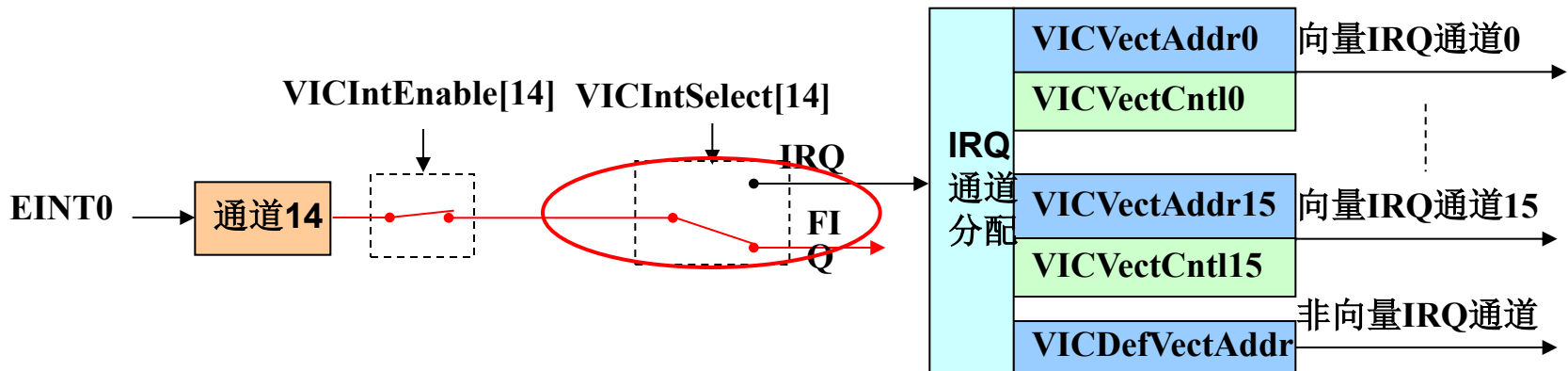
4.10 外部中断输入

外部中断与VIC的关系

外部中断0位于VIC通道14，中断选择寄存器VICIntSelect[14]用来选择通道14的中断类型：

当VICIntSelect[14] = 0时，通道14分配为IRQ中断

当VICIntSelect[14] = 1时，通道14分配为FIQ中断



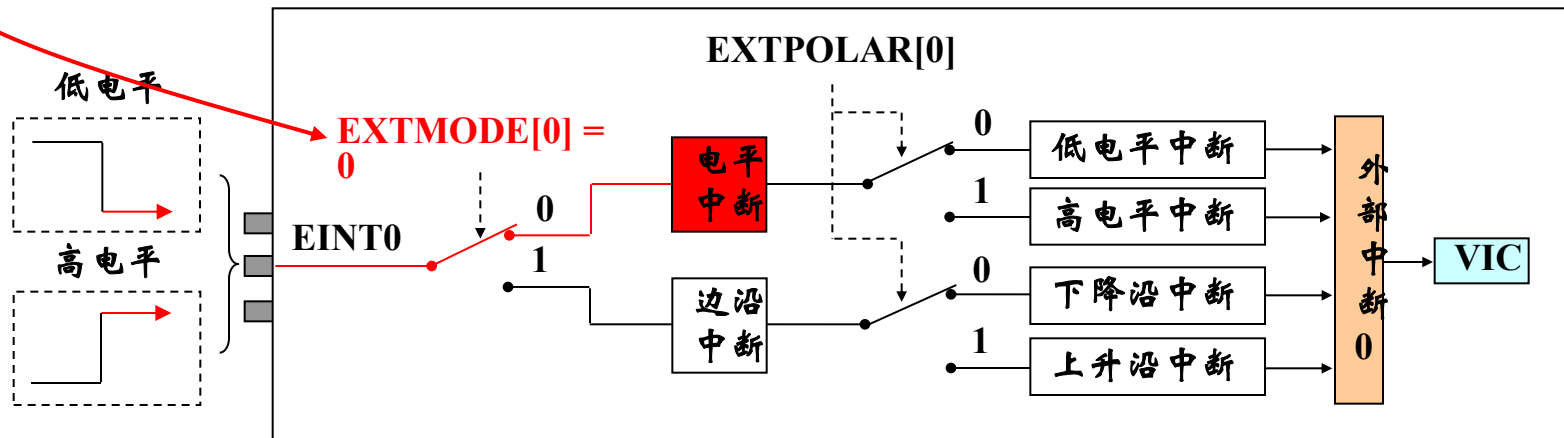
注意：这里仅以EINT0为例来进行讲解，EINT1~EINT3与之类似，此处不再重复。

4.10 外部中断输入

外部中断的设置

LPC2000系列ARM的电平中断可以设置为**电平中断**和**边沿中断**。

- 当EXTMODE[0] = 0时，外部中断0设置为电平触发。
- 当EXTMODE[0] = 1时，外部中断0设置为边沿触发。



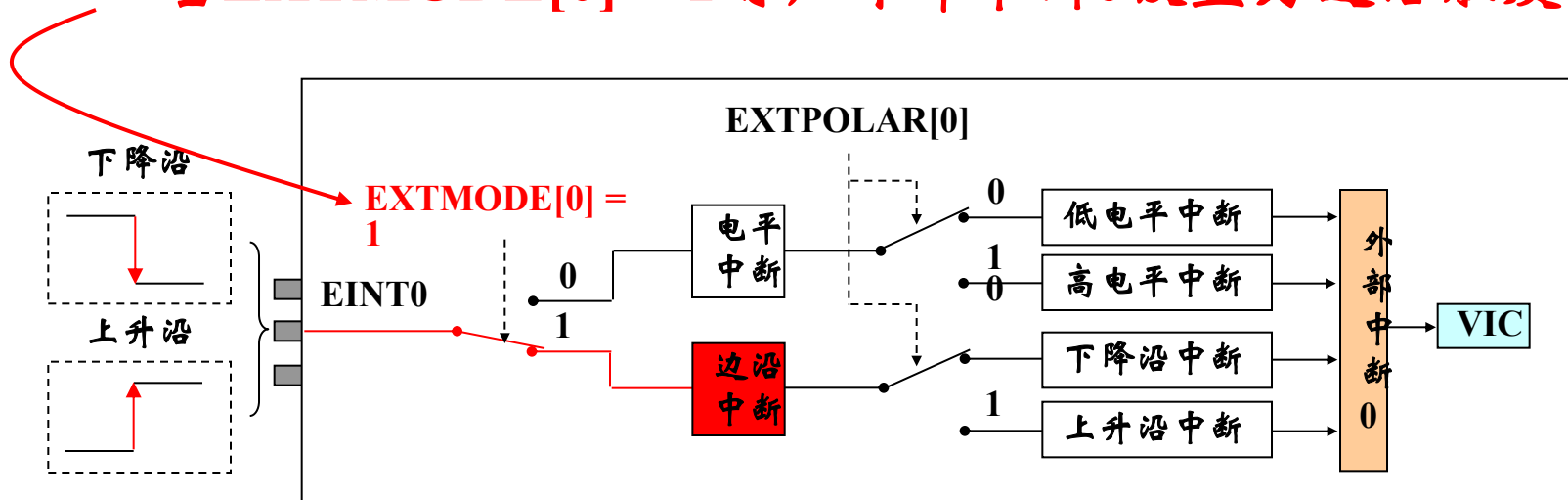
注意：这里仅以EINT0为例来进行讲解，EINT1~EINT3与之类似，此处不再重复。

4.10 外部中断输入

外部中断的设置

LPC2000系列ARM的电平中断可以设置为**电平中断**和**边沿中断**。

- 当EXTMODE[0] = 0时，外部中断0设置为电平触发。
- 当EXTMODE[0] = 1时，外部中断0设置为边沿触发。



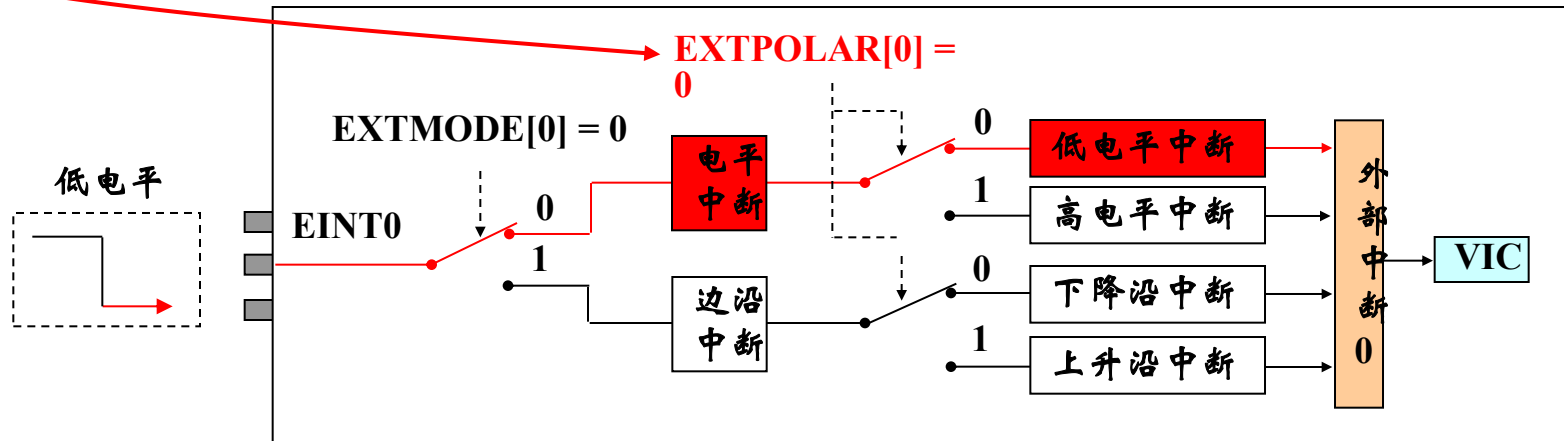
注意：这里仅以EINT0为例来进行讲解，EINT1~EINT3与之类似，此处不再重复。

4.10 外部中断输入

• 电平中断设置

LPC2000系列ARM的电平中断可以设置为**高电平**触发和**低电平**触发。

- 当 $\text{EXTPOLAR}[0] = 0$ 时，外部中断0设置为低电平触发。
- 当 $\text{EXTPOLAR}[0] = 1$ 时，外部中断0设置为高电平触发。



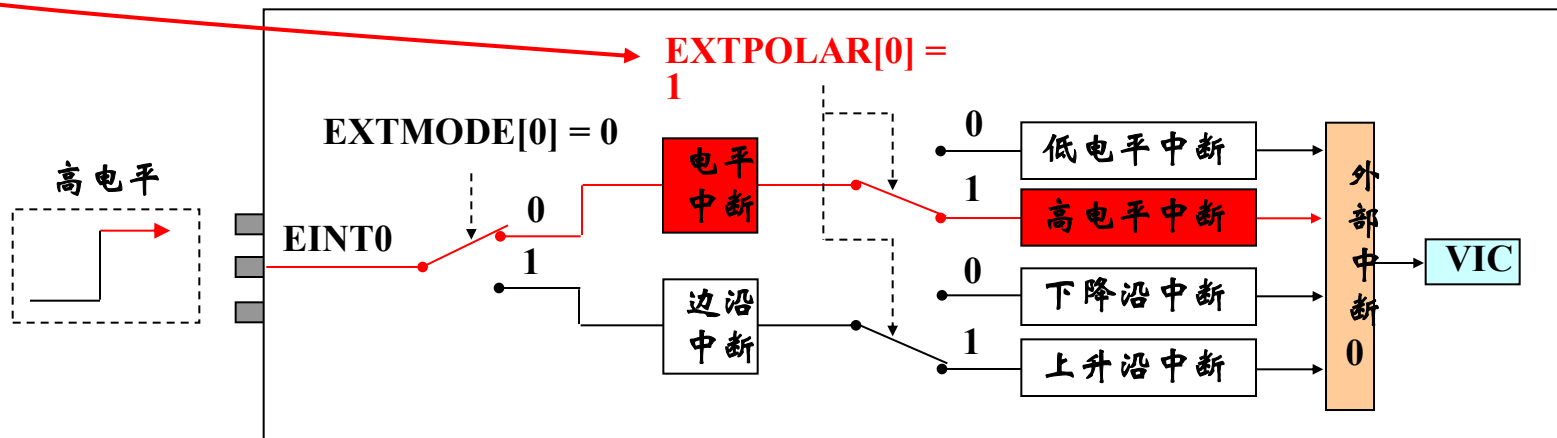
注意：这里仅以EINT0为例来进行讲解，EINT1~EINT3与之类似，此处不再重复。

4.10 外部中断输入

• 电平中断设置

LPC2000系列ARM的电平中断可以设置为**高电平**触发和**低电平**触发。

- 当 $EXTPOLAR[0] = 0$ 时，外部中断0设置为低电平触发。
- 当 $EXTPOLAR[0] = 1$ 时，外部中断0设置为高电平触发。



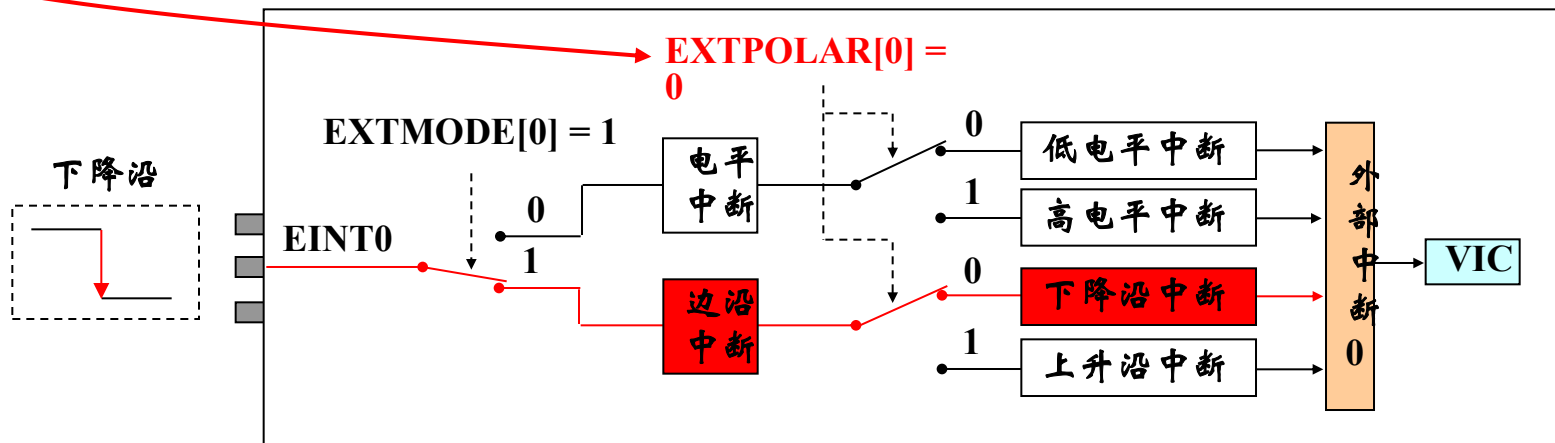
注意：这里仅以EINT0为例来进行讲解，EINT1~EINT3与之类似，此处不再重复。

4.10 外部中断输入

边沿中断设置

LPC2000系列ARM的边沿中断可以设置为**上升沿**触发和**下降沿**触发。

- 当 $EXTPOLAR[0] = 0$ 时，外部中断0设置为下降沿触发。
- 当 $EXTPOLAR[0] = 1$ 时，外部中断0设置为上升沿触发。



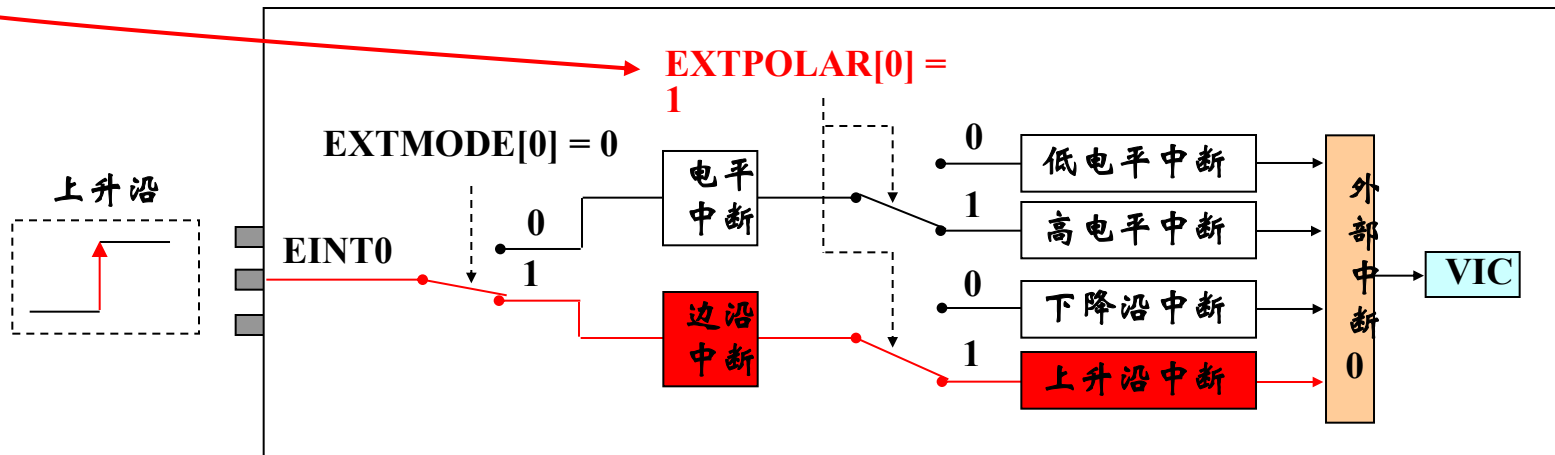
注意：这里仅以EINT0为例来进行讲解，EINT1~EINT3与之类似，此处不再重复。

4.10 外部中断输入

边沿中断设置

LPC2000系列ARM的边沿中断可以设置为**上升沿**触发和**下降沿**触发。

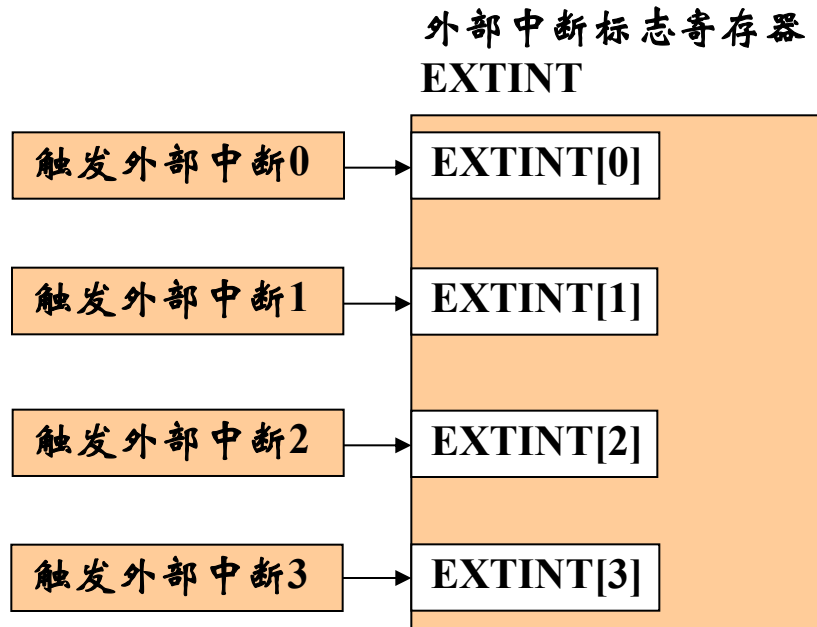
- 当EXTPOLAR[0] = 0时，外部中断0设置为下降沿触发。
- 当EXTPOLAR[0] = 1时，外部中断0设置为上升沿触发。



注意：这里仅以EINT0为例来进行讲解，EINT1~EINT3与之类似，此处不再重复。

4.10 外部中断输入

- 外部中断的设置——中断标志



注意： 外部中断标志写“1”清零。

4.10 外部中断输入

- 外部中断应用示例

初始化EINT0为电平中断:

```
PINSEL1 = (PINSEL1 & 0xFFFFFFF0) | 0x01;  
EXTMODE = EXTMODE & 0x0E;
```

初始化EINT0为下降沿中断:

```
PINSEL1 = (PINSEL1 & 0xFFFFFFF0) | 0x01;  
EXTMODE = EXTMODE | 0x01;  
EXTPOLAR = EXTPOLAR & 0x0E;
```

清除所有外部中断标志:

```
EXTINT = 0x0F;
```



LPC2000系列ARM硬件结构

- 1.LPC2000系列简介
- 2.引脚描述
- 3.存储器寻址
- 4.系统控制模块
- 5.存储器加速模块 (MAM)
- 6.外部存储器控制器 (EMC)
- 7.引脚连接模块
- 8. GPIO
- 9. 向量中断控制器
- 10.外部中断输入
- 11.定时器0和定时器1
- 12. SPI接口
- 13. I²C接口
- 14. UART(0、1)
- 15. A/D转换器
- 16. 看门狗
- 17. 脉宽调制器(PWM)
- 18. 实时时钟

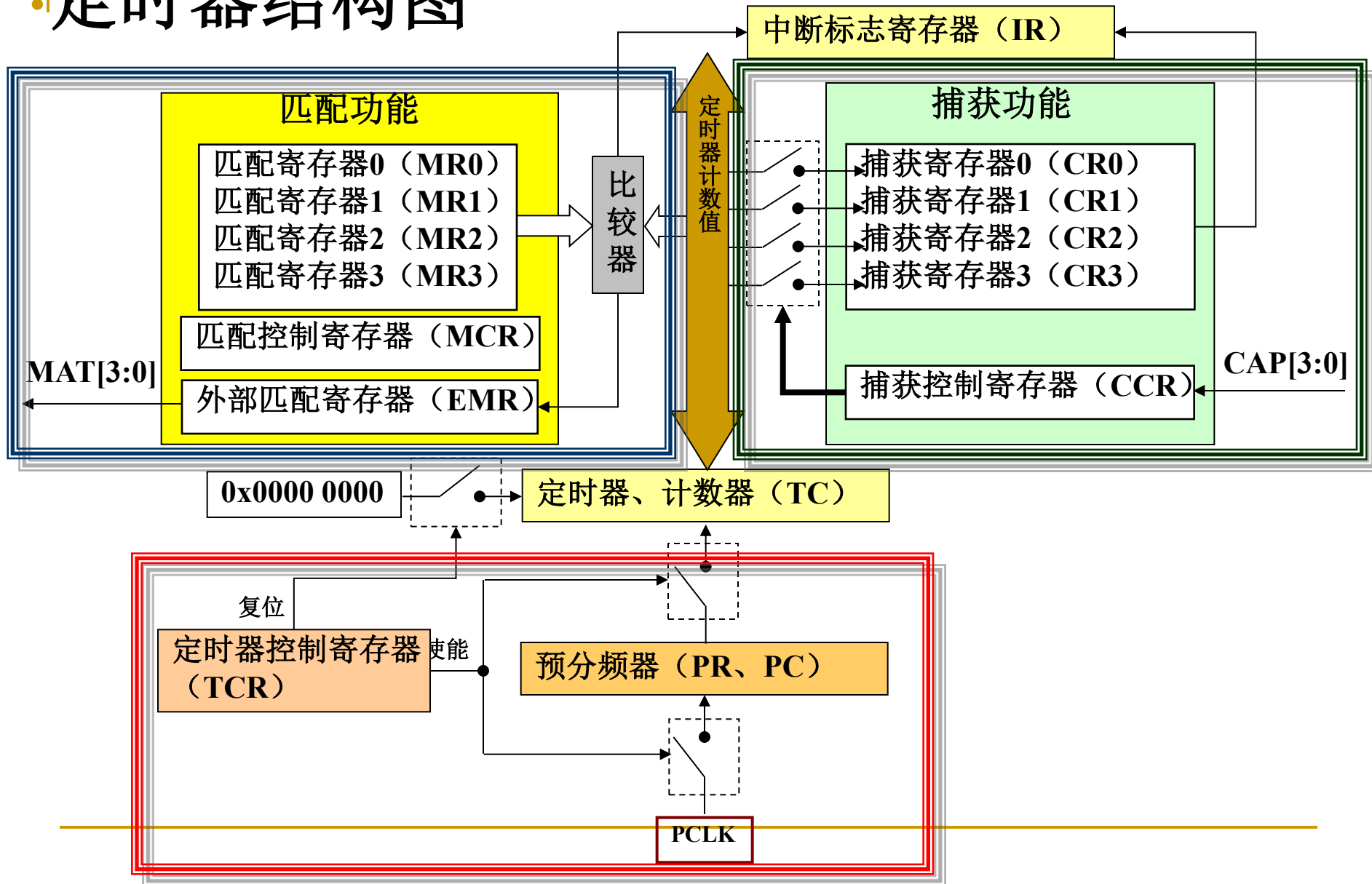


4.11 定时器0、1

■ 特性

- 32位可编程预分频器；
- 4路捕获通道；
- 4个匹配寄存器；
- 4个匹配输出通道。

定时器结构图





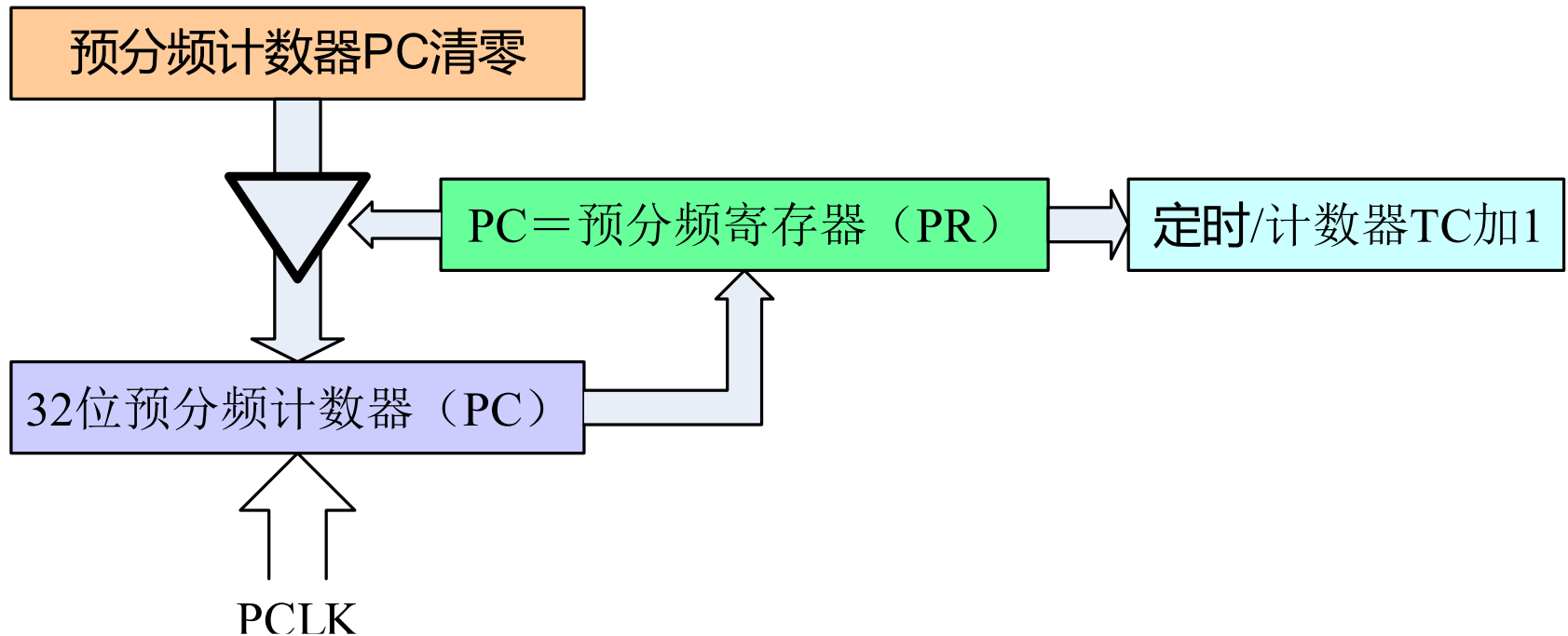
4.11 定时器0、1

■ 功能简介

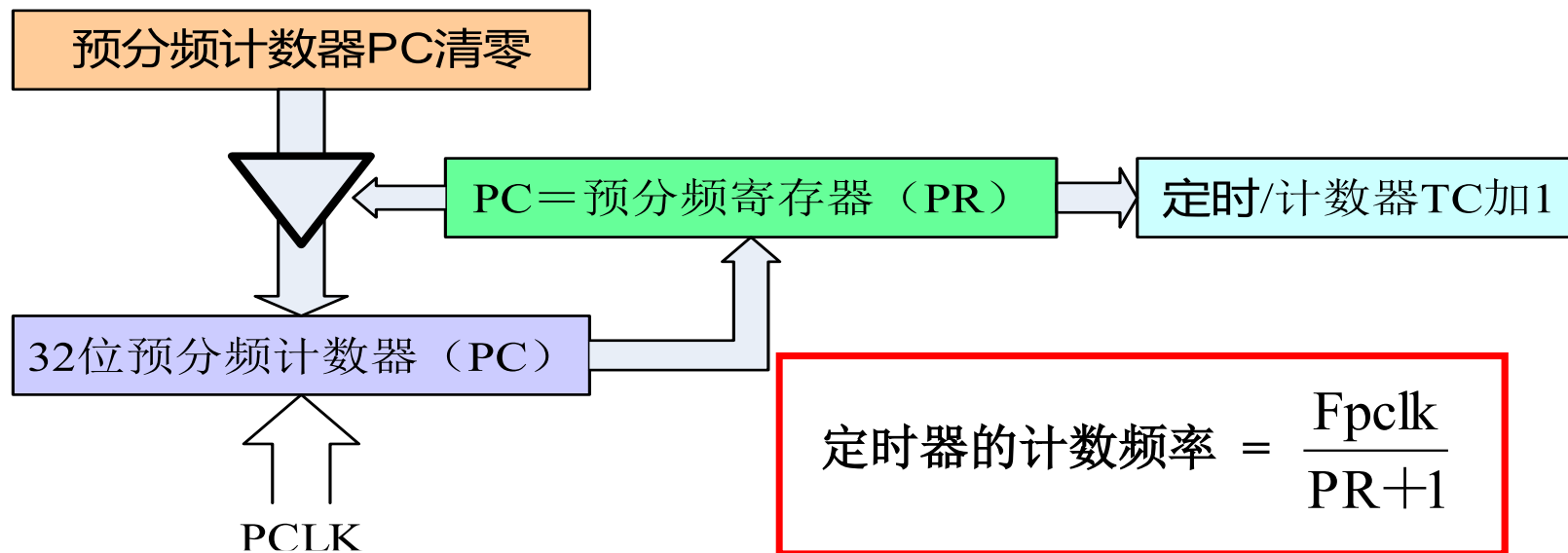
- 预分频器
- 匹配功能
- 捕获功能

4.11 定时器0、1

■ 分频器结构描述

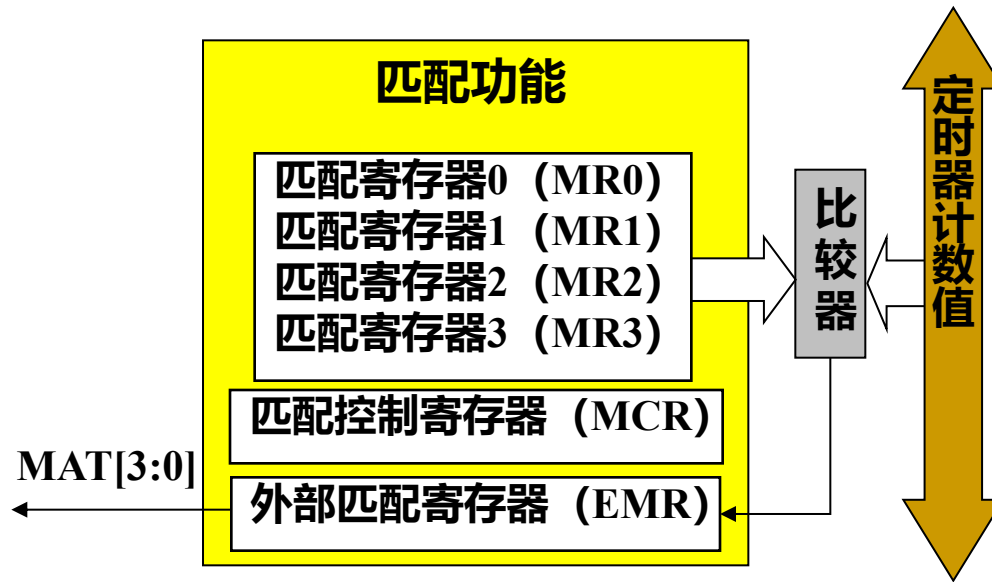


分频器寄存器描述



名称	描述	访问	复位值
PR	预分频控制寄存器。用于设定预分频值，为32位寄存器。	读写	0
PC	预分频计数器。为32位计数器，计数频率为PCLK，当计数值等于预分频计数器的值时，TC计数器加1。	读写	0
TC	定时器计数器。为32位计数器，计数频率为PCLK经过预分频计数器后频率值。	读写	0

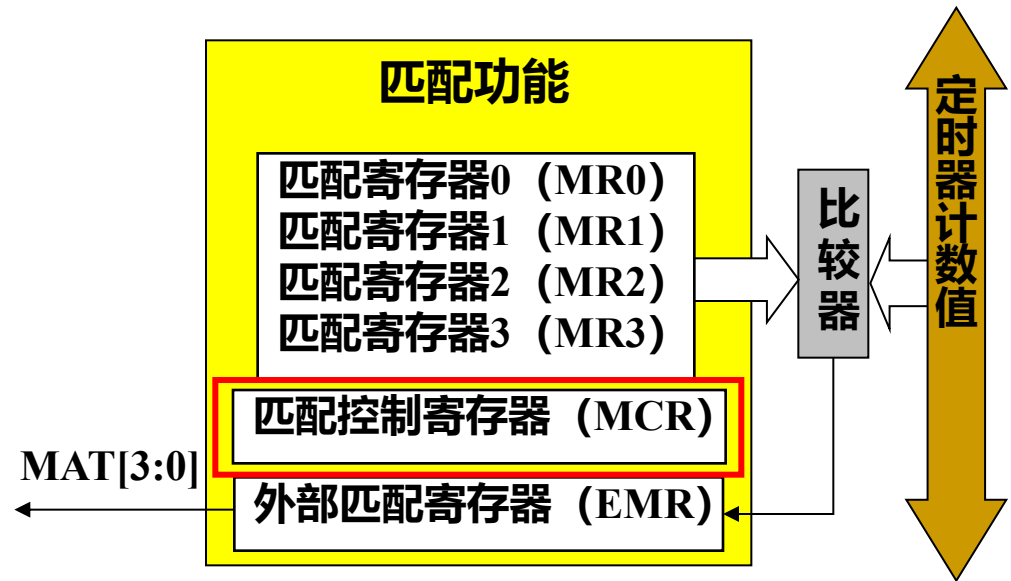
匹配功能



名称	描述	访问	复位值
MCR	匹配控制寄存器，用于控制在匹配时是否产生中断或复位TC	读写	0
MR0	匹配寄存器0，通过MCR寄存器可以设置匹配发生时的动作	读写	0
MR1	匹配寄存器1，通过MCR寄存器可以设置匹配发生时的动作	读写	0
MR2	匹配寄存器2，通过MCR寄存器可以设置匹配发生时的动作	读写	0
MR3	匹配寄存器3，通过MCR寄存器可以设置匹配发生时的动作	读写	0
EMR	外部匹配寄存器，EMR控制外部匹配管脚MATx.0～MATx.3	读写	0

匹配功能寄存器描述—匹配控制寄存器

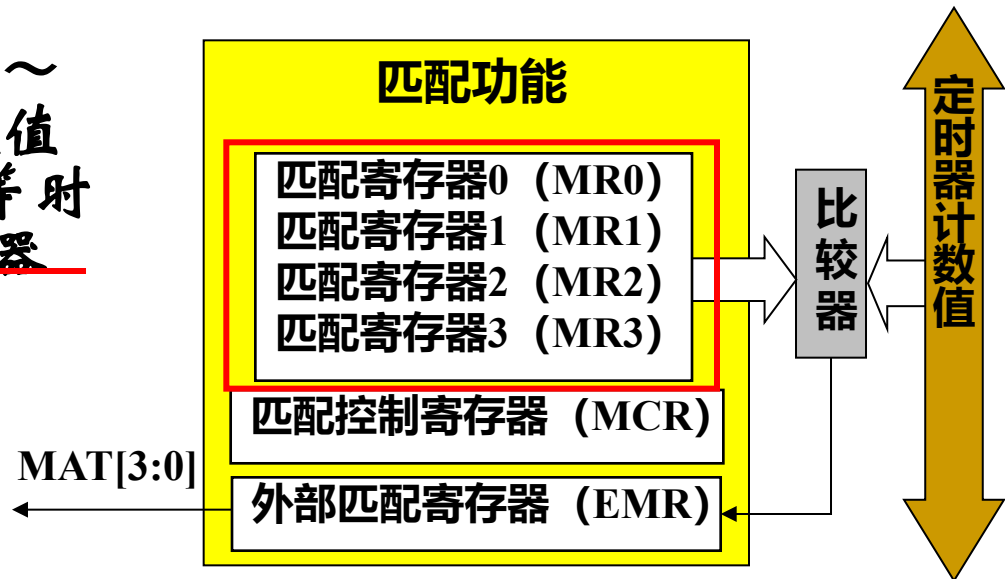
匹配控制寄存器
用于控制在发生匹配
时定时器所执行的操
作。



位	功能	描述	复位值
0	中断 (MR0)	为1时, MR0与TC值的匹配将产生中断。为0时禁止。	0
1	复位 (MR0)	为1时, MR0与TC值的匹配将使TC复位。为0时禁止。	0
2	停止 (MR0)	为1时, MR0与TC值的匹配将清零TCR的bit0位, 使TC和PC停止。为0时该特性被禁止。	0
5 : 3	MR1	与匹配0 (MR0) 对应位功能相同 (略)	0
8 : 6	MR2		0
11 : 9	MR3		0

匹配功能寄存器描述—匹配寄存器

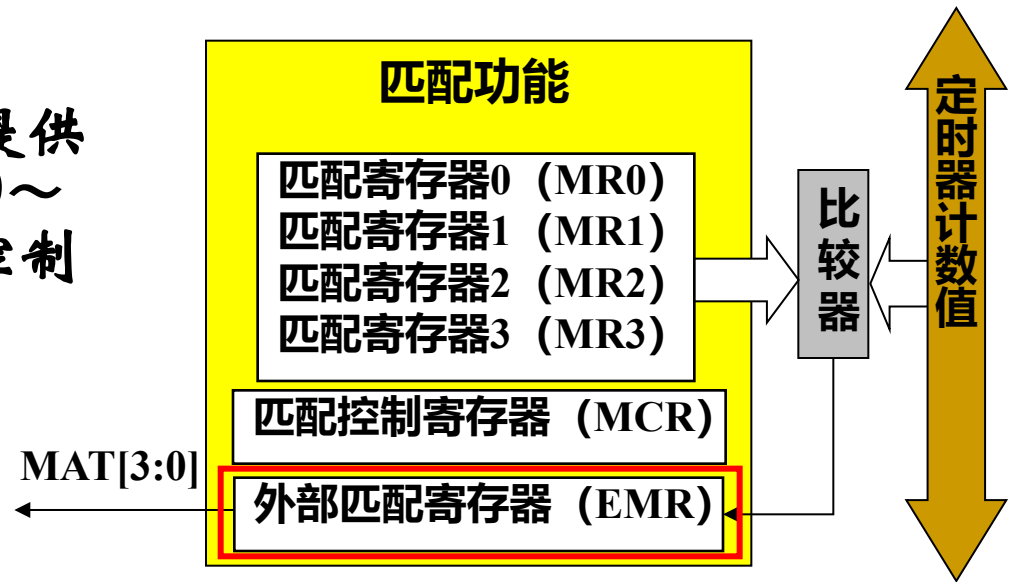
匹配寄存器(MR0~MR3)值与定时器计数值相比较, 当两个值相等时自动触发在MCR寄存器中设置的动作。



位	31 : 0	复位值
功能	匹配值	0

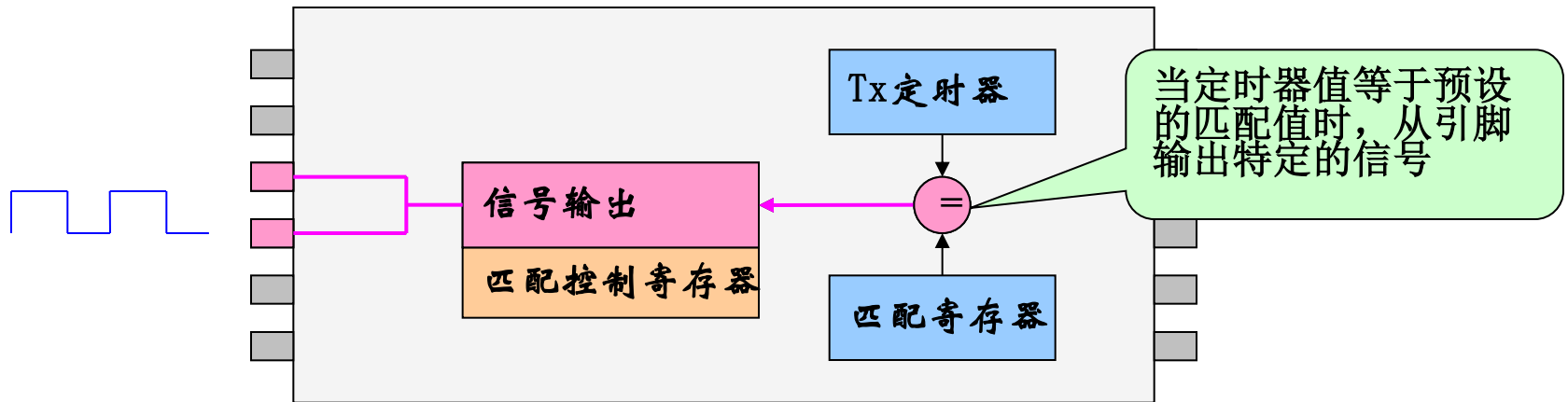
匹配功能寄存器描述—外部匹配寄存器

外部匹配寄存器提供外部匹配管脚MATn.0~MATn.3(n为0或1)的控制和状态。



定时器匹配输出引脚描述

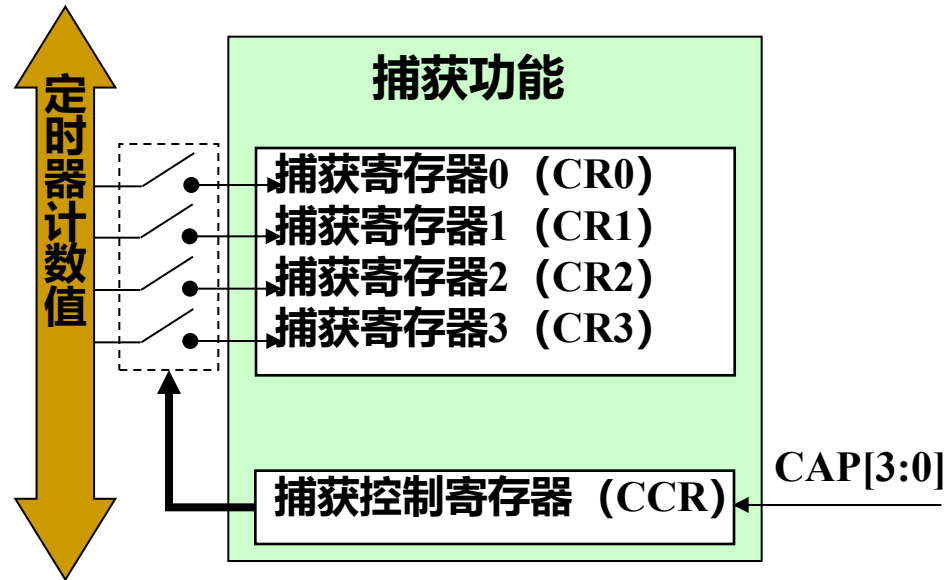
管脚名称	管脚方向	管脚描述
MAT0.3~MAT0.0 MAT1.3~MAT1.0	输出	外部匹配输出0/1。当匹配寄存器0/1（MR3:0）等于定时器计数器（TC）时，该输出可翻转、变为低电平、变为高电平或不变。外部匹配寄存器（EMR）控制该输出的功能。可选择多个管脚并行用作匹配输出功能。例如，同时选择2个管脚并行提供MAT1.3功能。



匹配功能寄存器描述—外部匹配寄存器

位	功能	描述	复位值
0	外部匹配0	反映相应外部匹配的状态，而不管是否连接到管脚。发生匹配时该位的动作由EMR中相应的控制位决定。	0
1	外部匹配1		0
2	外部匹配2		0
3	外部匹配3		0
5 : 4	外部匹配控制0	决定相应外部匹配的功能。 00: 不执行任何动作; 01: 将对应的外部匹配输出设置为0; 10: 将对应的外部匹配输出设置为1; 11: 使对应的外部匹配输出翻转。	0
7 : 6	外部匹配控制1		0
9 : 8	外部匹配控制2		0
11 : 10	外部匹配控制3		0

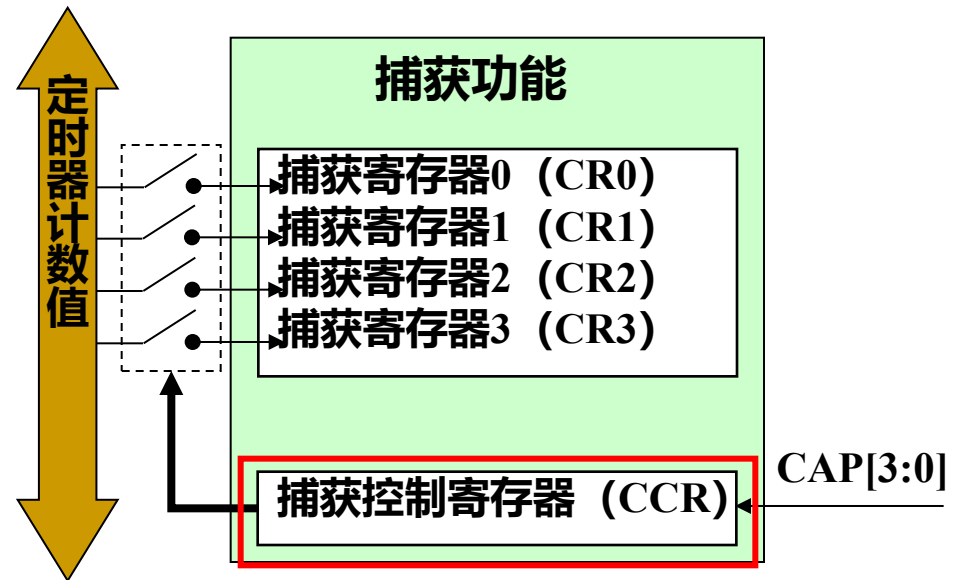
捕获功能



名称	描述	访问	复位值
CCR	捕获控制寄存器，用于设置捕获信号的触发特征，以及捕获发生时是否产生中断。	读写	0
CR0	捕获寄存器0，在捕获0引脚上产生捕获时间时，CR0装载TC的值。	只读	0
CR1	功能同上。	只读	0
CR2	功能同上。	只读	0
CR3	功能同上。	只读	0

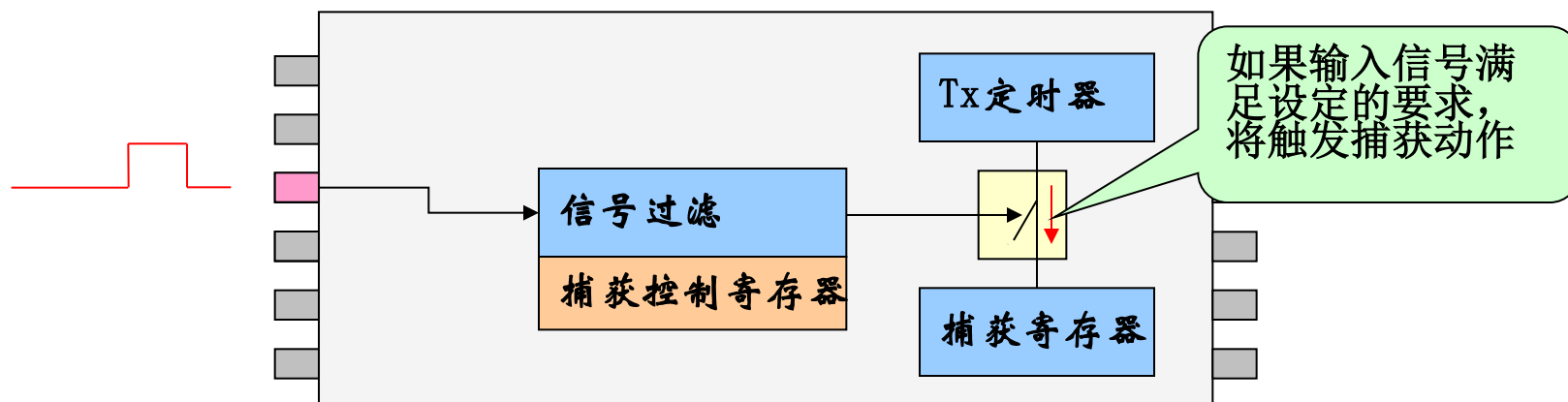
捕获功能寄存器描述—捕获控制寄存器

在发生捕获事件时，捕获控制寄存器用于控制是否将定时器计数值装入寄存器。同时还可设置捕获信号的特征。



定时器捕获引脚描述

管脚名称	管脚方向	管脚描述
CAPO.3~CAPO.0 CPA1.3~CAP1.0	输入	捕获信号，用来捕获管脚的跳变，可配置为将定时器值装入一个捕获寄存器，并可选择产生一个中断。

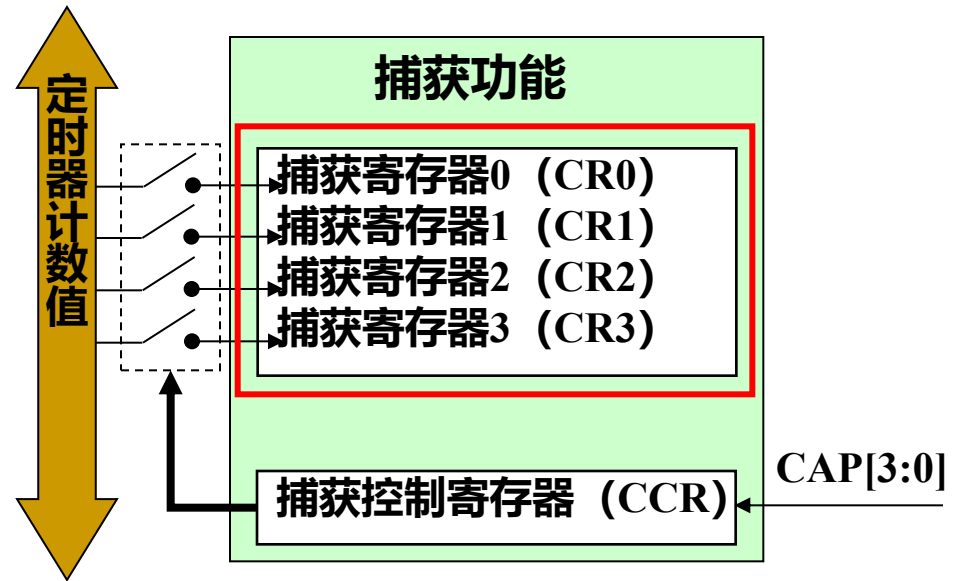


• 捕获功能寄存器描述—捕获控制寄存器

位	功能	描述	复位值
0	CAPn. 0 上升沿捕获	为1时，CAPn. 0引脚上0到1的跳变将导致TC的内容装入CR0。为0时，该特性被禁止。	0
1	CAPn. 1 下降沿捕获	为1时，CAPn. 0引脚上1到0的跳变将导致TC的内容装入CR0。为0时，该特性被禁止。	0
2	CAPn. 0 事件中断	为1时，CAPn. 0的捕获事件将产生一个中断。为0时该特性被禁止。	0
5 : 3	CAPn. 1	与CAPn. 0对应位功能相同（略）	0
8 : 6	CAPn. 2	与CAPn. 0对应位功能相同（略）	0
11 : 9	CAPn. 3	与CAPn. 0对应位功能相同（略）	0

捕获功能寄存器描述—捕获寄存器

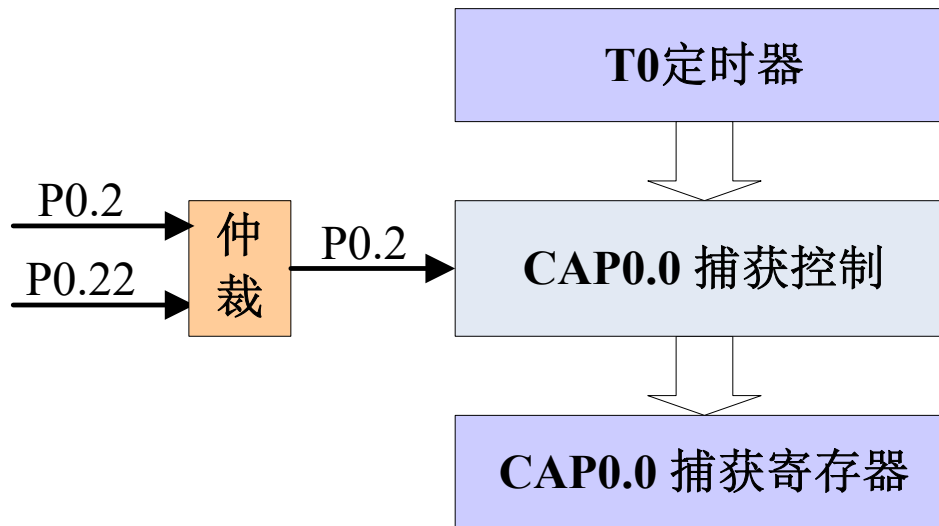
当发生捕获事件时，
可将定时器计数值装入
该寄存器。



位	31 : 0	复位值
功能	捕获值	0

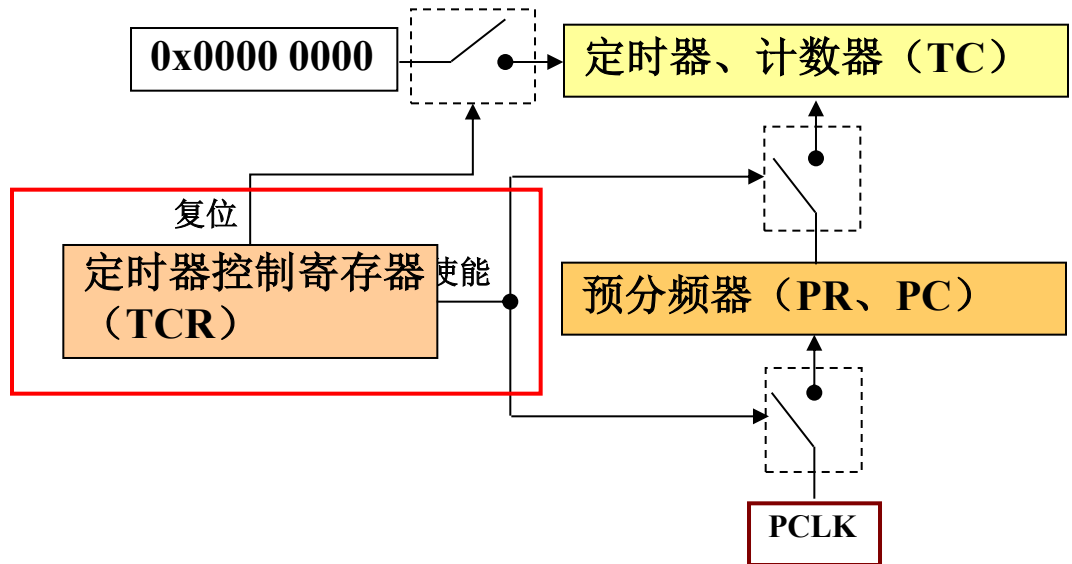
• 捕获功能注意事项

当选择多个管脚作捕获功能时，只有序号最低的那一个管脚是有效的。例，如果P0.2与P0.22均设置为CAP0.0，那么只有P0.2是有效的，P0.22的捕获功能无效。



• 控制寄存器—TCR

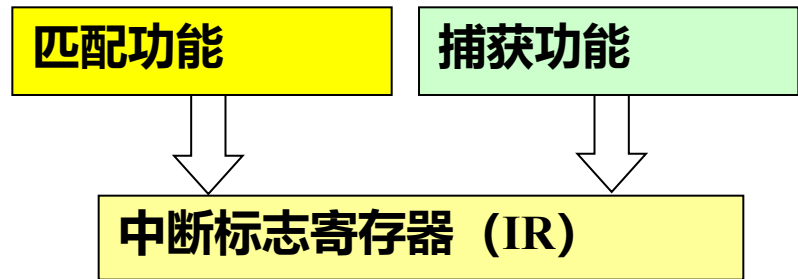
定时器控制寄存器TCR用于控制定时器计数器的操作。



TCR	功能	描述	复位值
0	计数器使能	1:定时器计数器和预分频计数器使能计数; 0:定时器计数器和预分频计数器停止计数。	0
1	计数器复位	为1时定时器计数器和预分频计数器在PCLK的下一个上升沿同步复位。计数器在TCR的bit1恢复为0之前保持复位状态。	0

中断标志寄存器—IR

中断寄存器包含4个位用于匹配中断，另外4个位用于捕获中断。如果有中断产生，IR中的对应位会置位。向对应的IR位写入1会复位中断，写入0无效。

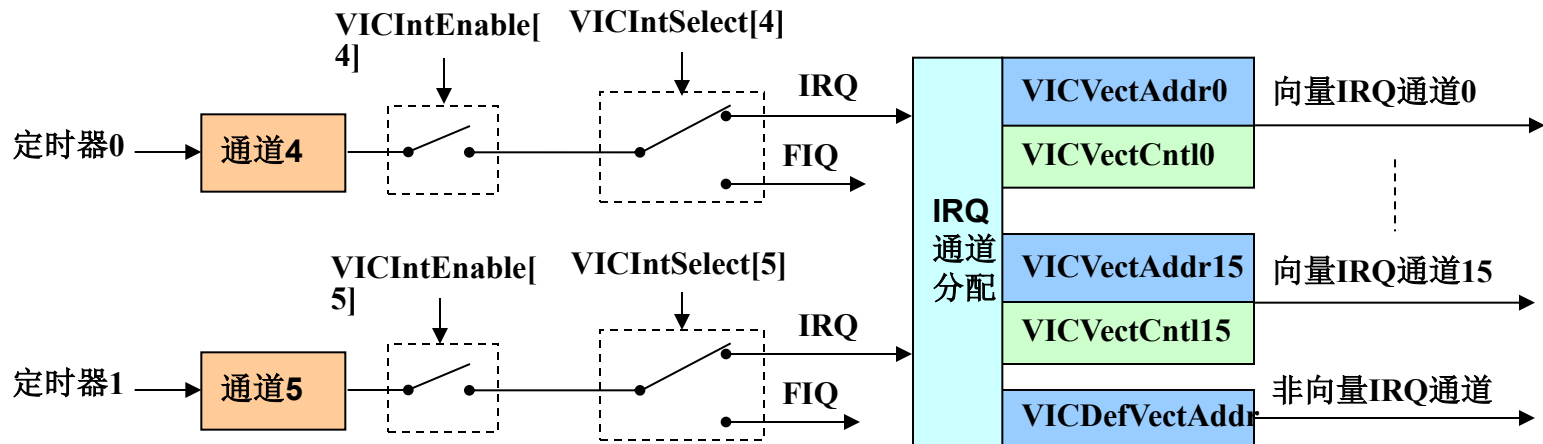


位	功能	描述	位	功能	描述
0	MR0中断	匹配0中断	4	CR0中断	捕获0中断
1	MR1中断	匹配1中断	5	CR1中断	捕获1中断
2	MR2中断	匹配2中断	6	CR2中断	捕获2中断
3	MR3中断	匹配3中断	7	CR3中断	捕获3中断

4.11 定时器0/1

■ 定时器中断——定时器与VIC的关系

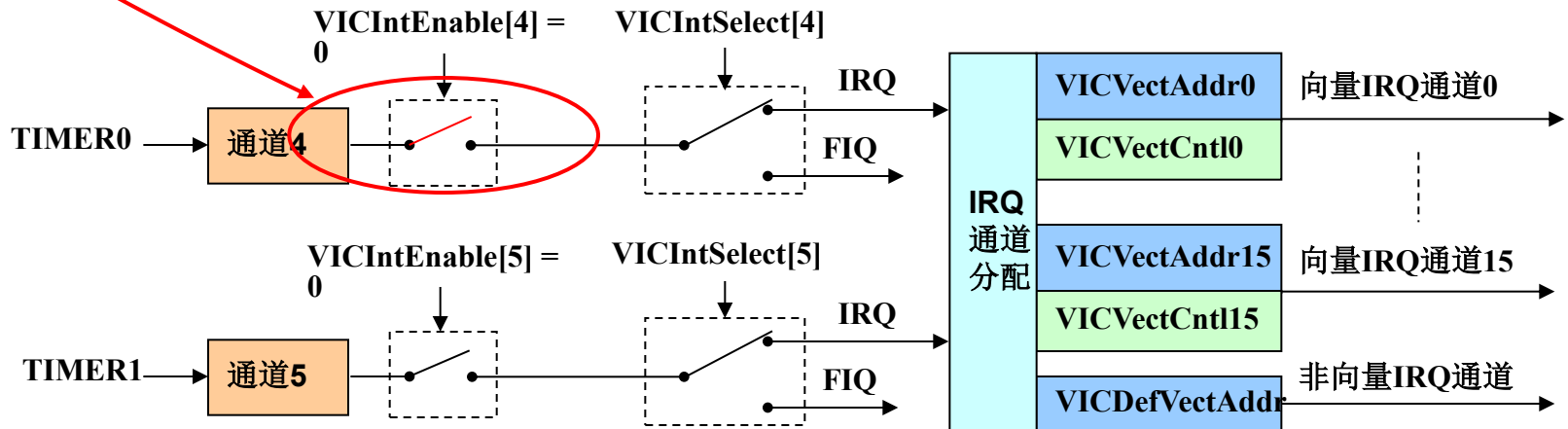
TIMER0、TIMER1分别位于VIC的通道4和通道5。中断使能寄存器VICIntEnable的Bit4和Bit5分别用来控制通道4和通道5的使能。



定时器中断

• TIMER0与VIC的关系

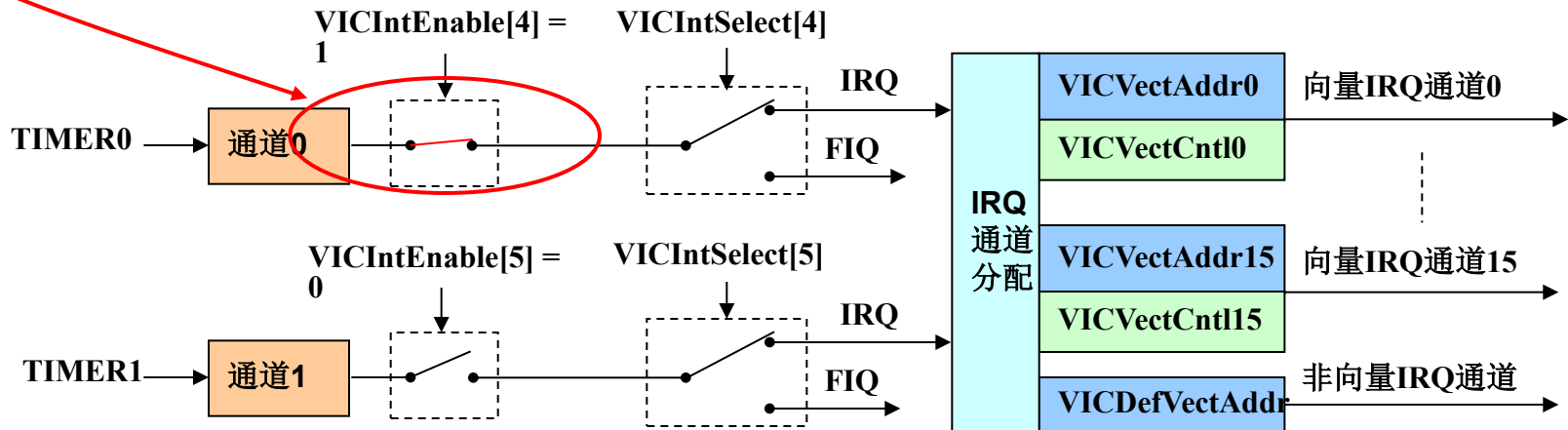
➤ 当 $VICIntEnable[4] = 0$ 时，通道4中断禁止；



定时器中断

• TIMER0与VIC的关系

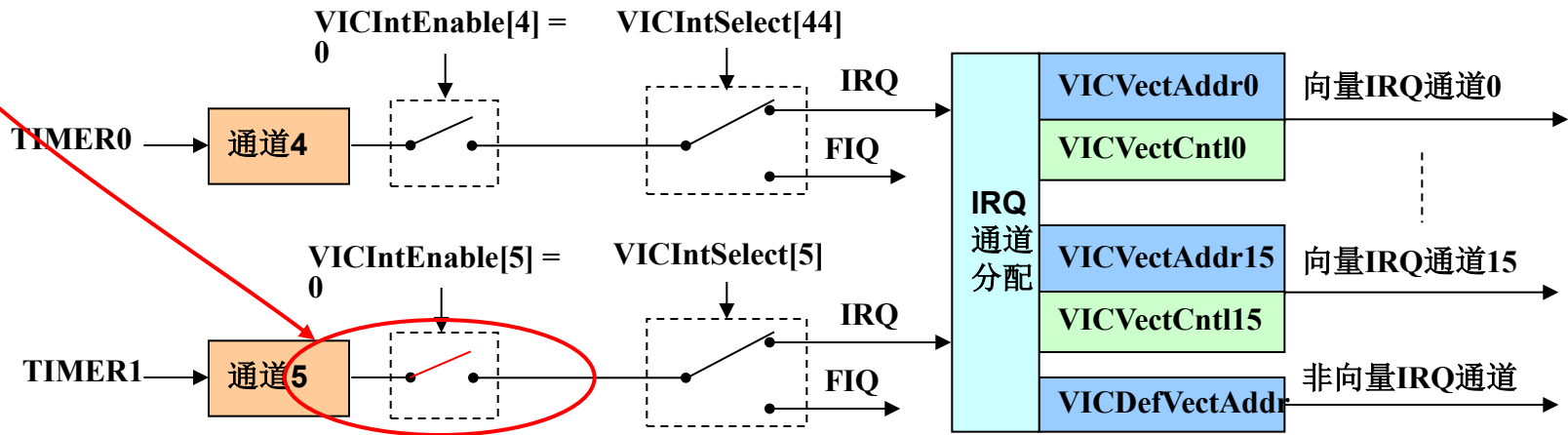
- 当 $VICIntEnable[4] = 0$ 时，通道4中断禁止；
- 当 $VICIntEnable[4] = 1$ 时，通道4中断使能。



定时器中断

• TIMER1与VIC的关系

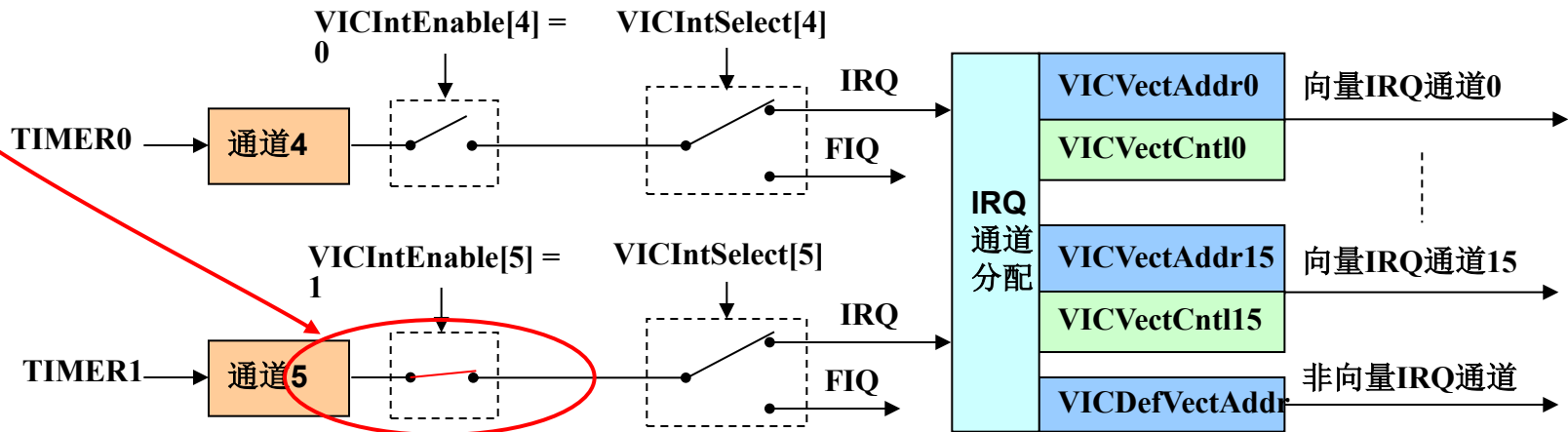
➤ 当 $VICIntEnable[5] = 0$ 时，通道5中断禁止；



定时器中断

• TIMER1与VIC的关系

- 当 $VICIntEnable[5] = 0$ 时，通道5中断禁止；
- 当 $VICIntEnable[5] = 1$ 时，通道5中断使能。



定时器中断

- 匹配中断

LPC2000系列ARM定时器计数溢出时不会产生中断，但是匹配时可以产生中断。每个定时器都具有4个匹配寄存器（MR0~MR3），可以用来存放匹配值。

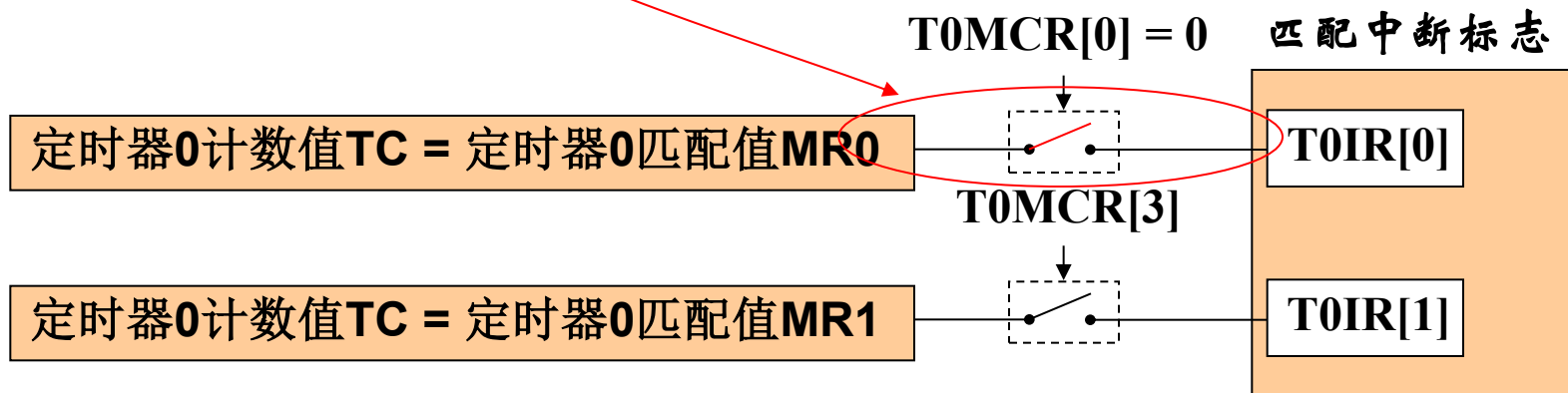
当计数值 = 匹配值时，产生匹配中断。

定时器中断

- 匹配中断

匹配控制寄存器控制着匹配中断的使能，以定时器0匹配通道0为例：

➤ 当 $T0TC = T0MR0$ 时，若 $T0MCR[0] = 0$ ，则匹配中断禁止；

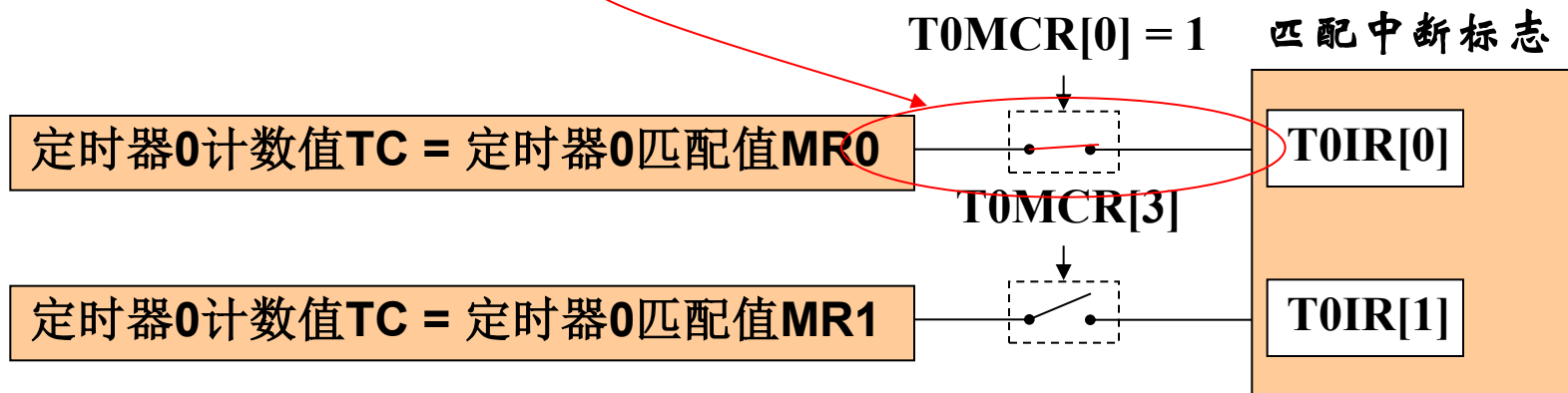


定时器中断

- 匹配中断

匹配控制寄存器控制着匹配中断的使能，以定时器0匹配通道0为例：

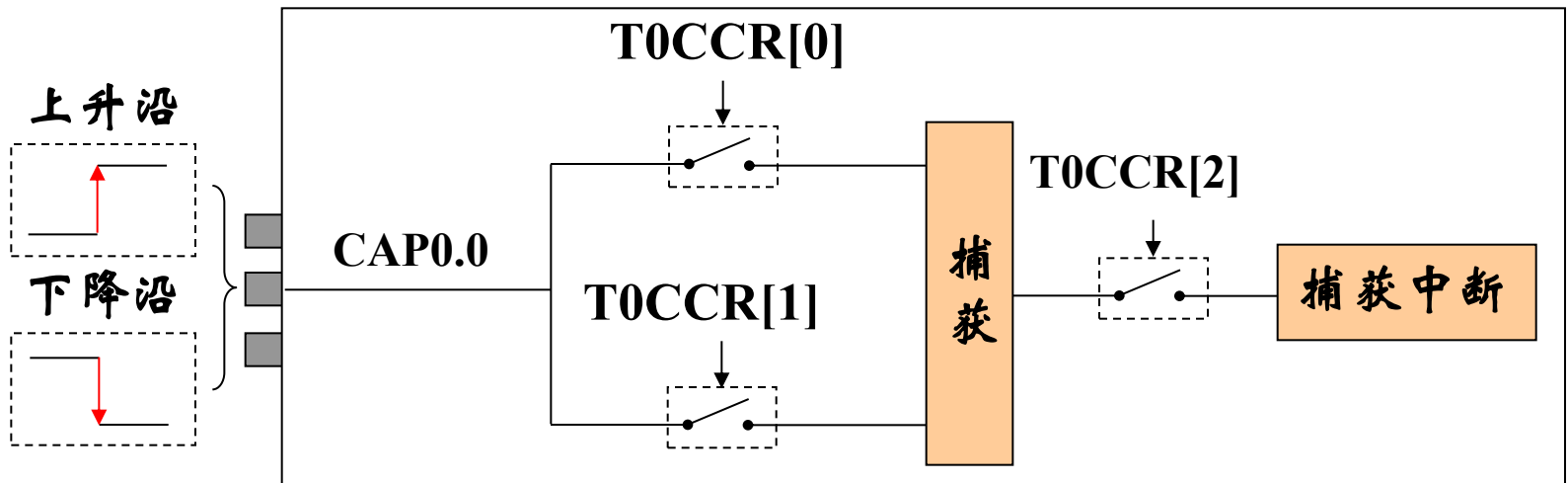
- 当 $T0TC = T0MR0$ 时，若 $T0MCR[0] = 0$ ，则匹配中断禁止；
- 当 $T0TC = T0MR0$ 时，若 $T0MCR[0] = 1$ ，则匹配中断使能。



定时器中断

- 捕获中断

当定时器的捕获引脚CAP上出现特定的捕获信号时，可以产生中断。以CAP0.0为例：

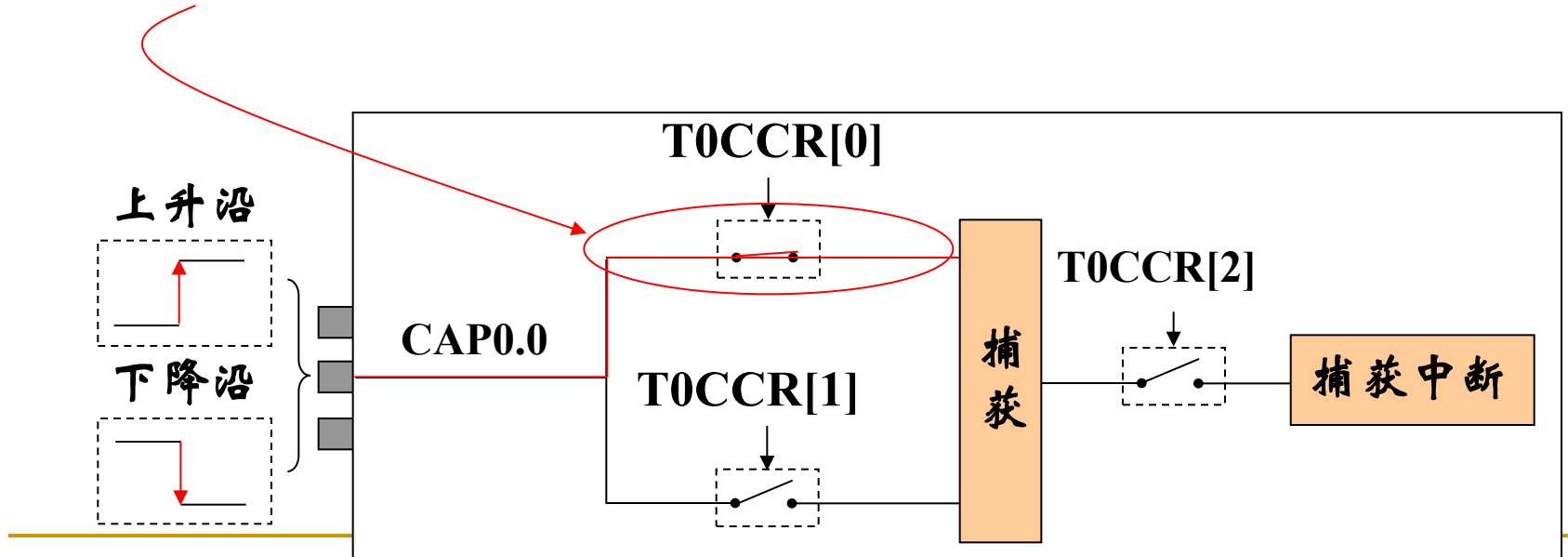


定时器中断

• 捕获中断

当定时器的捕获引脚CAP上出现特定的捕获信号时，可以产生中断。以CAP0.0为例：

➤ 若 $T0CCR[0] = 1$ ，捕获引脚CAP0.0上出现“上升沿”信号时，发生捕获事件；

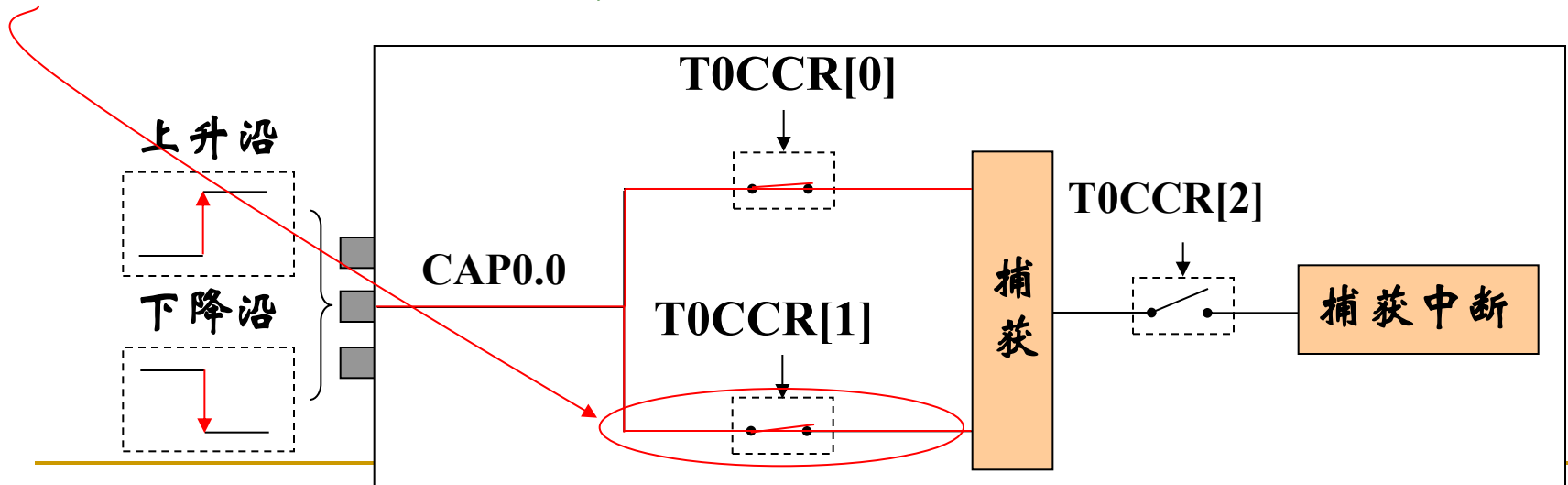


定时器中断

• 捕获中断

当定时器的捕获引脚CAP上出现特定的捕获信号时，可以产生中断。以CAP0.0为例：

- 若T0CCR[0] = 1，捕获引脚CAP0.0上出现“上升沿”信号时，发生捕获事件；
- 若T0CCR[1] = 1，捕获引脚CAP0.0上出现“下降沿”信号时，发生捕获事件；

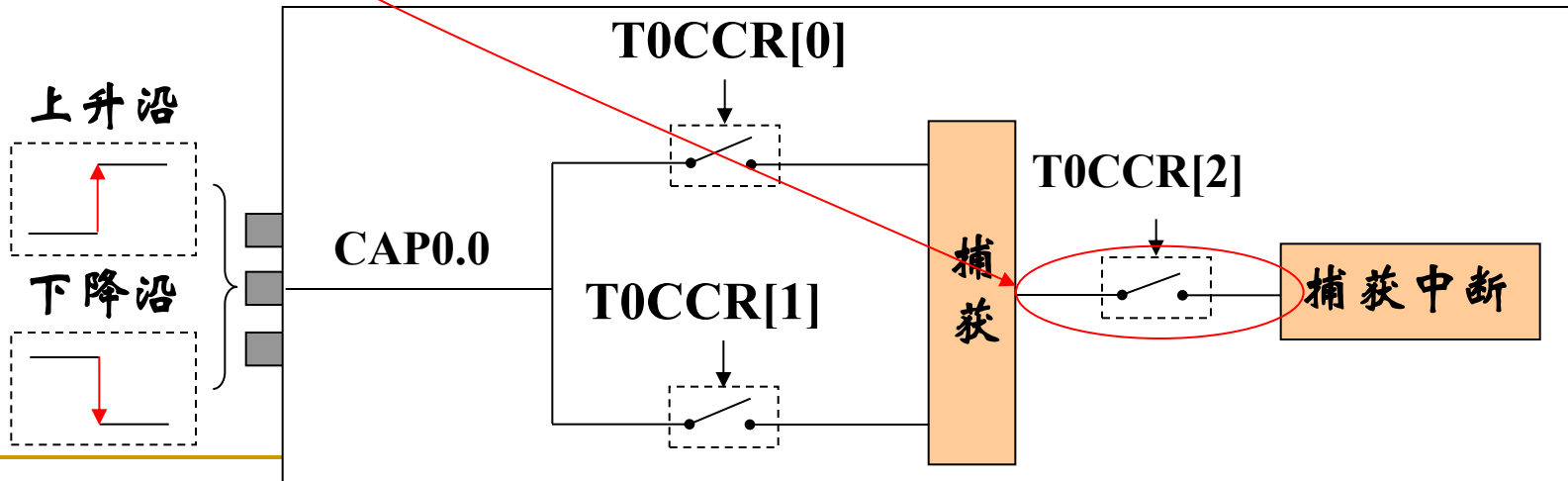


定时器中断

• 捕获中断

捕获控制寄存器CCR控制捕获中断的使能。以CAP0.0为例，发生捕获事件时，T0CCR[2]控制着捕获中断的使能：

➤ 当T0CCR[2] = 0时，捕获中断禁止；

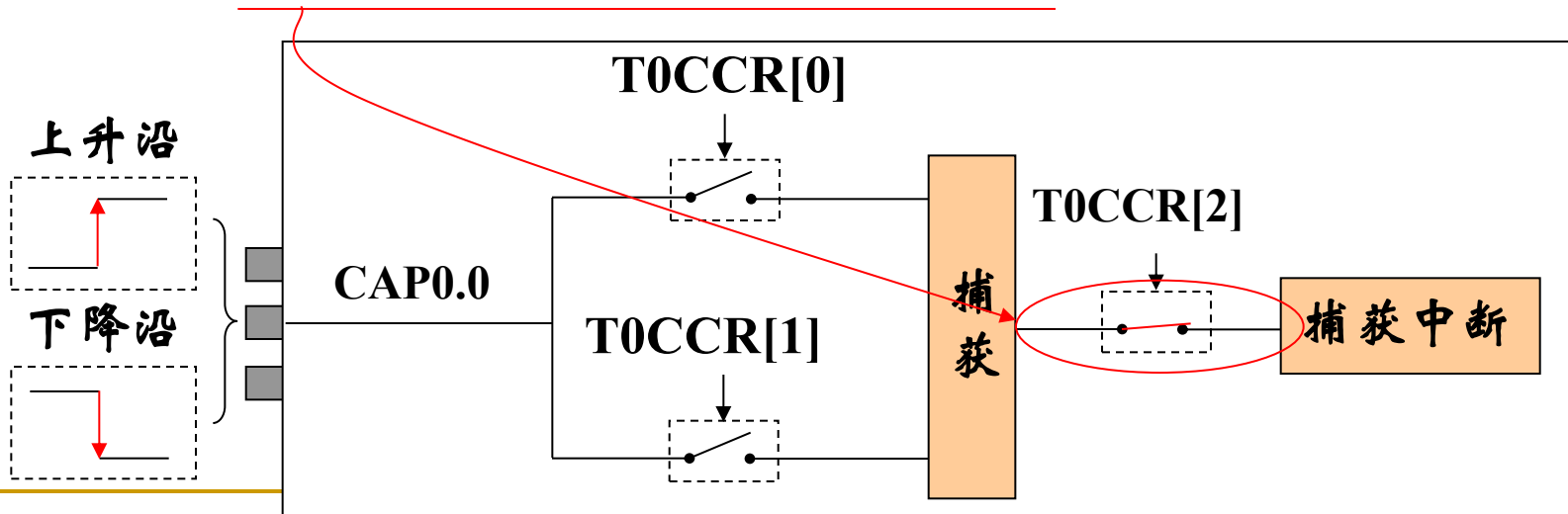


定时器中断

• 捕获中断

捕获控制寄存器CCR控制捕获中断的使能。以CAP0.0为例，发生捕获事件时，T0CCR[2]控制着捕获中断的使能：

- 当T0CCR[2] = 0时，捕获中断禁止；
- 当T0CCR[2] = 1时，捕获中断使能。



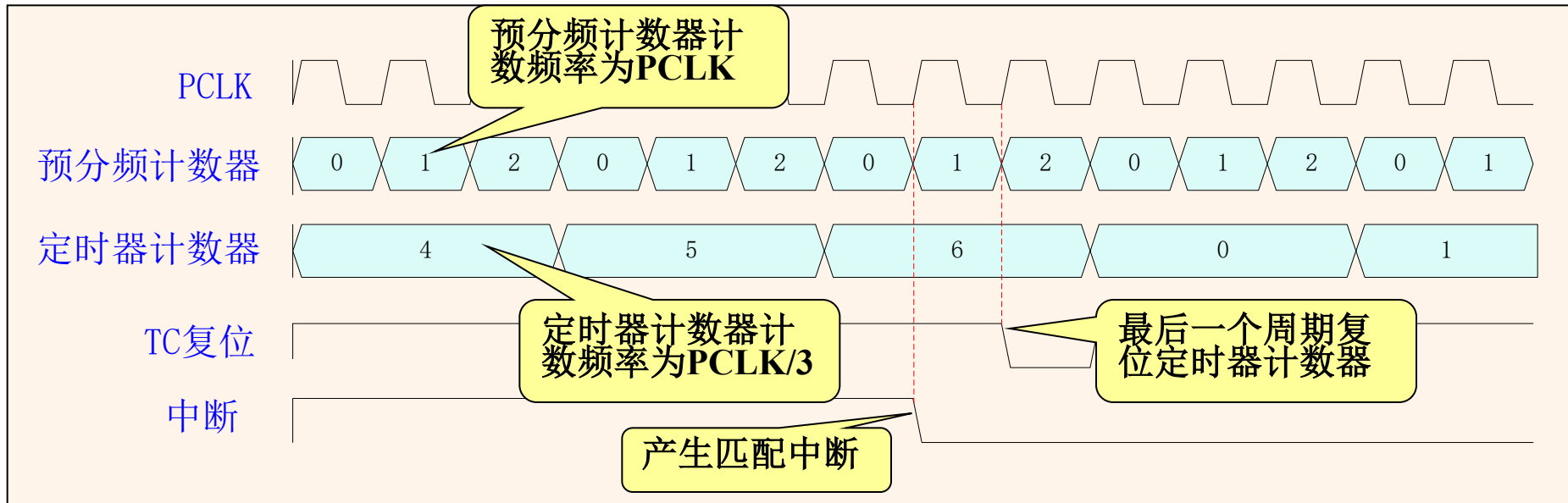
4.11 定时器0/1

■ 使用定时器的注意要点

- 定时计数器(TC)本身不能产生中断，只有与匹配寄存器发生匹配后才能引起中断事件；
- 在定时器匹配发生后，可以不停止定时器工作，而动态修改匹配寄存器的值；
- 定时器使用匹配功能的同时，还可以使用捕获功能，而不必分时使用；
- 定时器计数时钟频率 = $F_{\text{pclk}} / (PR+1)$

• 定时器操作示例

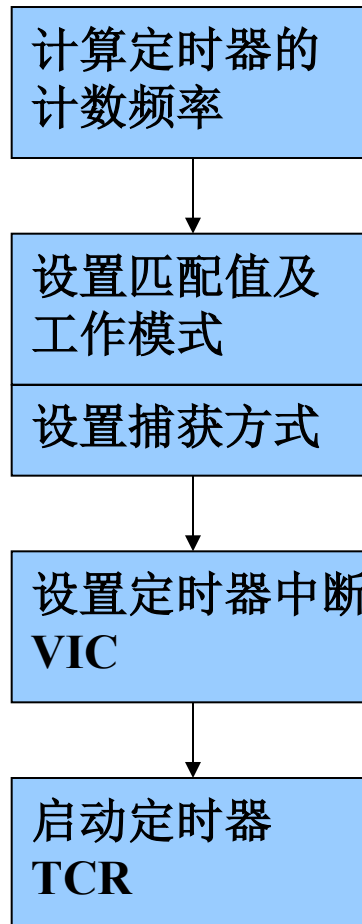
定时器设置为匹配时复位计数器并产生中断。预分频设置为2，匹配寄存器设置为6。在发生匹配的定时器周期结束时，定时器计数值复位。这样就使匹配值具有完整长度的周期。



PR=2, MRx=6, 匹配时使能中断和复位

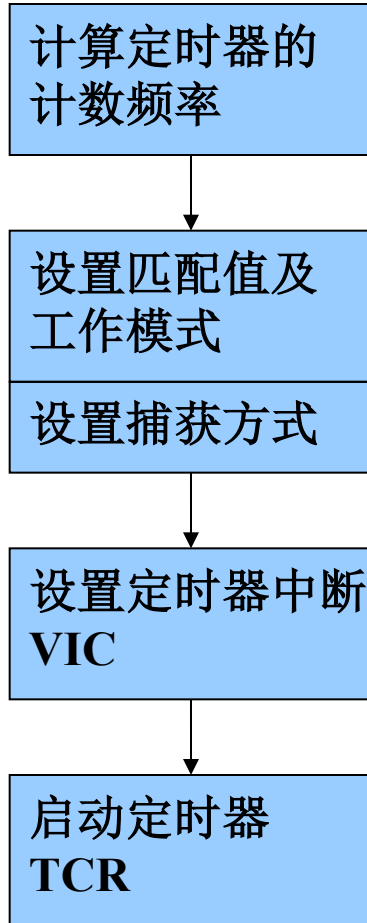
• 定时器操作示例

操作流程

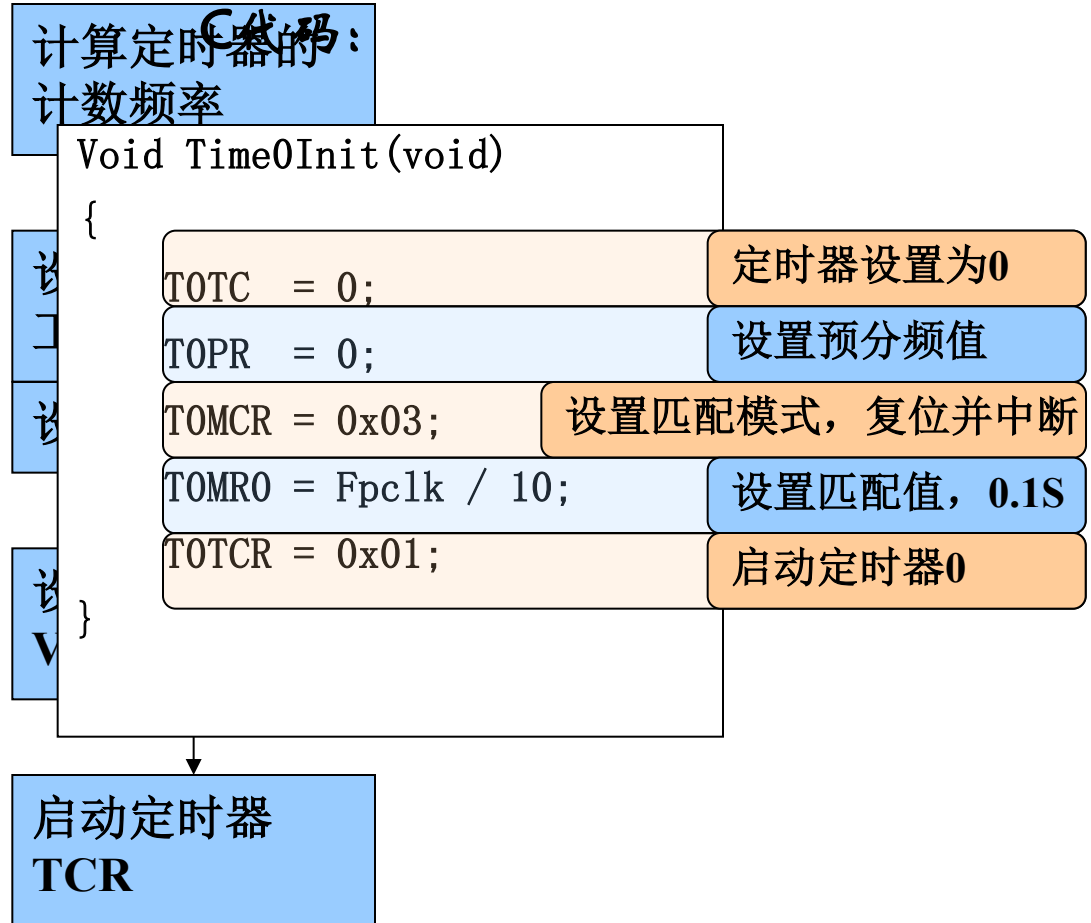


定时器操作示例—定时器0初始化

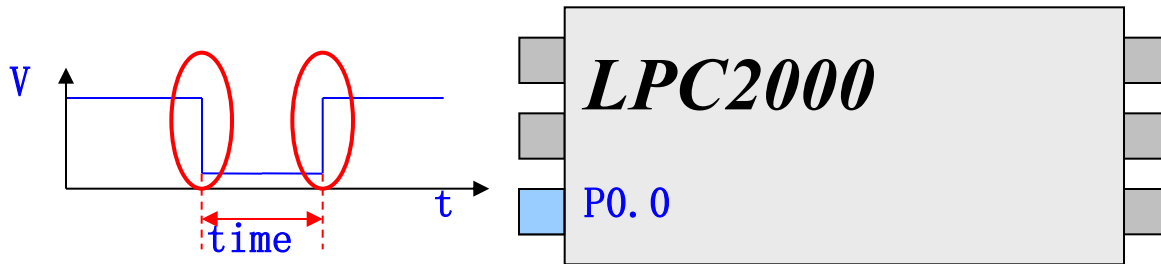
操作流程



操作流程



• 定时器操作示例一—用定时器测量脉冲宽度



C代码:

```
...  
T0TC = 0;           定时器设置为0  
T0PR = 0;           设置预分频值  
while((I00PIN & 0x01) != 0); 等待引脚电平变低  
T0TCR = 0x01;        启动定时器0  
while((I00PIN & 0x01) == 0); 等待引脚电平变高  
T0TCR = 0x00;        关闭定时器0  
time = T0TC;         读取定时器值，即为脉宽  
...
```

• 定时器操作示例一 匹配输出

将引脚P0.5设置为输出50%的方波，程序设置了MR1匹配后复位定时器，并且MAT0.1输出电平翻转。

C代码：

```
Void MATOut(void)
```

```
{
```

```
PINSEL0 = 0x00000800;
```

设置引脚连接模块

```
TOTC = 0;
```

定时器设置为0

```
TOPR = 0;
```

设置预分频值

```
TOMCR = 0x02;
```

设置匹配后复位TC

```
TOEMR = 0xC0;
```

设置匹配后MAT0.1输出翻转

```
TOMR1 = 5000;
```

输出频率周期控制

```
TOTCR = 0x01;
```

启动定时器0

```
}
```

• 定时器操作示例—定时器捕获

示例使用定时器对P0.2引脚的信号进行捕获，并设置为下降沿捕获。当有捕获事件产生时自动把定时器的当前值装载到TOCRO寄存器中。

C代码：

```
Void TimeCAP(void)
```

```
{
```

```
PINSEL0 = 0x20;
```

设置引脚连接模块

```
TOPR = 0;
```

设置预分频值为0

```
TOCCR = 0x02;
```

设置为下降沿捕获

```
TOTC = 0;
```

清零TC

```
TOTCR = 0x01;
```

启动定时器

```
}
```



LPC2000系列ARM硬件结构

- 1.LPC2000系列简介
- 2.引脚描述
- 3.存储器寻址
- 4.系统控制模块
- 5.存储器加速模块(MAM)
- 6.外部存储器控制器(EMC)
- 7.引脚连接模块
- 8. GPIO
- 9. 向量中断控制器
- 10.外部中断输入
- 11.定时器0和定时器1
- **12. SPI接口**
- 13. I²C接口
- 14. UART(0、1)
- 15. A/D转换器
- 16. 看门狗
- 17. 脉宽调制器(PWM)
- 18. 实时时钟

4.12 SPI接口

- SPI接口的全称是“Serial Peripheral Interface”(串行外围接口),是Motorola首先在其MC68HCXX系列处理器上定义的。SPI接口主要应用在EEPROM,FLASH,实时时钟,AD转换器,还有数字信号处理器和数字信号解码器之间。
- SPI接口是在CPU和外围低速器件之间进行同步串行数据传输,在主器件的移位脉冲下,数据按位传输,高位在前,低位在后,为全双工通信,数据传输速度总体来说比I²C总线要快,速度可达几Mbps。



4.12 SPI接口

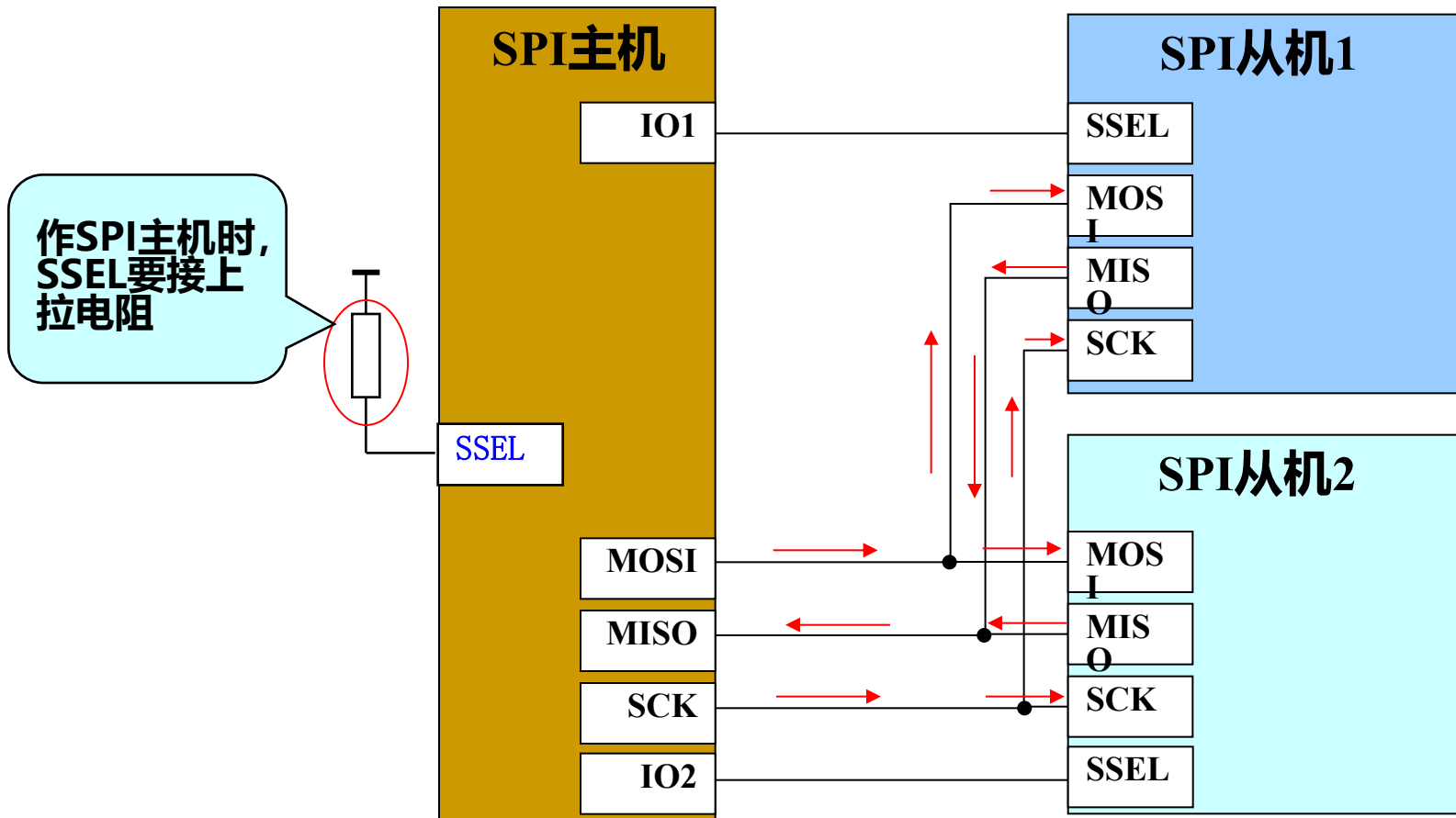
■ 引脚描述

SPI总线系统可直接与各个厂家生产的多种标准外围器件直接接口，该接口一般使用**4**条线：串行时钟线（**SCLK**）、主机输入/从机输出数据线**MISO**、主机输出/从机输入数据线**MOSI**和低电平有效的从机选择线**SS**(有的**SPI**接口芯片带有中断信号线**INT**、有的**SPI**接口芯片没有主机输出/从机输入数据线**MOSI**)。

引脚名称	类型	描述
SCK	输入/输出	串行时钟 。用于同步SPI接口间数据传输的时钟信号。该时钟信号总是由主机输出。
SSEL	输入	从机选择 。SPI从机选择信号是一个低有效信号。
MISO	输入/输出	主入从出 。MISO信号是一个单向的信号，它将数据由从机传输到主机。
MOSI	输入/输出	主出从入 。MOSI信号是一个单向的信号，它将数据从主机传输到从机。

4.12 SPI接口

■ 硬件连接



4.12 SPI接口

- SPI数据传输

- 时钟极性控制位 —— CPOL

该位决定了SPI总线空闲时，SCK时钟线的电平状态。

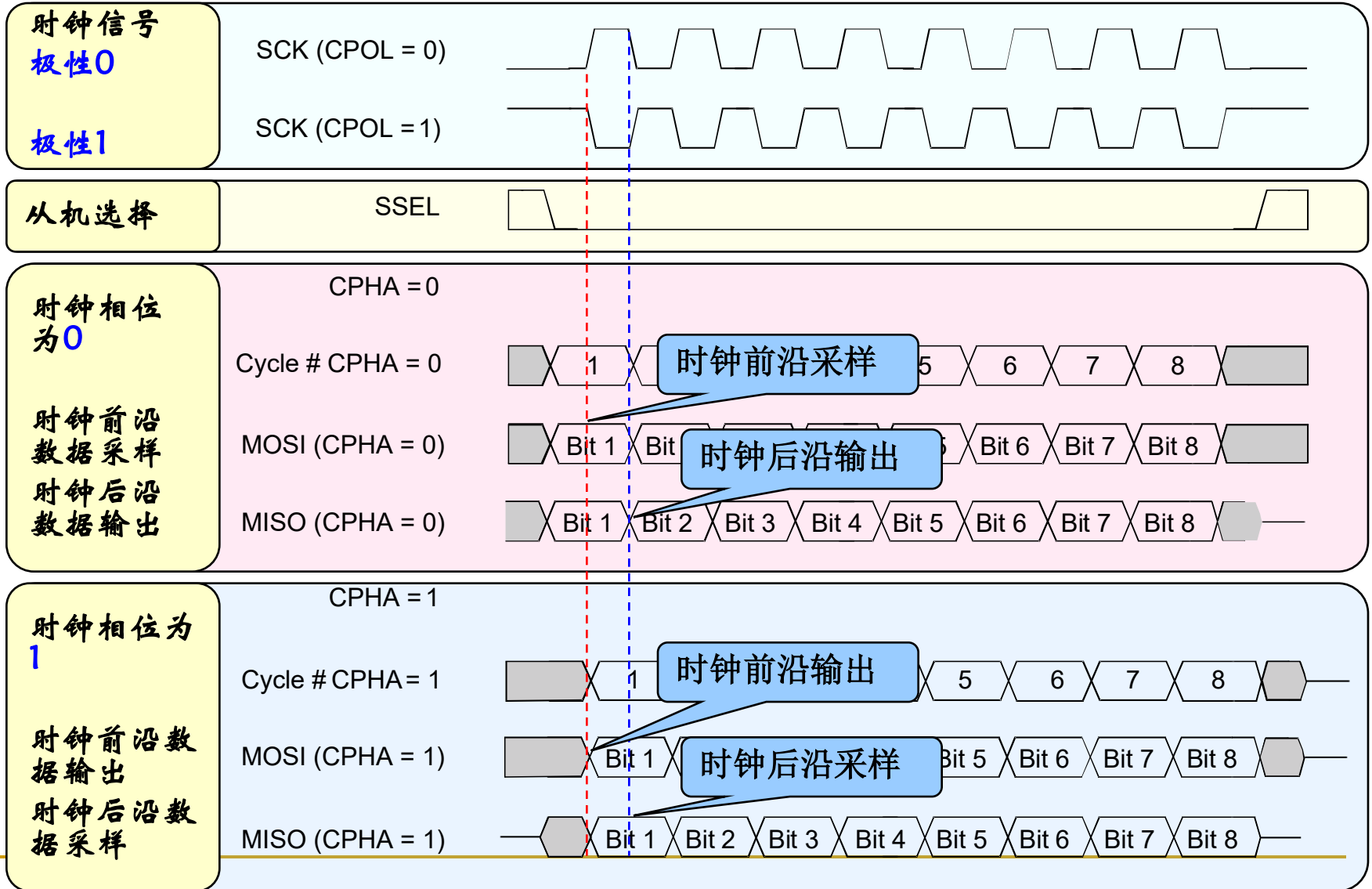
CPOL = 0, 当SPI总线空闲时, SCK时钟线为 低 电平;
CPOL = 1, 当SPI总线空闲时, SCK时钟线为 高 电平。

- 时钟相位控制位 —— CPHA

该位决定SPI总线上数据的采样位置。

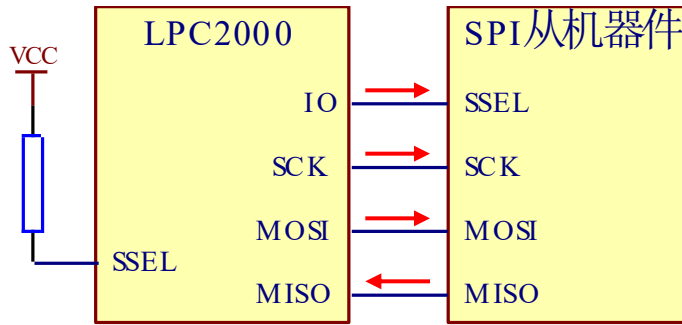
CPHA = 0: SPI总线在时钟线的第 1 个跳变沿处采样数据;
CPHA = 1: SPI总线在时钟线的第 2 个跳变沿处采样数据。

• SPI传输时序



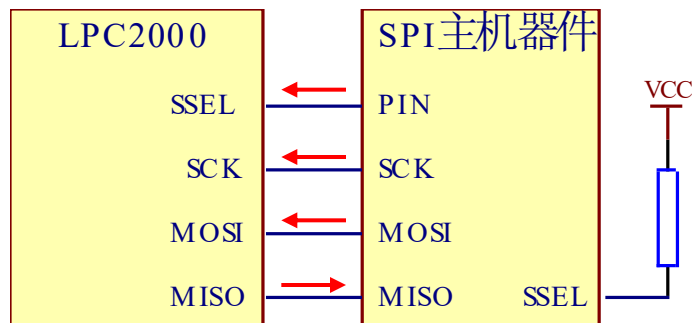
• SPI接口工作模式

• 主机模式



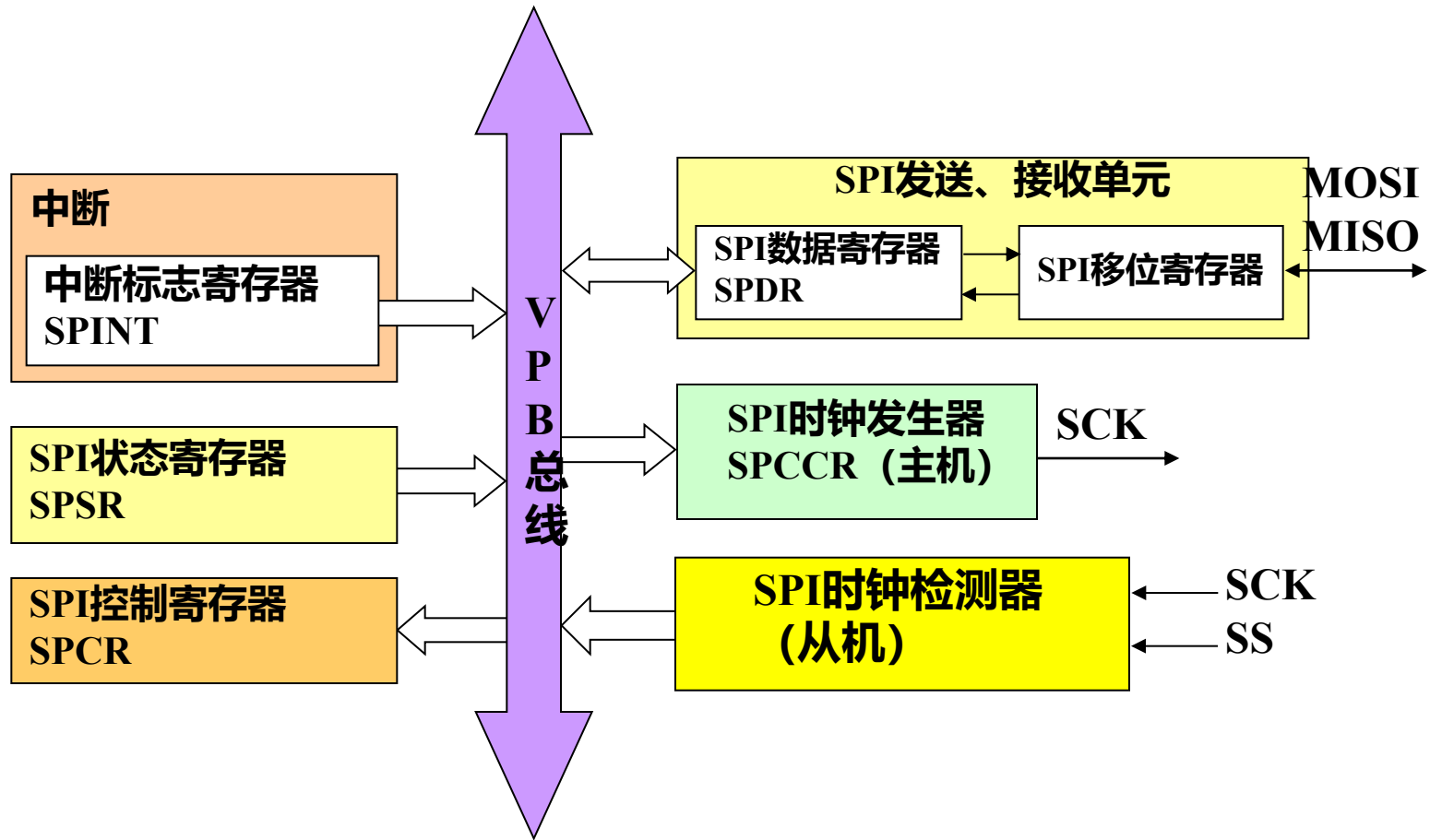
- 主机使用一个IO引脚选择从机；
- 传输的起始由主机发送数据来启动；
- 时钟(SCK)信号由主机产生；
- 通过MOSI发送数据；
- 通过MISO引脚接收数据。

• 从机模式



- 数据传输在SSEL被主机拉低后开始；
- 接收主机输出的时钟信号；
- 通过MOSI引脚接收数据；
- 通过MISO引脚发送数据。

• SPI接口内部框图



4.12 SPI接口

- 寄存器描述

名称	描述	访问	复位值	SPI0名称	SPI1名称
SPCR	SPI控制寄存器 。该寄存器控制SPI的操作模式。	读写	0	S0SPCR	S1SPCR
SPSR	SPI状态寄存器 。该寄存器显示SPI的状态。	只读	0	S0SPSR	S1SPSR
SPDR	SPI数据寄存器 。该双向寄存器为SPI提供发送和接收的数据。发送数据通过写该寄存器提供。SPI接收的数据可以从该寄存器读出。	读写	0	S0SPDR	S1SPDR
SPCCR	SPI时钟计数寄存器 。该寄存器控制主机SCK的频率。	读写	0	S0SPCCR	S0SPCCR
SPINT	SPI中断标志寄存器 。该寄存器包含SPI接口的中断标志。	读写	0	S0SPINT	S0SPINT

4.12 SPI接口

- SPI寄存器描述——SPI控制寄存器

位	7	6	5	4	3	2 : 0
功能	SPIE	LSBF	MSTR	CPOL	CPHA	保留

SPCR寄存器包含一些可编程位来控制SPI功能模块的功能，该寄存器必须在数据传输之前进行设定。

SPI寄存器描述——SPI控制寄存器

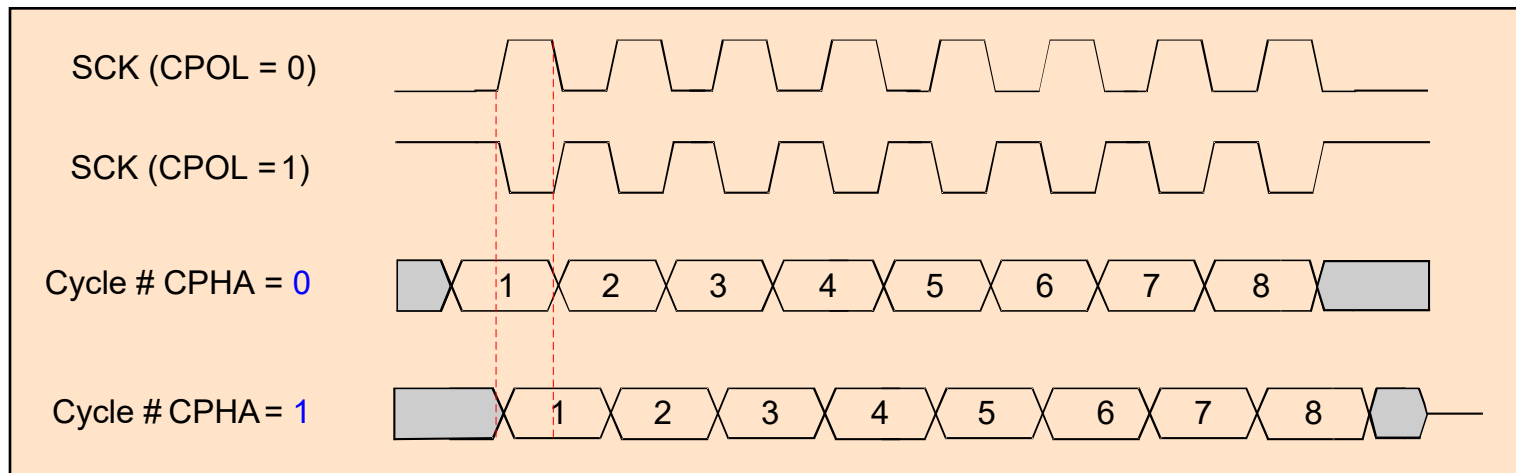
位	7	6	5	4	3	2 : 0
功能	SPIE	LSBF	MSTR	CPOL	CPHA	保留

CPHA : 时钟相位控制。

该位决定SPI传输时数据和时钟的关系，并控制从机传输的起始和结束。当该位为：

1 : 时钟前沿数据输出，后沿数据采样；

0 : 时钟前沿数据采样，后沿数据输出；



• SPI寄存器描述——SPI控制寄存器

位	7	6	5	4	3	2:0
功能	SPIE	LSBF	MSTR	CPOL	CPHA	保留

CPOL:时钟极性控制。

1 : SCK为低电平有效;

0 : SCK为高电平有效;

SCK (CPOL = 0)



SCK (CPOL = 1)



• SPI寄存器描述——SPI控制寄存器

位	7	6	5	4	3	2:0
功能	SPIE	LSBF	MSTR	CPOL	CPHA	保留

CPOL:主模式控制。

1 : SPI处于主模式;

0 : SPI处于从模式;

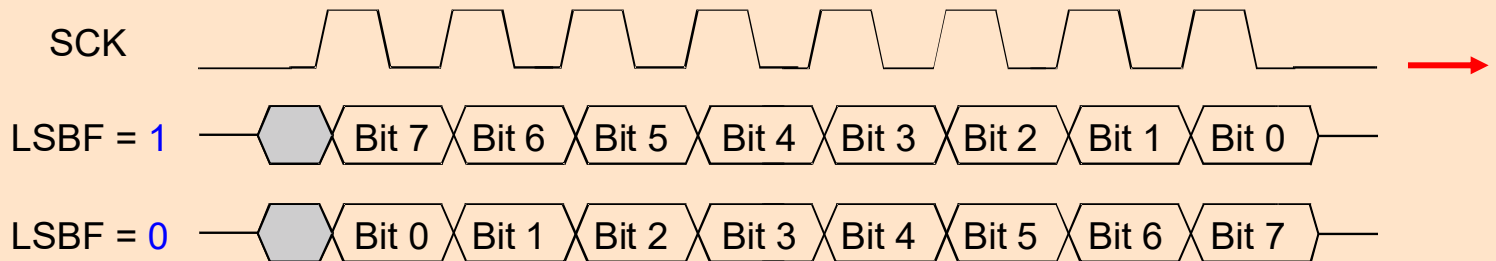
• SPI寄存器描述——SPI控制寄存器

位	7	6	5	4	3	2:0
功能	SPIE	LSBF	MSTR	CPOL	CPHA	保留

LSBF : 字节移动方向控制。

1 : 每字节数据从低位(LSB)开始传输;

0 : 每字节数据从高位(MSB)开始传输;



• SPI寄存器描述——SPI控制寄存器

位	7	6	5	4	3	2:0
功能	SPIE	LSBF	MSTR	CPOL	CPHA	保留

SPIE:SPI中断使能。

1 : 每次SPIF或MODE置位时都会产生硬件中断;

0 : SPI中断被禁止;



• SPI寄存器描述——SPI状态寄存器

SPSR寄存器为只读寄存器，用于监视SPI功能模块的状态，包括一般性功能和异常状况。

SPSR	功能	描述	复位值
2 :0	保留	用户程序不要向这些保留位写入1。	NA
3	ABRT	从机中止标志。为1时表示发生了从机中止。读取该位清零。	0
4	MODF	模式错误。为1时表示发生了模式错误。先通过读取该寄存器清零MODF位，再写SPI控制寄存器。	0
5	ROVR	读溢出。为1时表示发生了读溢出。当读取该寄存器时，该位清零。	0
6	WCOL	写冲突。为1时表示发生了写冲突。先通过读取该寄存器清零WCOL位，再访问SPI数据寄存器。	0
7	SPIF	SPI传输完成标志。为1时表示一次SPI数据传输完成。当第一次读取该寄存器时，该位清零。然后才能访问SPI数据寄存器。 注：SPIF不是SPI中断标志。中断标志位于SPINT寄存器中。	0

4.12 SPI接口

■ 异常状况一 读溢出

SPI模块内部的读缓冲区大小为1个字节， $SPIF = 1$ 表示读缓冲区满。当SPI功能模块内部读缓冲区满时，又接收到新的数据，就会发生读溢出。新接收到的数据将会丢失，而状态寄存器的读溢出 (ROVR) 位将置位。

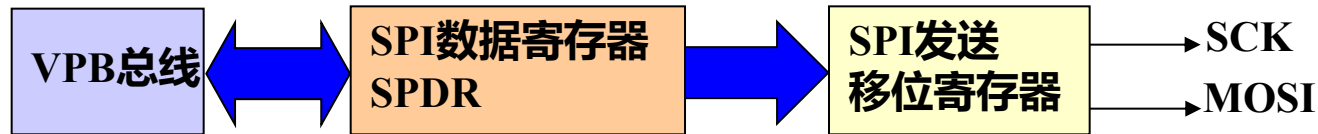


SPI主机模式接收数据示意图

4.12 SPI接口

■ 异常状况一写冲突

SPI总线接口与发送移位寄存器之间没有写缓冲区。只能在SPI总线空闲期间向SPI数据寄存器写入数据。启动传输到SPIF置位（包括读取状态寄存器）期间，不能向SPI数据寄存器写入数据。否则，新写入的数据将会丢失，状态寄存器中的写冲突位（WCOL）置位。



SPI主机模式发送数据示意图

4.12 SPI接口

■ 异常状况一模式错误

SPI主机接口的SSEL信号被外界拉低，引发模式错误，此时该主机的变化：

- 时钟驱动器被关闭；
- 主机模式变为从机模式；
- 中断标志置位。

如果要清除模式错误位 (MODE)，必须要先读取SPI状态寄存器，然后再重新初始化SPI控制寄存器。

4.12 SPI接口

■ 异常状况—从机中止

在从模式下，如果SSEL信号在传输结束之前变为高电平，从模式数据传输将被中止。正在传输的数据将丢失。

• SPI寄存器描述——SPI数据寄存器

SPDR寄存器为SPI提供数据的发送和接收。处于主模式时，向该寄存器写入数据，将启动SPI数据传输。从数据传输开始到SPIF状态位置位并且没有读取状态寄存器的这段时间内不能对该寄存器执行写操作。

SPDR	功能	描述	复位值
7:0	数据	SPI双向数据	0

• SPI寄存器描述——SPI时钟计数寄存器

作为主机时，SPCCR寄存器控制SCK的频率。寄存器的值为一位SCK时钟所占用的PCLK周期数。该寄存器的值必须为偶数，并且必须不小于8。如果寄存器的值不符合以上条件，可能会导致产生不可预测的动作。

$$\text{SPI速率} = F_{\text{pclk}} / \text{SPCCR}$$

SPCCR	功能	描述	复位值
7 : 0	计数值	设定SPI时钟计数值	0

• SPI寄存器描述——SPI中断寄存器

该寄存器包含SPI接口的中断标志。

SPCCR	功能	描述	复位值
0	SPI中断	SPI中断标志。向该位写入1清零。 注：当SPIE位置“1”，并且SPIF和MODF位种至少有一位为1时，该位置位。但是只有当SPI中断位置位并且SPI中断在VIC中被使能，SPI中断才能由中断处理程序处理。	0
7 : 1	保留	用户程序不要向这些位写入1	NA

引发SPI中断的事件：

- 数据传输完成；
- 发生模式错误。

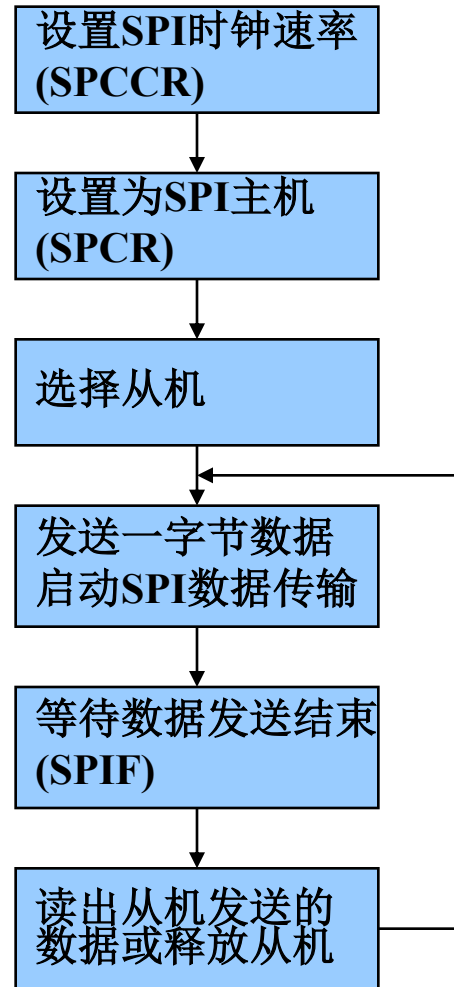
4.12 SPI接口

■ 使用SPI接口的注意要点

- 作主机时，SSEL引脚必须接上拉电阻，不能作为IO口使用；
- 作主机时，在发送一字节数据的同时接收一字节数据；
- SPI时钟分频值必须大于或等于8；
- 数据寄存器与内部移位寄存器之间没有缓冲区，写SPDR会使数据直接进入移位寄存器。因此数据只能在上一次数据发送完成后写入SPDR寄存器。

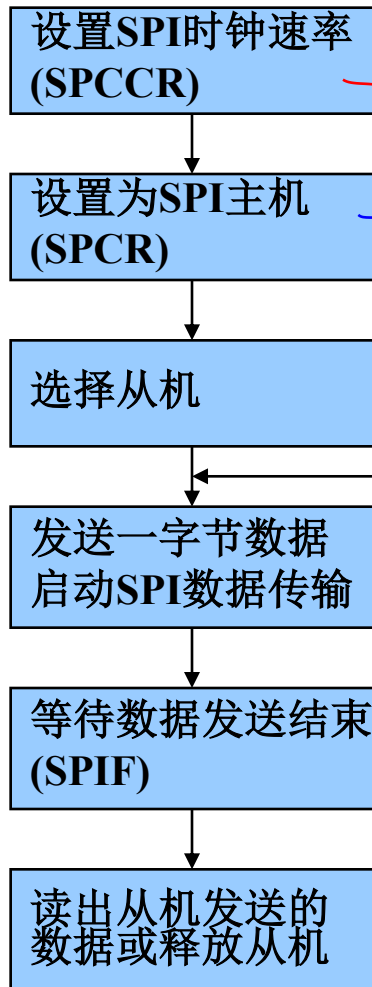
• SPI应用示例——作为主机

操作流程



SPI应用示例——作为主机

操作流程



操作流程I初始化代码:

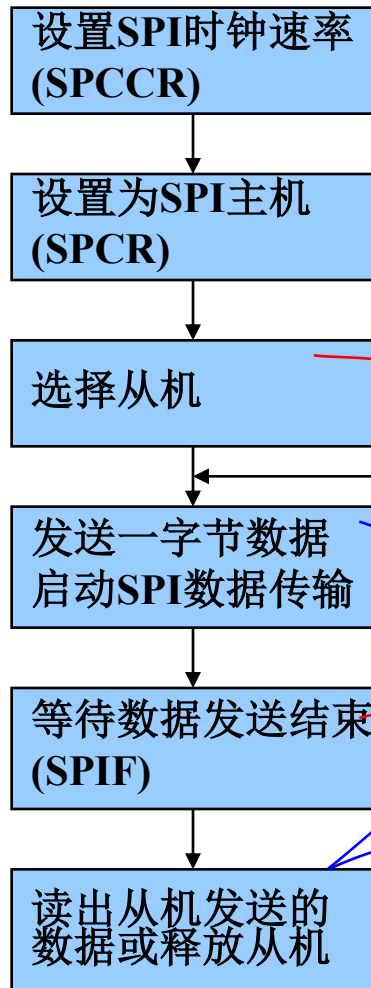
```
#define MSTR      (1 << 5)
#define CPOL      (1 << 4)
#define CPHA      (1 << 3)
#define LSBF      (1 << 6)
#define SPI_MODE (MSTR | CPOL)

void MSpiIni(uint8 fdiv)
{
    if(fdiv < 8)
        fdiv = 8;
    SOPCCR = fdiv & 0xFE;
    SPCR = SPI_MODE;
}
```

过滤分频值，如果
小于8为非法

SPI应用示例——作为主机

操作流程



SPI主机发送和接收程序：

```
#define MSTR (1 << 5)
#define CPOL (1 << 4)
#define CPHA (1 << 3)
#define LSBF (1 << 6)
#define SPI_MODE (MSTR | CPOL)
```

```
uint8 MSendData(uint8 data)
```

```
{
```

```
IO0CLR = HC595_CS;
```

```
SOPDR = data;
```

```
while(0 == (SOPSR & 0x80));
```

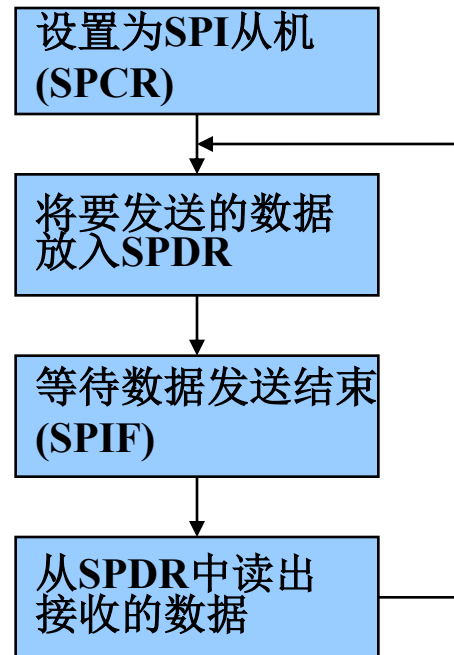
```
IO0SET = HC595_CS;
```

```
return(SOPDR);
```

```
}
```

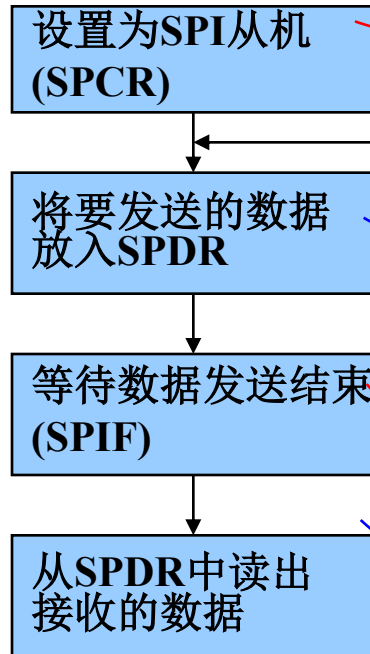
• SPI应用示例——作为从机

操作流程



SPI应用示例——作为从机

操作流程



操作代码

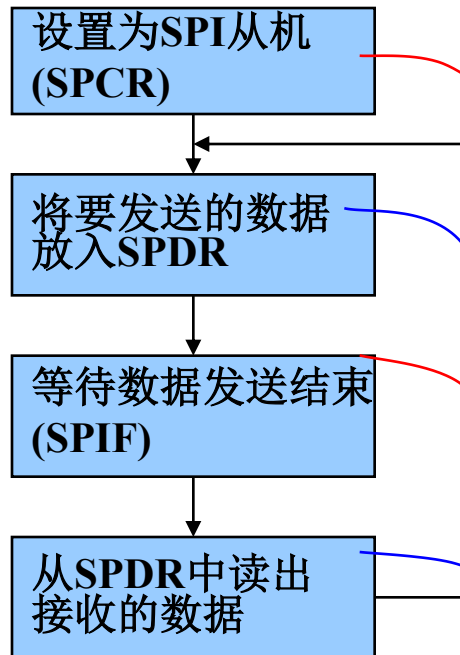
SPI初始化代码:

```
//初始化从机
void SSpiIni(uint8 fdiv)
{
    SPCR = (1 << 4);
}

//收发一字节数据
uint8 SSwapData(uint8 data)
{
    SPCR = data;
    while(0 == (SPSR & 0x80));
    return(SPCR);
}
```

SPI应用示例——作为从机

操作流程



SPI初始化代码:

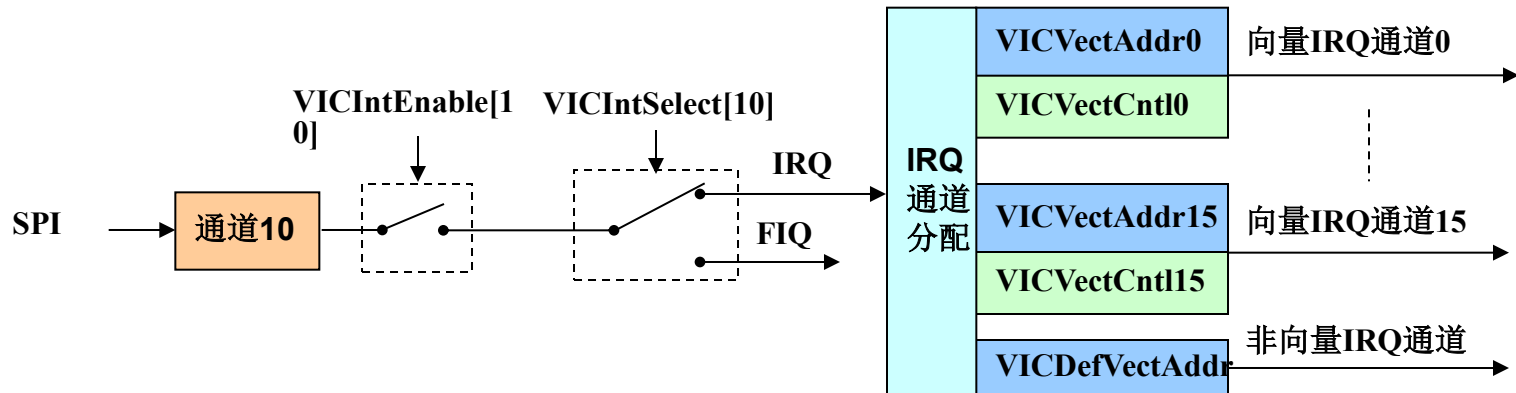
```
void SSpiIni(uint8 fdiv)
{
    SPCR = (1 << 4);
}

uint8 SSwapData(uint8 data)
{
    SOPDR = data;
    while(0 == (SOPSR & 0x80));
    return(SOPDR);
}
```

SPI中断

- SPI与VIC的关系

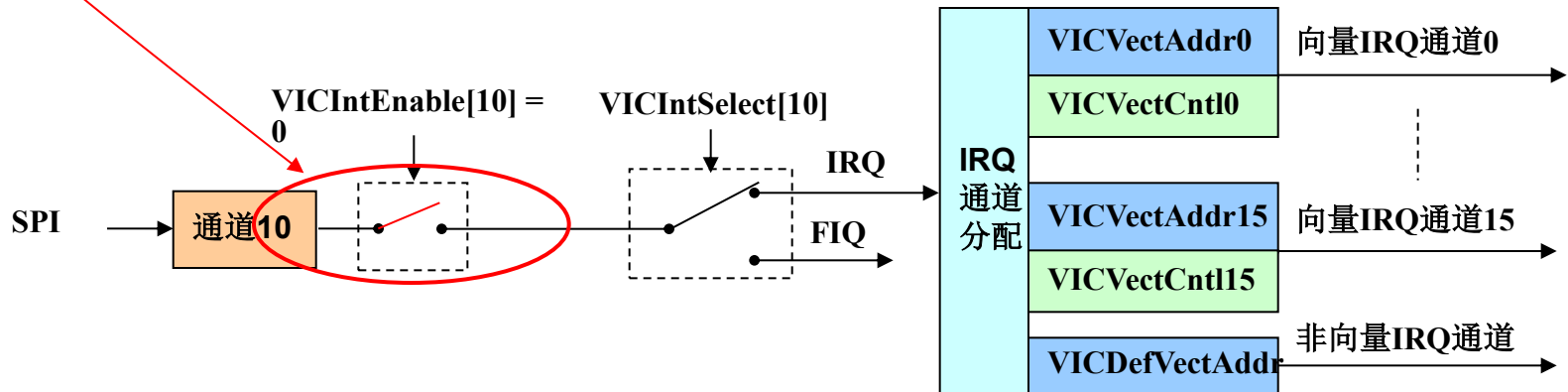
SPI分别位于VIC的通道10。中断使能寄存器VICIntEnable的Bit10用来控制通道10的使能。



SPI中断

- SPI与VIC的关系

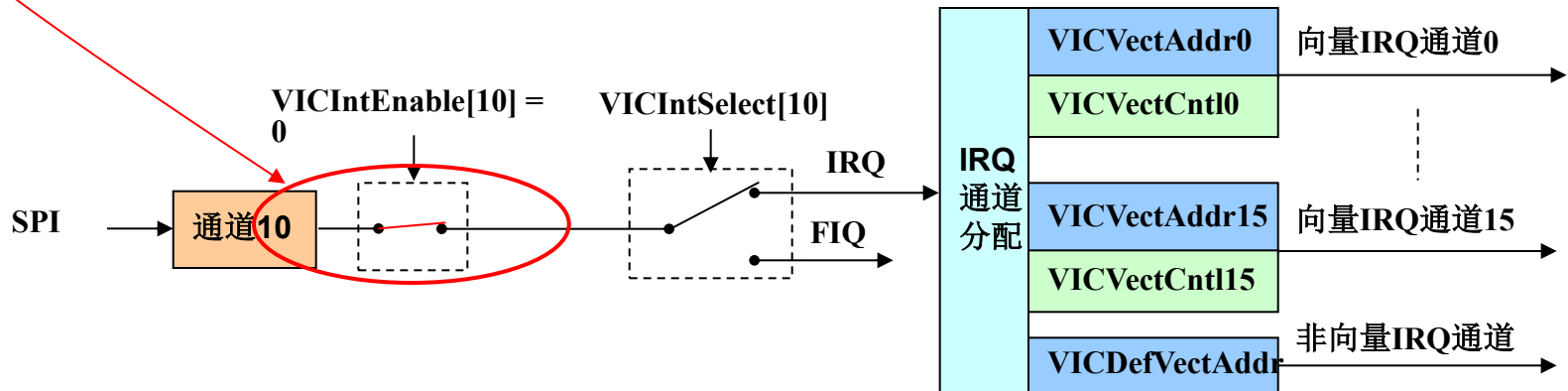
➤ 当 $VICIntEnable[10] = 0$ 时，通道10中断禁止；



SPI中断

• UART0与VIC的关系

- 当 $VICIntEnable[10] = 0$ 时，通道10中断禁止；
- 当 $VICIntEnable[10] = 1$ 时，通道10中断使能。





LPC2000系列ARM硬件结构

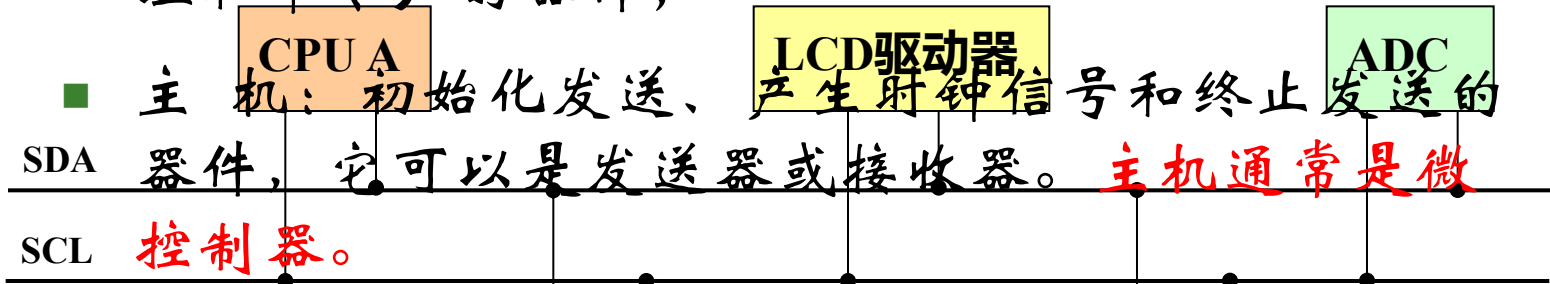
- 1.LPC2000系列简介
- 2.引脚描述
- 3.存储器寻址
- 4.系统控制模块
- 5.存储器加速模块 (MAM)
- 6.外部存储器控制器 (EMC)
- 7.引脚连接模块
- 8. GPIO
- 9. 向量中断控制器
- 10.外部中断输入
- 11.定时器0和定时器1
- 12. SPI接口
- 13. I²C接口
- 14. UART(0、1)
- 15. A/D转换器
- 16. 看门狗
- 17. 脉宽调制器(PWM)
- 18. 实时时钟

4.13 I²C接口

概述

■ I²C总线是Philips推出的一种串行数据传输总线，它仅用2根连线实现了低成本的全双工同步数据传送，可以极方便地构成多机系统和外围器件扩展系统。

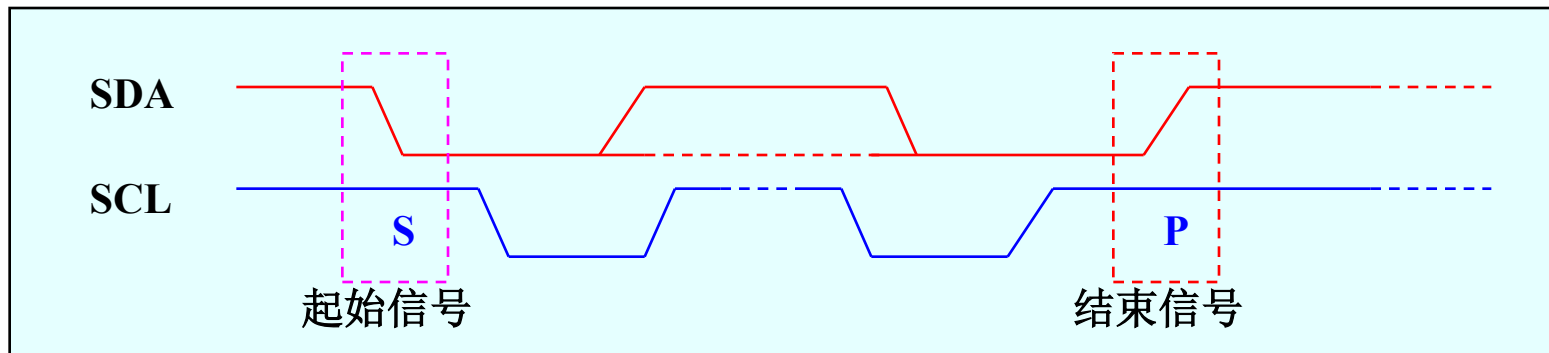
■ 接收器：本次传送中从总线接收数据（不包括地址和命令）的器件；



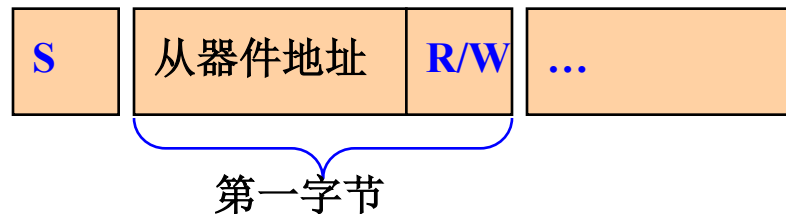
■ 从机：被主机寻址的器件，它可以是发送器或接收器。

总线时序

在数据传送过程中，必须确认数据传送的开始和结束，这通过起始和结束信号识别。



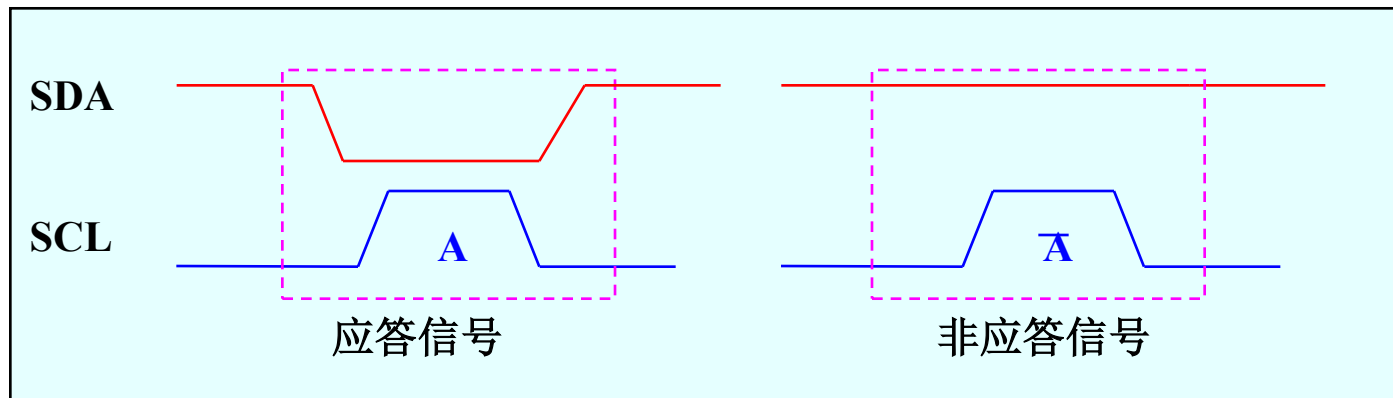
发送起始信号后传送的第一字节数据具有特别的意义，其中前七位为从机地址，最后一位为读写方向位（0表示写，1表示读）。



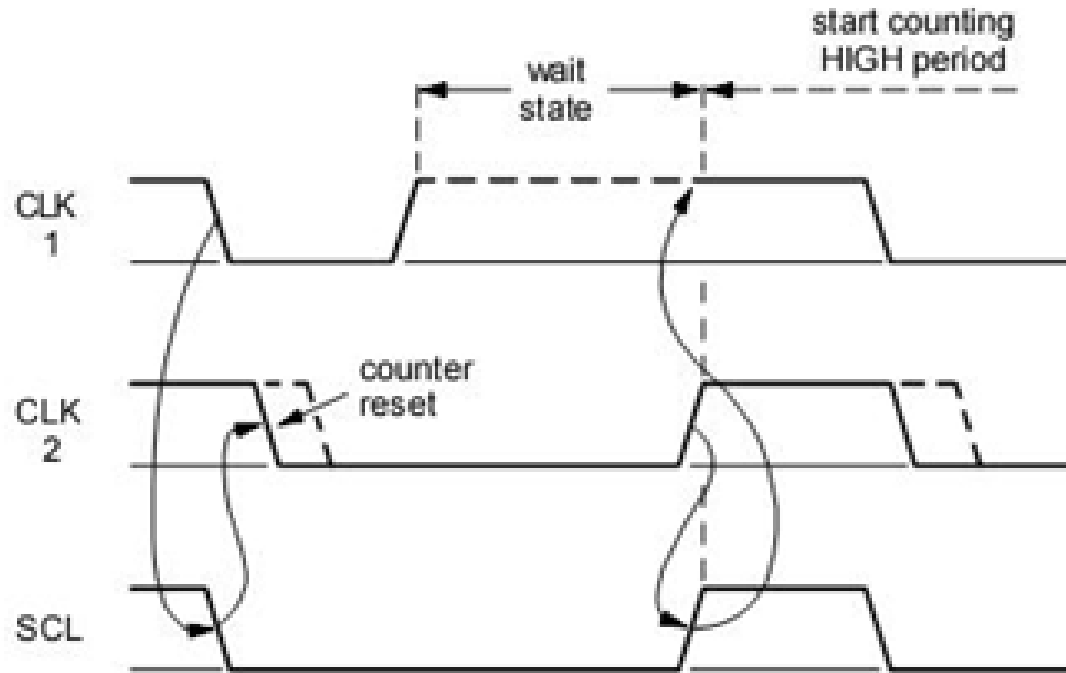
4.13 I²C接口

- 总线时序

I²C总线数据传输时，每传送一个字节数据后都必须有应答信号（A）。主控器接收数据时，如果要结束通信时，将在停止位之前发送非应答信号（ $\overline{A}$ ）。



■ SCL线的同步（时钟同步）



SDA仲裁

DA线的仲裁也是建立在总线具有线“与”逻辑功能的原理上的。节点在发送1位数据后，比较总线上所呈现的数据与自己发送的是否一致。是，继续发送；否则，退出竞争。**SDA**线的仲裁可以保证**I2C**总线系统在多个主节点同时企图控制总线时通信正常进行并且数据不丢失。总线系统通过仲裁只允许一个主节点可以继续占据总线

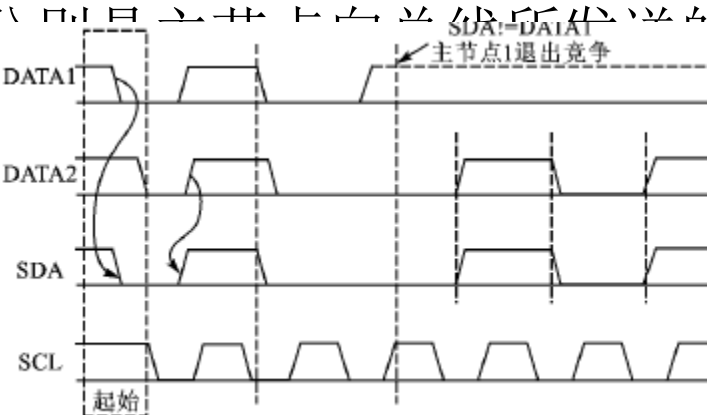
DATA1和**DATA2**分别是主节点向总线所发送的数据信号，**SDA**为总线上所呈现的数据信号，**SCL**是总线上所呈现的时钟信号。当主节点1、2同时发送起始信号时，两个主节点都发送了高电平信号。这时总线上呈现的信号为高电平，两个主节点都检测到总线上的信号与自己发送的信号相同，继续发送数据。第2个时钟周期，2个主节点都发送低电平信号，在总线上呈现的信号为低电平，仍继续发送数据。在第3个时钟周期，主节点1发送高电平信号，而主节点2发送低电平信号。根据总线的线“与”的逻辑功能，总线上的信号为低电平，这时主节点1检测到总线上的数据和自己所发送的数据不一样，就断开数据的输出级，转为从机接收状态。这样主节点2就赢得了总线，而且数据没有丢失，即总线的数据与主节点2所发送的数据一样，而主节点1在转为从节点后继续接收数据，同样也没有丢掉**SDA**线上的数据。因此在仲裁过程中数据没有丢失。

SDA仲裁

DA线的仲裁也是建立在总线具有线“与”逻辑功能的原理上的。节点在发送1位数据后，比较总线上所呈现的数据与自己发送的是否一致。是，继续发送；否则，退出竞争。**SDA**线的仲裁可以保证**I2C**总线系统在多个主节点同时企图控制总线时通信正常进行并且数据不丢失。总线系统通过仲裁只允许一个主节点可以继续占据总线

DATA1和DATA2

线上所呈现的数据
1、2同时发送起始
线上呈现的信号为
发送的信号相同，
低电平信号，在总

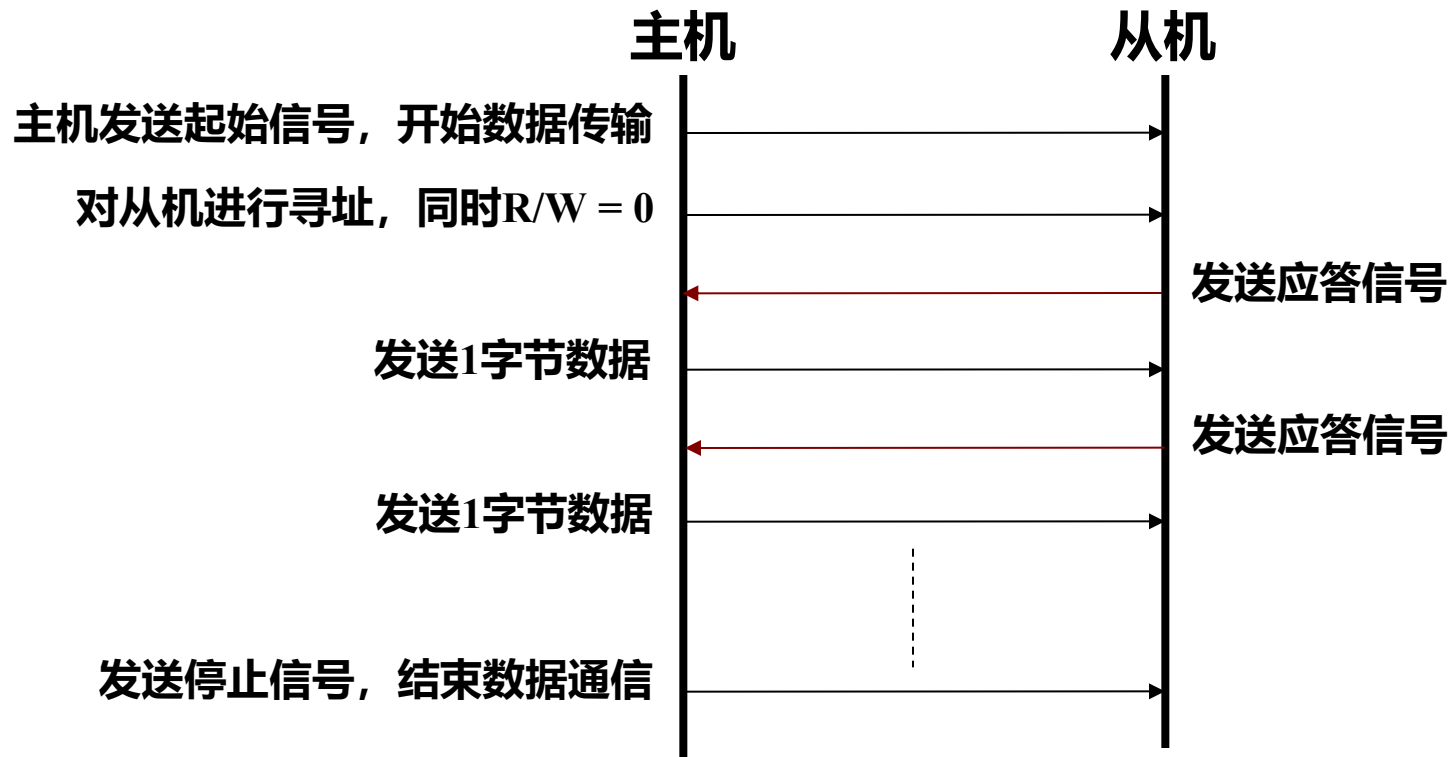


数据信号，**SDA**为总
的时钟信号。当主节点
了高电平信号。这时总
到总线上的信号与自己
期，2个主节点都发送
与继续发送数据。在第

3个时钟周期，主节点1发送高电平信号，而主节点2发送低电平信号。根据总线的线“与”的逻辑功能，总线上的信号为低电平，这时主节点1检测到总线上的数据和自己所发送的数据不一样，就断开数据的输出级，转为从机接收状态。这样主节点2就赢得了总线，而且数据没有丢失，即总线的数据与主节点2所发送的数据一样，而主节点1在转为从节点后继续接收数据，同样也没有丢掉**SDA**线上的数据。因此在仲裁过程中数据没有丢失。

I²C总线规范——传输协议

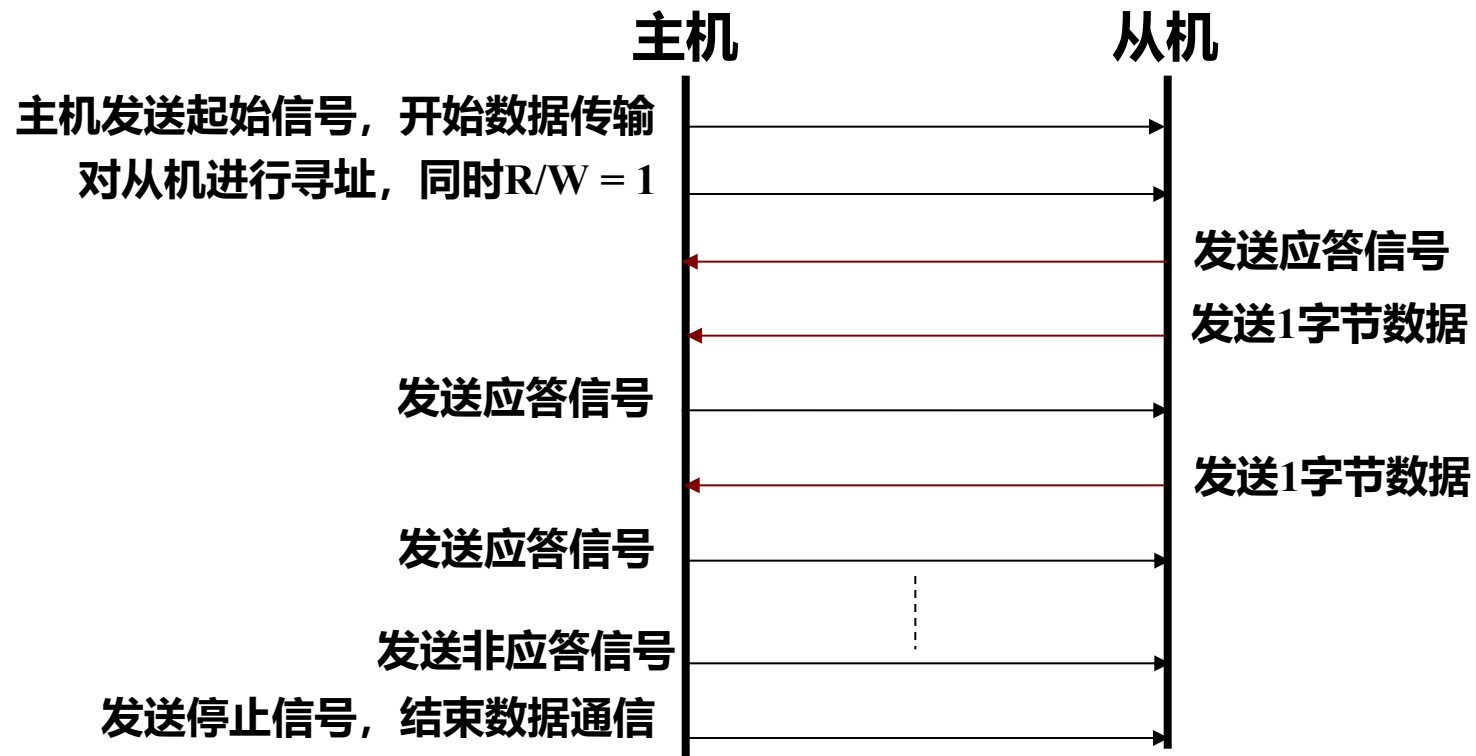
■ 主机发送数据到从机



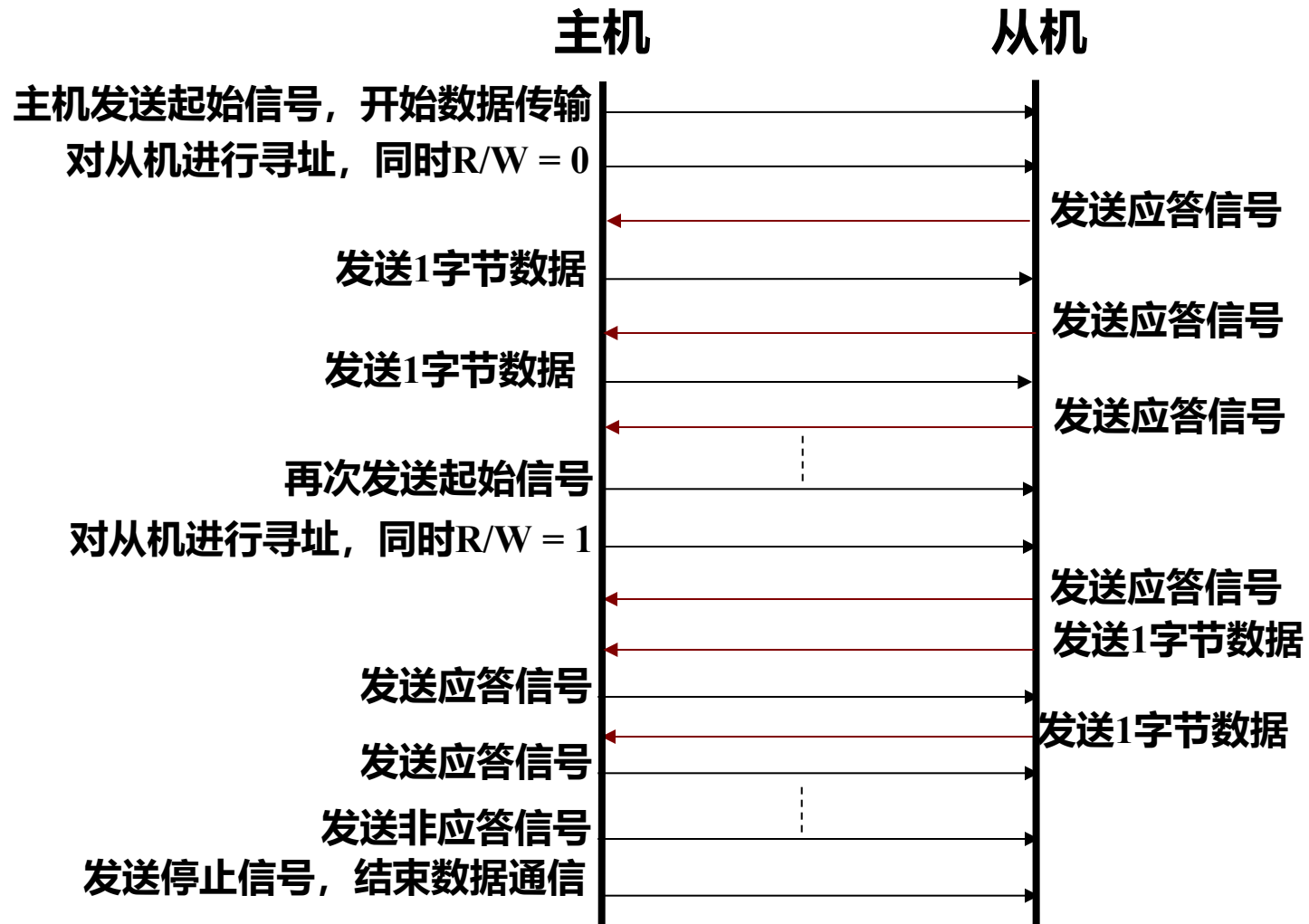


I²C总线规范——传输协议

■ 主机读取从机数据



■ 复合格式



4.13 I²C接口

■ 寄存器描述

I²C接口包含7个寄存器。

名称	描述	访问	复位值	地址
I2CONSET	I ² C控制置位寄存器	读/置位	0	0xE001C000
I2STAT	I ² C状态寄存器	只读	0xF8	0xE001C004
I2DAT	I ² C数据寄存器	读/写	0	0xE001C008
I2ADR	I ² C从地址寄存器	读/写	0	0xE001C00C
I2SCLH	SCL占空比寄存器高半字	读/写	0x04	0xE001C010
I2SCLL	SCL占空比寄存器低半字	读/写	0x04	0xE001C014
I2CONCLR	I ² C控制清零寄存器	只清零	NA	0xE001C018

• 寄存器描述——I²C控制置位寄存器

I2CONSET寄存器用于置位I²C通信的相关标志位，该寄存器只能对某位置位，而不能清零，清零通过I2CONCLR寄存器完成。

位	功能	描述	复位值
1:0	保留	用户程序不要向这些位写入1	NA
2	AA	应答标志	0
3	SI	I2C中断标志	0
4	STO	停止标志	0
5	STA	起始标志	0
6	I2EN	I2C接口使能	0
7	保留	用户程序不要向该位写入1	NA

• 寄存器描述——I²C控制清零寄存器

I2CONCLR寄存器与I2CONSET寄存器的功能相反，它用于清零I²C通信的相关标志位，该寄存器只能对某位清零，而不能置位。

位	功能	描述	复位值
1:0	保留	用户程序不要向这些位写入1	NA
2	AA	应答标志	NA
3	SI	I ² C中断标志	NA
4	STO	停止标志	NA
5	STA	起始标志	NA
6	I2EN	I ² C接口使能	NA
7	保留	用户程序不要向该位写入1	NA

• 寄存器描述——I²C状态寄存器

I2STAT寄存器包含了I²C接口的状态代码，它是一个只读寄存器。一共有26种可能存在的状态代码。当代码为0xF8时，无可用的相关信息，SI位不会置位。所有其它25种状态代码都对应一个已定义的I²C状态。当进入其中一种状态时，SI位将置位。

I²C处理程序就是根据该寄存器反映的状态来进行相应的处理。

位	功能	描述	复位值
2 : 0	状态	这3个位总是为0	0
7 : 3	状态	状态位	1

• 寄存器描述——I²C数据寄存器

I2DAT寄存器包含要发送或刚接收的数据。当它没有处理字节的移位时，CPU可对其进行读写。该寄存器只能在SI置位时访问。在SI置位期间，I2DAT中的数据保持稳定。I2DAT中的数据移位总是从右至左进行：第一个发送的位是MSB（位7），在接收字节时，第一个接收到的位存放在I2DAT的MSB。

位	功能	描述	复位值
7 : 0	数据	发送/接收数据位	0

• 寄存器描述——I²C从地址寄存器

I2ADR寄存器只能在I²C设置为从模式时才能使用。在主模式中，该寄存器无效。I2ADR的LSB为通用调用位。当该位置位时，通用调用位（0x00）被识别（即可以对广播地址0x00作出响应）。

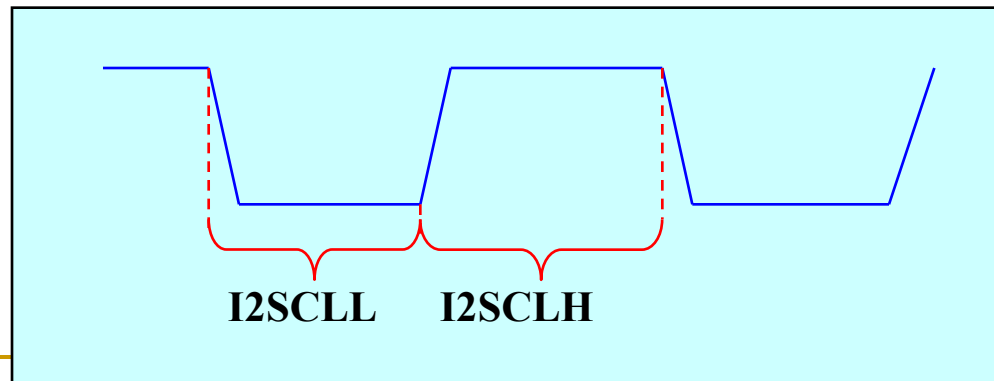
位	功能	描述	复位值
0	GC	通用调用位	0
7 : 1	地址	从模式地址	0

• 寄存器描述——I²C 占空比寄存器

I2SCLH、I2SCLL寄存器用于控制I²C通信的波特率。其中I2SCLH定义SCL高电平所保持的PCLK周期数，而I2SCLL定义SCL低电平所保持的PCLK周期数。那么位频率（即总线速率）由下面的公式得出：

$$\text{位频率} = F_{\text{pclk}} / (I2SCLH + I2SCLL)$$

寄存器	功能	描述	复位值
I2SCLH	计数值	SCL高电平周期占用PCLK周期数	0x04
I2SCLL	计数值	SCL低电平周期占用PCLK周期数	0x04



4.13 I²C接口

■ 使用I²C接口的注意要点

- I²C接口的引脚为开漏输出，必须在I²C总线上接上拉电阻。通信速率越快，电阻值越小；
- 总线上各器件的地址不能冲突；
- 编程时需要仔细处理每个状态，注意各状态之间的转移关系。



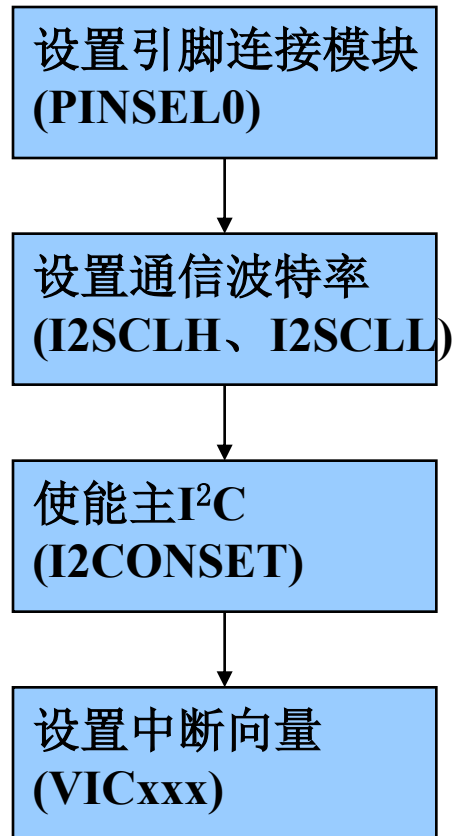
4.13 I²C接口

■ I²C应用示例

LPC2000对于I²C通信的处理是基于状态标志进行的，不同的模式之间具有相通的分析方法，这里仅介绍常用的主发送和主接收模式。

• I²C应用示例——主模式设置

主模式初始化流程



I²C应用示例——主模式设置

初始化代码

主模式初始化流程

主模式初始化流程

设置引脚连接模块
(PINSEL0)

设置通信波特率
(I2SCLH、I2SCLL)

使能主I²C
(I2CONSET)

设置中断向量
(VICxxx)

```
Void I2C_init(uint32 fi2c)
```

```
{
```

```
    if(fi2c > 400000)
```

```
        fi2c = 400000;
```

```
    PINSEL0 = (PINSEL0 & 0xFFFFF0F) | 0x50;
```

```
    I2SCLH = (Fpclk / fi2c + 1) / 2;
```

```
    I2SCLL = (Fpclk / fi2c) / 2;
```

```
    I2CONCLR = 0x2C;
```

```
    I2CONSET = 0x40;
```

```
    VICIntSelect = 0x00000000;
```

```
    VICVectCntl0 = 0x29;
```

```
    VICVectAddr0 = (int)IRQ_I2C;
```

```
    VICIntEnable = 0x0200;
```

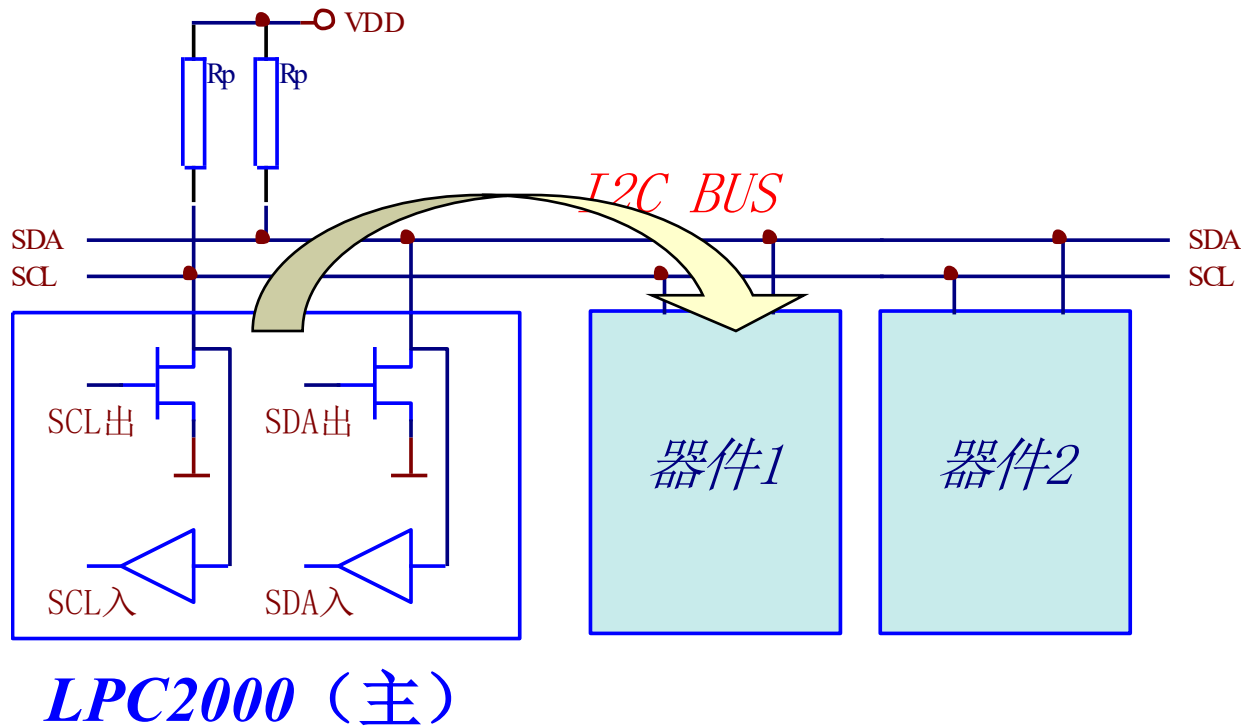
```
}
```

传入参数为I²C时钟频率

过滤传入参数，最高
400K时钟频率

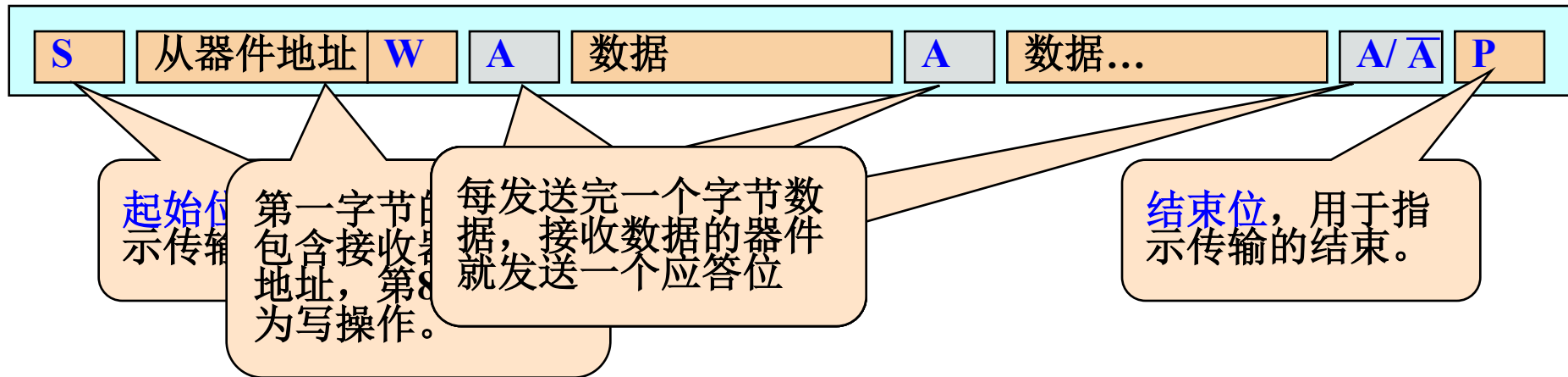
• I²C应用示例——主机发送

LPC2000在该模式下作为主控器，向从机发送数据。数据流向如下图所示：



I²C应用示例——主机发送

主模式数据发送的时序格式



I²C应用示例——主机发送

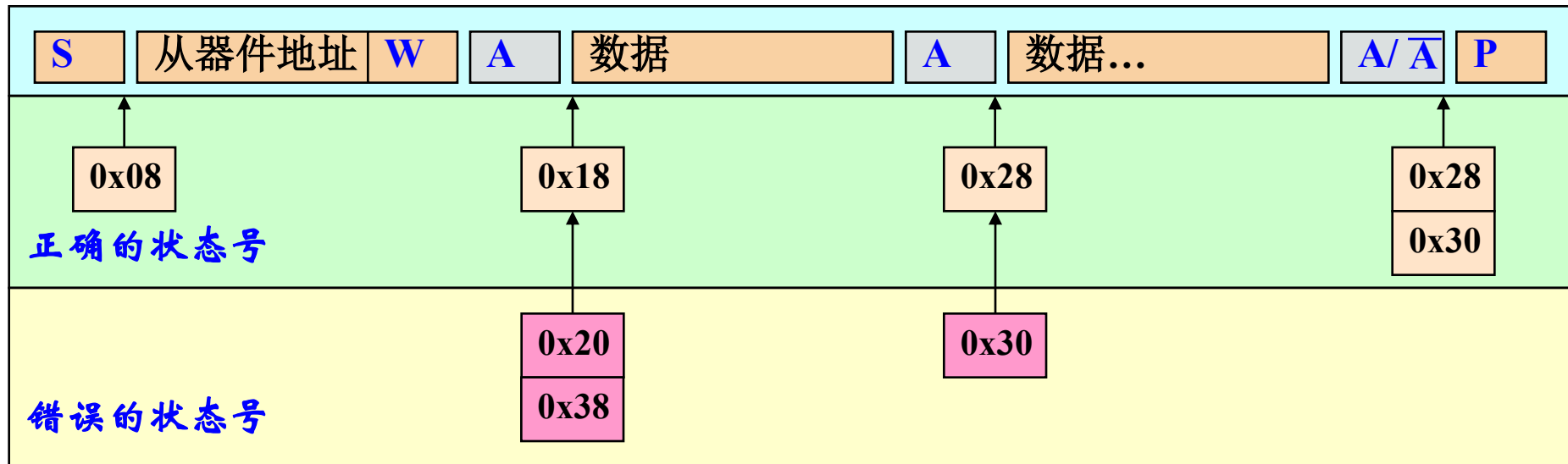
主模式数据发送的时序格式



- (1) 通过软件置位STA进入I²C主发送模式，I²C逻辑在总线空闲后立即发送一个起始信号；
- (2) 在起始信号发送结束后，SI置位。将从机地址和写操作位装入I2DAT，然后清零SI，将第一字节数据发出；
- (3) 当从机地址和W位发送结束并收到应答位(A)后，SI位再次置位。此时将要发送的数据装入I2DAT，开始发送数据；
- (4) 在数据正确发送后，SI置位。此时如果要结束本次操作，那么置位STO位，发送结束信号。

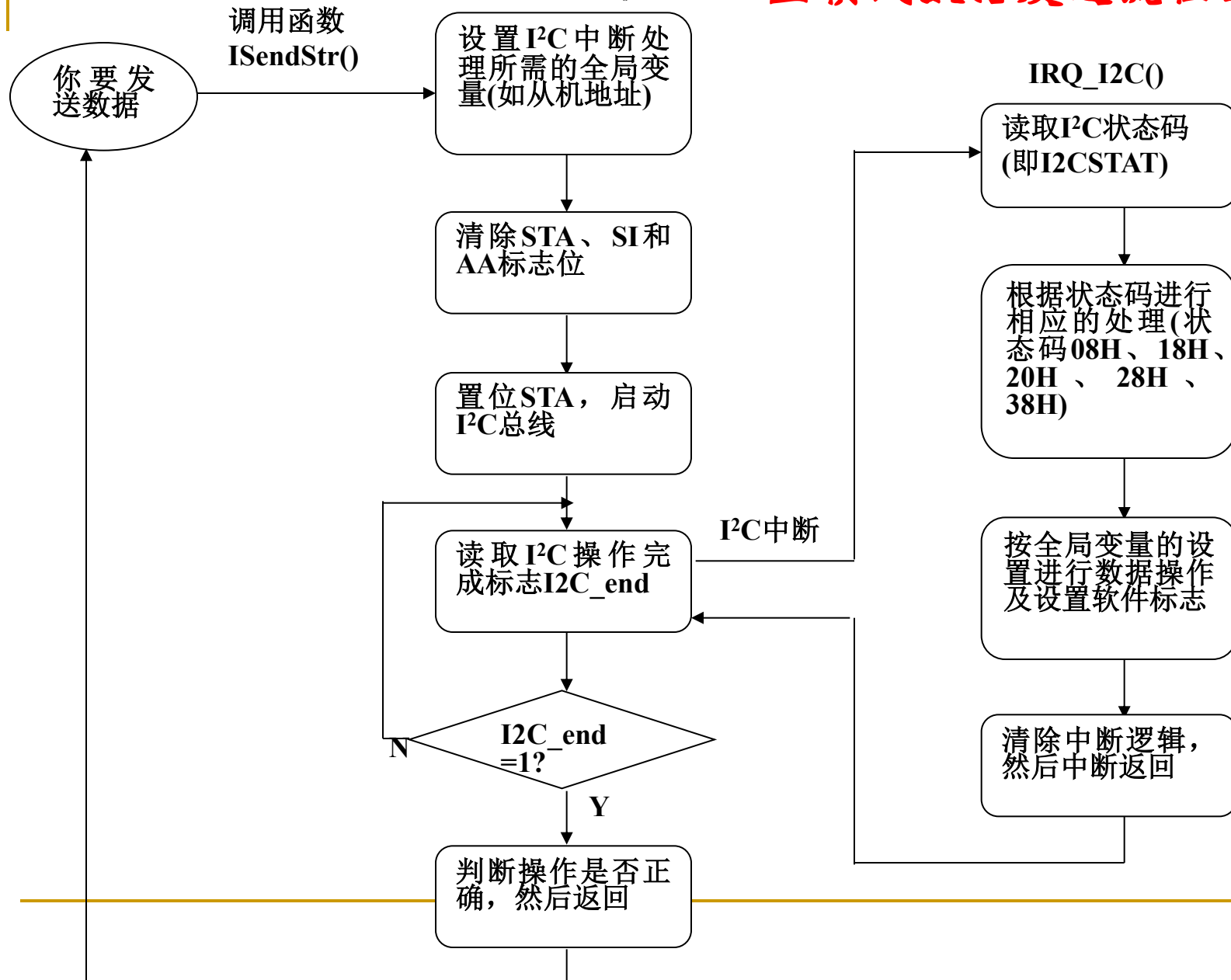
I²C应用示例——主机发送

主模式数据发送的时序格式



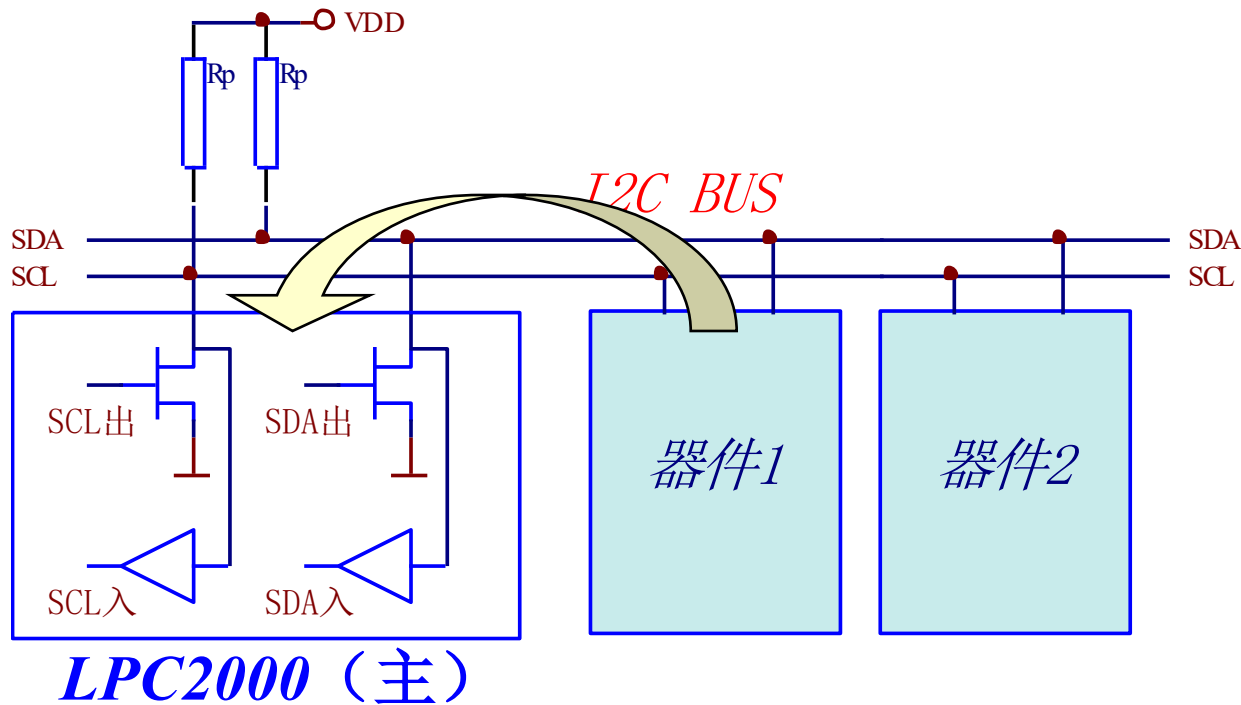
在通信过程中，随着通信阶段的不同，I2STAT寄存器中的状态号也相应的变化，并引起中断。在中断服务程序中，根据当前的状态号来决定下一步的处理。如果当前的状态号不符合正常操作的流程，那么就要作出相应的错误处理，比如重新启动总线等。

主模式数据发送流程图



• I²C应用示例——主机接收

LPC2000在该模式下作为主控器，接收从机发出的数据。数据流向如下图所示：



I²C应用示例——主机接收

主模式数据接收的时序格式

主接收模式:



起始信号

第一字节的前七位包含接收器件的从地址，第8位为1，表示读操作。

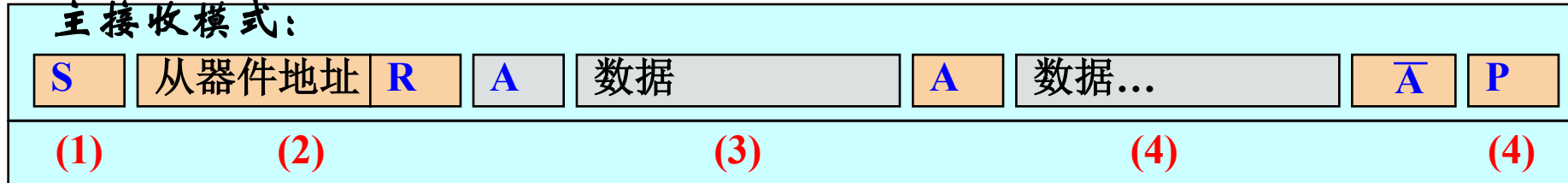
每接收完一个字节数据，就发送一个应答位。在接收最后一个字节数据后，发送非应答位，通知从机停止发送数据。

结束信号

I²C应用示例——主机接收

主模式数据接收的时序格式

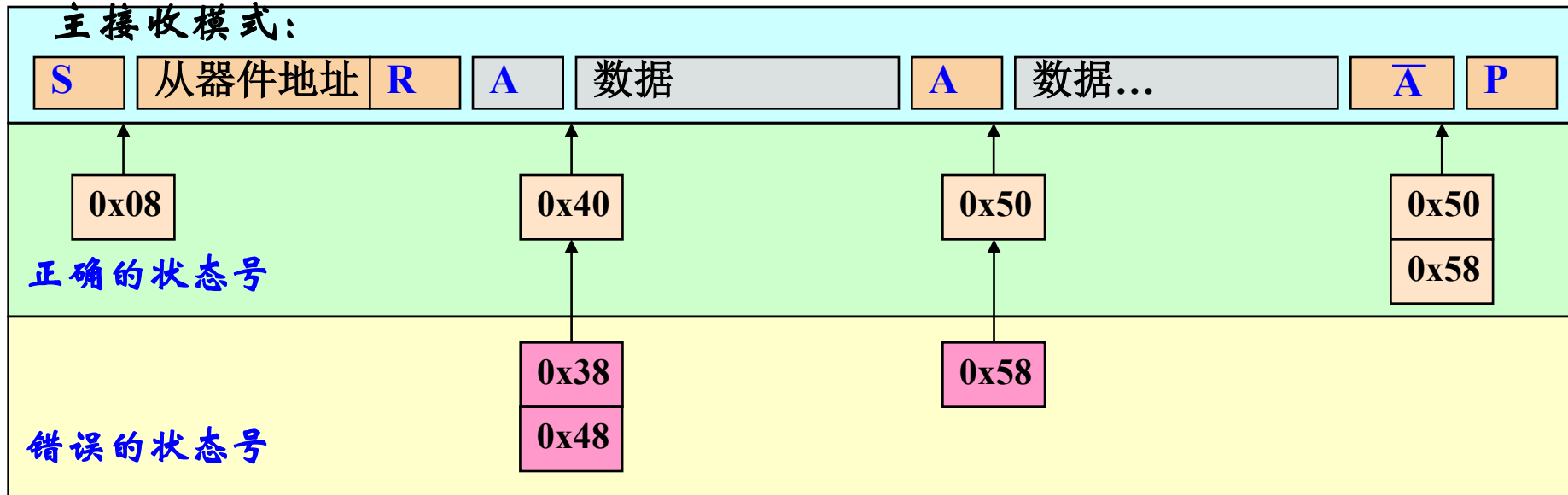
主接收模式:



- (1) 通过软件置位STA进入I²C主发送模式，I²C逻辑在总线空闲后立即发送一个起始信号；
- (2) 在起始信号发送结束后，SI置位。将从机地址和读操作位装入I2DAT，然后清零SI，将第一字节数据发出；
- (3) 当从机地址和R位发送结束并收到应答位(A)后，SI位再次置位。此时设置AA位，然后清零SI位，开始接收数据；
- (4) 每接收到一字节数据，SI位再次置位，此时可以再次接收数据，或者置位STO结束总线。

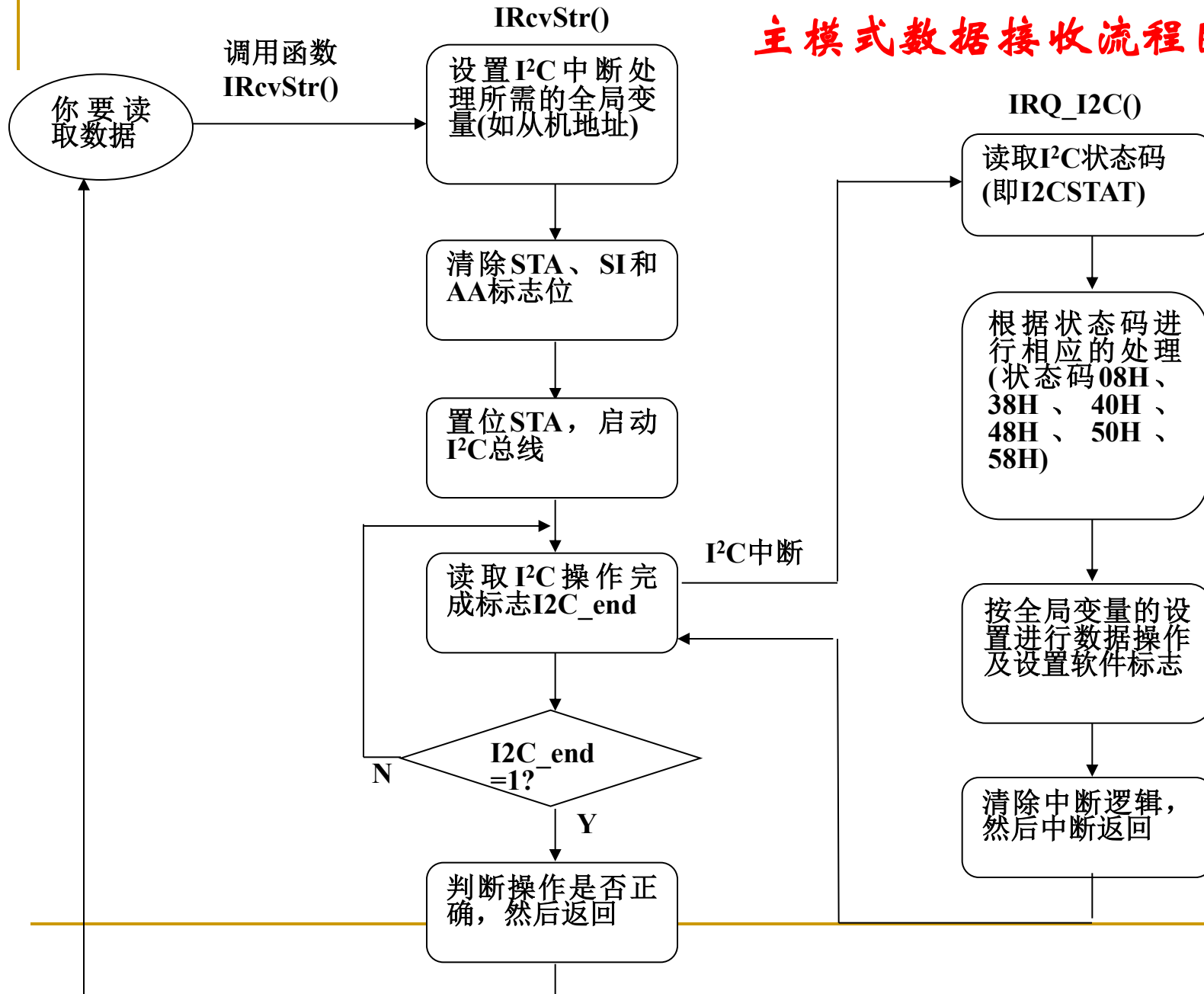
I²C应用示例——主机接收

主模式数据接收的时序格式



在通信过程中，随着通信阶段的不同，I2STAT寄存器中的状态号也相应的变化，并引起中断。在中断服务程序中，根据当前的状态号来决定下一步的处理。如果当前的状态号不符合正常操作的流程，那么就要作出相应的错误处理，比如重新启动总线等。

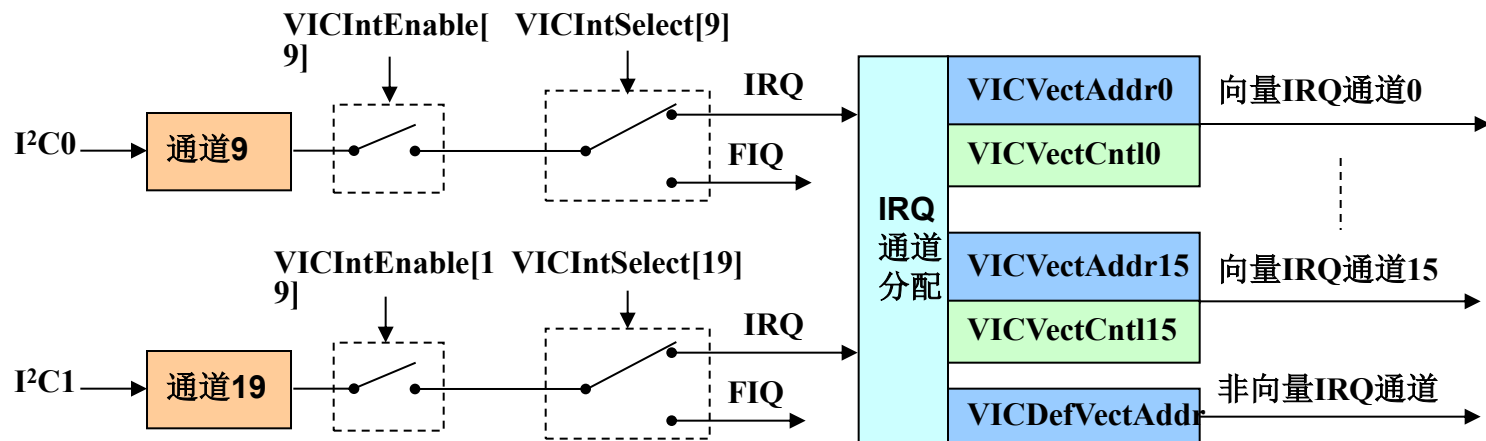
主模式数据接收流程图



I²C中断

- I²C与VIC的关系

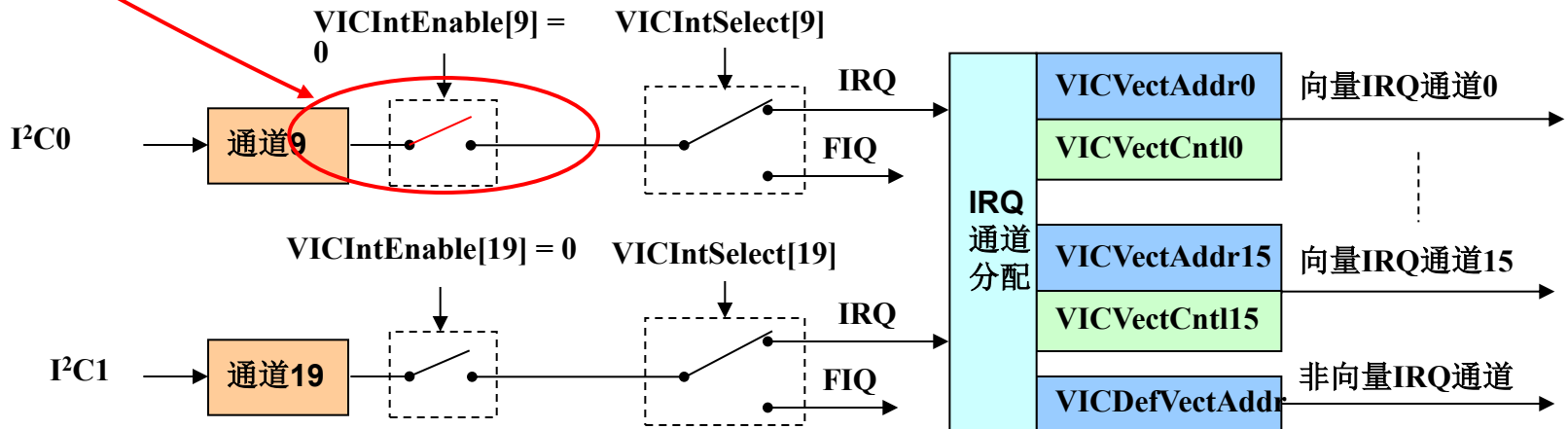
I²C0、I²C1分别位于VIC的通道9和通道19。中断使能寄存器VICIntEnable的Bit9和Bit19分别用来控制通道9和通道19的使能。



I²C中断

• UART0与VIC的关系

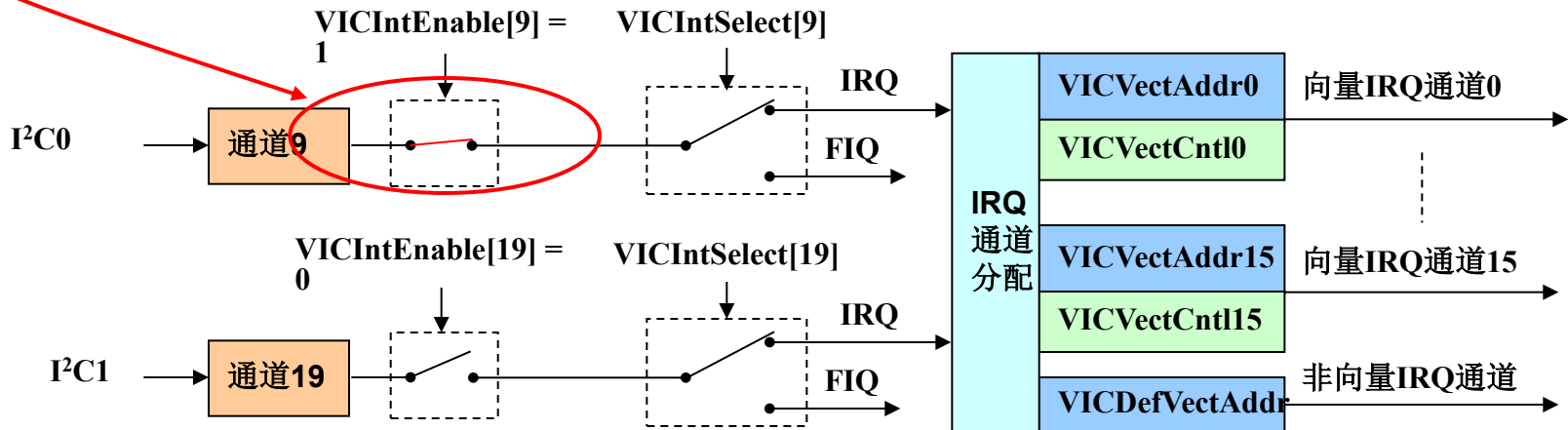
➤ 当VICIntEnable[9] = 0时，通道9中断禁止；



I²C中断

• UART0与VIC的关系

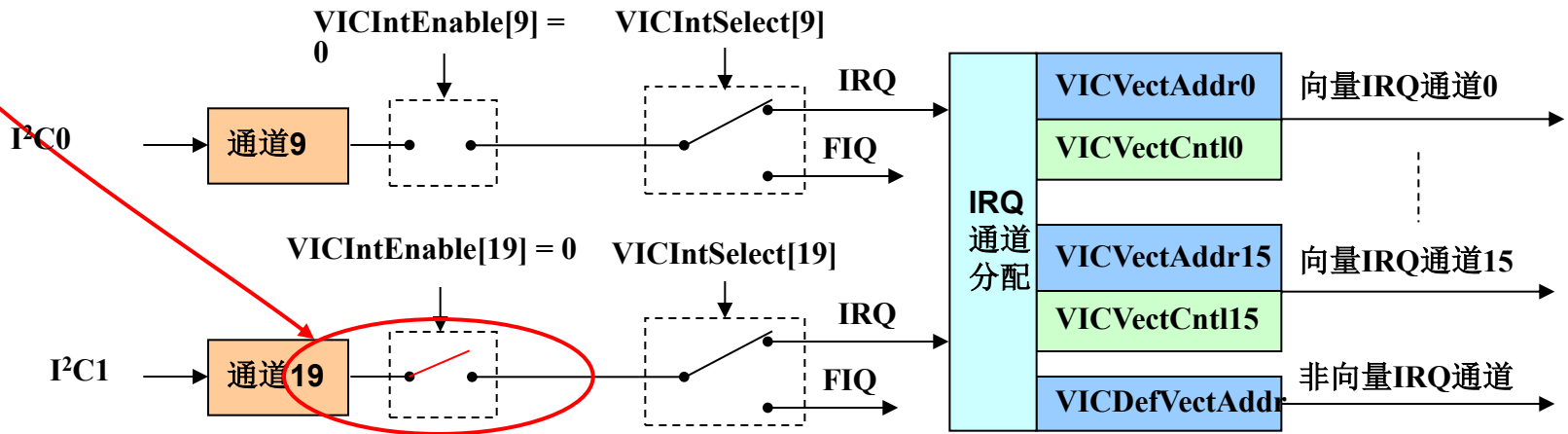
- 当VICIntEnable[9] = 0时，通道9中断禁止；
- 当VICIntEnable[9] = 1时，通道9中断使能。



I²C中断

• UART1与VIC的关系

➤ 当VICIntEnable[19] = 0时，通道19中断禁止；



I²C中断

• UART1与VIC的关系

- 当VICIntEnable[19] = 0时，通道19中断禁止；
- 当VICIntEnable[19] = 1时，通道19中断使能。

